2026 중간고사 · Node.js & SQL & MongoDB

The Family 통합 정보시스템

기술 보고서

학번	202600000
이름	(이름 기입)
제출 경로	~/midterm (실행: `node app.js`)
구현 규모	REST API 221개 · MySQL 테이블 20개 · MongoDB 컬렉션 19개 · 웹 화면(EJS) 21개
검증	요구사항 58개 · 확인 절차 207단계 자동 검증 통과 (scripts/verify-report.js)

목차

1. 개요
2. 실행 및 테스트 환경
3. 시스템 구조 · 웹 화면 · 운영 보안
4. 역할 · 권한 설계
5. 상태 체계
6. 데이터베이스 설계
7. 업무 규칙 구현
8. API 공통 규격
9. 요구사항 추적표
10. 기능 확인 방법 (테스트 절차)
11. 전체 API 명세 (221개)

1. 개요

The Family 는 여러 지역의 Crew 와 구성원을 운영하는 조직이다. 구성원 정보·Crew 소속·업무 배정·공용 자원·활동/평가 기록이 분산 관리되어 발생하는 정보 불일치, 중복 배정, 자원 기록 누락, 권한 오남용, 이력 추적 문제를 해결하기 위해 하나의 웹 기반 통합 정보시스템을 설계·구현하였다.

설계 원칙

- **요구사항 중심 설계**: 모든 API 는 정의 시점에 연결 요구사항(reqs)을 명시하며, 같은 정의로부터 라우팅·입력 검증·권한 검사·Swagger 문서·요구사항 추적표가 자동 생성된다.
- **서버 측 규칙 보장**: 권한(역할별 권한 코드 + 관리 범위)과 업무 규칙(BR-01~10)을 서버의 서비스 계층에서 검사하고, 동시 요청은 DB 행 잠금(SELECT … FOR UPDATE)과 UNIQUE 제약으로 보장한다.
- **데이터 특성별 DB 분리**: 관계·일관성이 중요한 데이터는 MySQL, 시간에 따라 계속 증가하거나 구조가 유연한 이력·기록 데이터는 MongoDB 에 저장한다.
- **재현 가능한 테스트**: 최초 실행 시 데모 데이터가 자동 생성되며, 데모 초기화 API 로 언제든 동일한 상태에서 본 보고서의 확인 절차를 반복할 수 있다.

기술 스택

구분	사용 기술
서버	Node.js, Express 4, EJS (서버 사이드 렌더링)
DB	MySQL 8 (mysql2), MongoDB (공식 드라이버 6.x)
인증	JWT (jsonwebtoken), bcryptjs 비밀번호 해시, 토큰 폐기 목록 + 토큰 버전
문서화	OpenAPI 3.0 자동 생성 + swagger-ui-express (/api-docs)

API 구성

Method	POST	PUT	GET	PATCH	DELETE	합계
개수	56	8	106	26	25	**221**

모듈 (Swagger 태그)	API 수	모듈	API 수
01. Auth	7	02. Me (내 정보)	17
03. Roles & Permissions	7	04. Members	19
05. Promotion Requests	6	06. Discipline	4
07. Crews	16	08. Regions	6
09. Crew Join Requests	6	10. Crew Announcements	5
11. Tasks	19	12. Task Assignments	6
13. Task Comments & Checklist	8	14. Task Templates	6
15. Meetings	9	16. Resources	10
17. Resource Types	5	18. Resource Allocations	7

19. Resource Requests	6	20. Resource Issues	4
21. Evaluations & Reputation	12	22. Activities	3
23. Search	2	24. Statistics	11
25. Audit Logs	4	26. Admin	14
27. System	2		

2. 실행 및 테스트 환경

2.1 실행

```
cd ~/midterm
node app.js
```

실행 시 MySQL 데이터베이스·테이블과 MongoDB 인덱스를 자동 생성하고, DB 가 비어 있으면 데모 데이터를 자동 입력한다. 콘솔에 다음이 출력된다.

```
=== The Family 통합 정보시스템 (학번 202600000) ===
  웹 화면    : http://localhost:3000/
  Swagger UI : http://localhost:3000/api-docs
  등록된 REST API : 221개
```

2.2 설정 (config/index.js)

항목	값 (기본)	환경변수
학번	202600000	STUDENT_ID
MySQL	127.0.0.1:3306 / DB **family_202600000**	MYSQL_HOST, MYSQL_PORT, MYSQL_USER, MYSQL_PASSWORD
MongoDB	mongodb://127.0.0.1:27017 / DB **family_log_202600000**	MONGO_URI
포트	3000	PORT

2.3 데모 계정 (비밀번호 모두 `pass1234`)

ID	아이디	역할	상태	소속 / 비고
1	admin	ADMINISTRATOR	ACTIVE	시스템 관리자
2	boss	BOSS	ACTIVE	조직 전체
3	underboss1	UNDERBOSS	ACTIVE	Brooklyn·Harbor 감독
4	underboss2	UNDERBOSS	ACTIVE	Uptown 감독
5	capo1	CAPO	ACTIVE	Brooklyn Crew 책임자
6	capo2	CAPO	ACTIVE	Harbor Crew 책임자
7	capo3	CAPO	ACTIVE	Uptown Crew 책임자
8	soldier1	SOLDIER	ACTIVE	Brooklyn
9	soldier2	SOLDIER	ACTIVE	Brooklyn (경고 1회)
10	soldier3	SOLDIER	ACTIVE	Harbor
11	soldier4	SOLDIER	ACTIVE	Harbor
12	soldier5	SOLDIER	ACTIVE	Uptown

13	soldier6	SOLDIER	ACTIVE	Uptown
14	associate1	ASSOCIATE	ACTIVE	Brooklyn (승급 요청 대상)
15	associate2	ASSOCIATE	ACTIVE	Brooklyn·Harbor 준소속
16	suspended1	SOLDIER	SUSPENDED	Brooklyn (정지 징계)
17	standby1	SOLDIER	STANDBY	Harbor
18	withdrawn1	SOLDIER	WITHDRAWN	탈퇴 (로그인 불가)
19	pending1	ASSOCIATE	PENDING	무소속, Uptown 가입 요청 중

2.4 주요 데모 데이터

대상	내용
Crew	1 Brooklyn Crew (Capo capo1, 감독 underboss1) · 2 Harbor Crew (Capo capo2, 감독 underboss1) · 3 Uptown Crew (Capo capo3, 감독 underboss2)
업무	1 물류 창고 정기 점검 (Brooklyn, 내일 09~12시, soldier1·soldier2 배정, VH-001 예약) · 2 거래처 미팅 호위 (Brooklyn, 내일 10~11시, 1번과 시간 겹침, 미배정) 3 항만 화물 인수 (Harbor, 진행중, soldier3·soldier4) · 4 월간 장부 정리 (완료) · 5 신규 거점 답사 (Uptown) · 6 야간 순찰 (동시 수행 가능) · 7 완료 · 8 취소 · 9 보류(ON_HOLD) · 10~24 과거 완료 업무(통계용)
자원	1 VH-001 검정 세단(정상, Brooklyn) · 2 VH-002 화물 밴(사용중) · 3 VH-003 구형 트럭(정비중) · 4 WH-001 창고(등급3) · 5 CM-001 무전기(정상) · 6 CM-002 무전기(사용불가) · 7 SH-001 은신처(등급3) · 8 EQ-001 금고 장비(점검필요) · 9 VH-004 오토바이(Uptown)
요청	승급 요청 #1 (associate1 → SOLDIER, capo1 요청) · 가입 요청 #1 (pending1 → Uptown) · 자원 사용 요청 #1 (soldier2 → CM-001)

2.5 Swagger UI 테스트 방법

1. 브라우저에서 `http://{서버주소}:3000/api-docs` 접속
2. **01. Auth → POST /api/auth/login** → Try it out → Body 에 `{"username":"boss","password":"pass1234"}` → Execute → 응답의 `data.token` 복사
3. 우측 상단 **Authorize** → 토큰 붙여넣기 → Authorize. (다른 계정으로 바꿀 때는 Logout 후 새 토큰 입력)
4. 확인할 API → Try it out → Parameter/Body 입력 → Execute → 응답 코드와 본문 확인
5. **각 요구사항 확인 전에 데모 데이터를 초기화**: admin 토큰으로 `POST /api/admin/system/reset-demo` Body `{"confirm":"RESET"}` 실행 (모든 ID 가 본 보고서와 동일해짐)

3. 시스템 구조

3.1 디렉터리 구성 (CR-05)

```
midterm/
├── app.js                 # 진입점 (node app.js) - DB 연결, 스키마/시드, 라우트, Swagger, EJS
├── config/index.js        # 학번, DB 접속 정보, JWT 설정
├── db/
│   ├── mysql.js           # MySQL 풀, 쿼리/트랜잭션 헬퍼 (UTC 세션)
│   ├── mongo.js           # MongoDB 연결, 컬렉션/인덱스
│   ├── schema.js          # MySQL 테이블 DDL (20개)
│   └── seed.js            # 역할·권한 매트릭스, 데모 데이터, 초기화
├── lib/
│   ├── api.js             # API 레지스트리 → 라우팅/검증/권한/Swagger/추적표 자동 생성
│   ├── validate.js        # 요청 검증·형변환, 페이지/정렬 해석
│   ├── auth.js            # JWT 인증, 권한 코드 검사, CSRF 방어
│   ├── security.js        # 보안 헤더, 로그인 실패 제한, 접근 로그, 비밀키 관리
│   ├── errors.js          # 공통 오류 타입
│   ├── errorHandler.js    # 전역 오류 처리 (일관된 오류 응답, 거부 감사 기록)
│   ├── records.js         # MongoDB 이력/감사/알림 기록 헬퍼
│   └── util.js
├── services/              # 업무 규칙 (BR-01~BR-10)
│   ├── scope.js           # 역할별 관리/조회 범위
│   ├── members.js         # 상태 전이, 역할 변경 규칙
│   ├── tasks.js           # 업무 전이, 시간 중복 검사, 배정 가능 검사
│   ├── resources.js       # 자원 상태, 배정 중복/사용불가 검사
│   ├── reputation.js      # 평판 점수 계산
│   ├── jobs.js            # 주기 작업 (요청 만료, 회의 완료, 반납 지연 알림)
│   └── settings.js        # 운영 설정
├── routes/                # 기능별 REST API (26개 모듈) + pages.js(화면 라우터·권한별 메뉴)
├── views/                 # EJS 템플릿 (layout, login, error, catalog)
├── public/
│   ├── css/app.css        # 디자인 시스템
│   └── js/                # core.js(API·폼·테이블·차트 공통) + pages/*.js(21개 화면)
├── docs/requirements.js   # 요구사항 카탈로그 + 확인 절차 (보고서·자동 검증 공용)
└── scripts/               # generate-report.js, verify-report.js
```

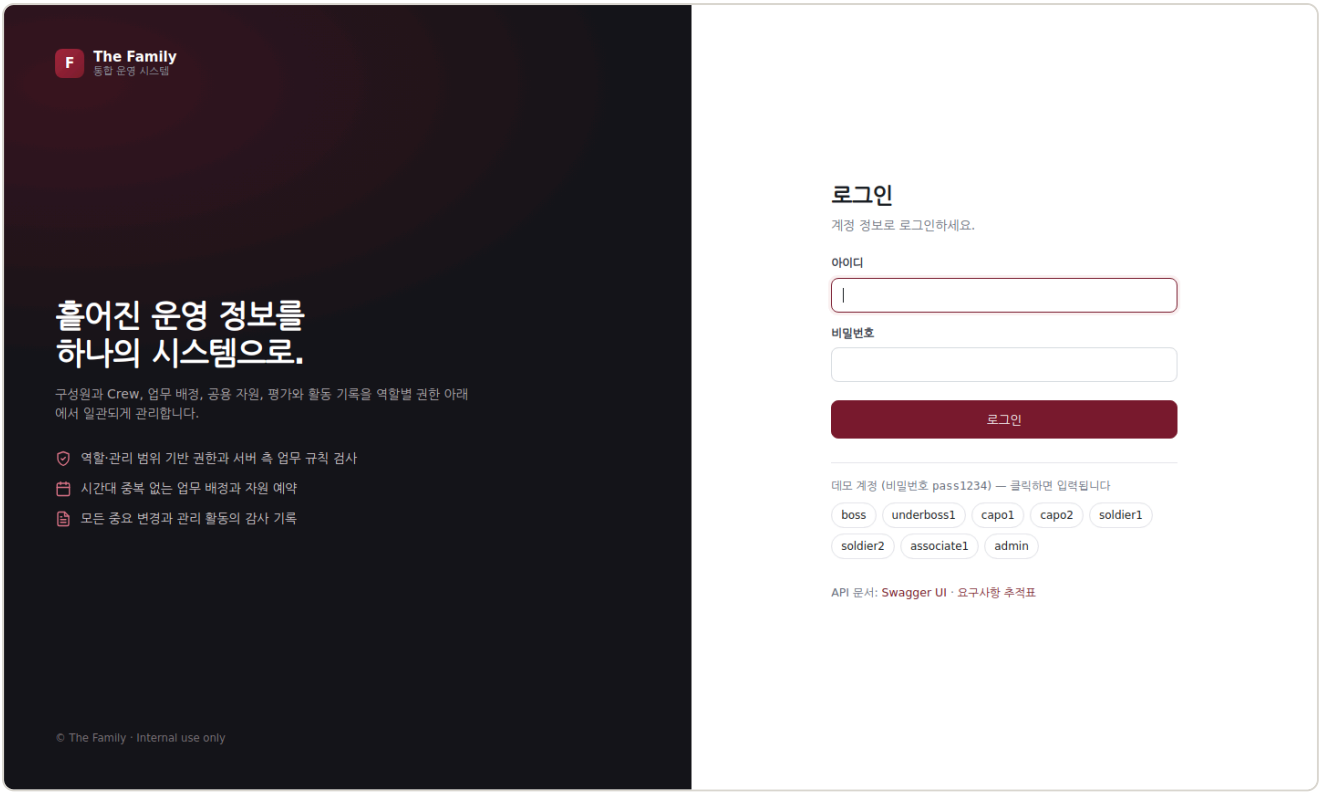
3.2 요청 처리 흐름

```
요청 → express.json → [authenticate: JWT 검증 → 폐기 토큰 확인(MongoDB) → 최신 계정/역할/상태/권한 로딩(MySQL)]
     → [requirePerm: 권한 코드 검사] → [validator: path/query/body 검증·형변환·불필요 필드 제거, 페이지/정렬 해석]
     → handler → services (관리 범위 · 업무 규칙 검사, 트랜잭션 + 행 잠금)
     → MySQL 반영 + MongoDB 이력/감사/알림 기록
     → 응답 { success, message, data, meta }  /  오류 → errorHandler → { success:false, error: { code, message, details } }
```

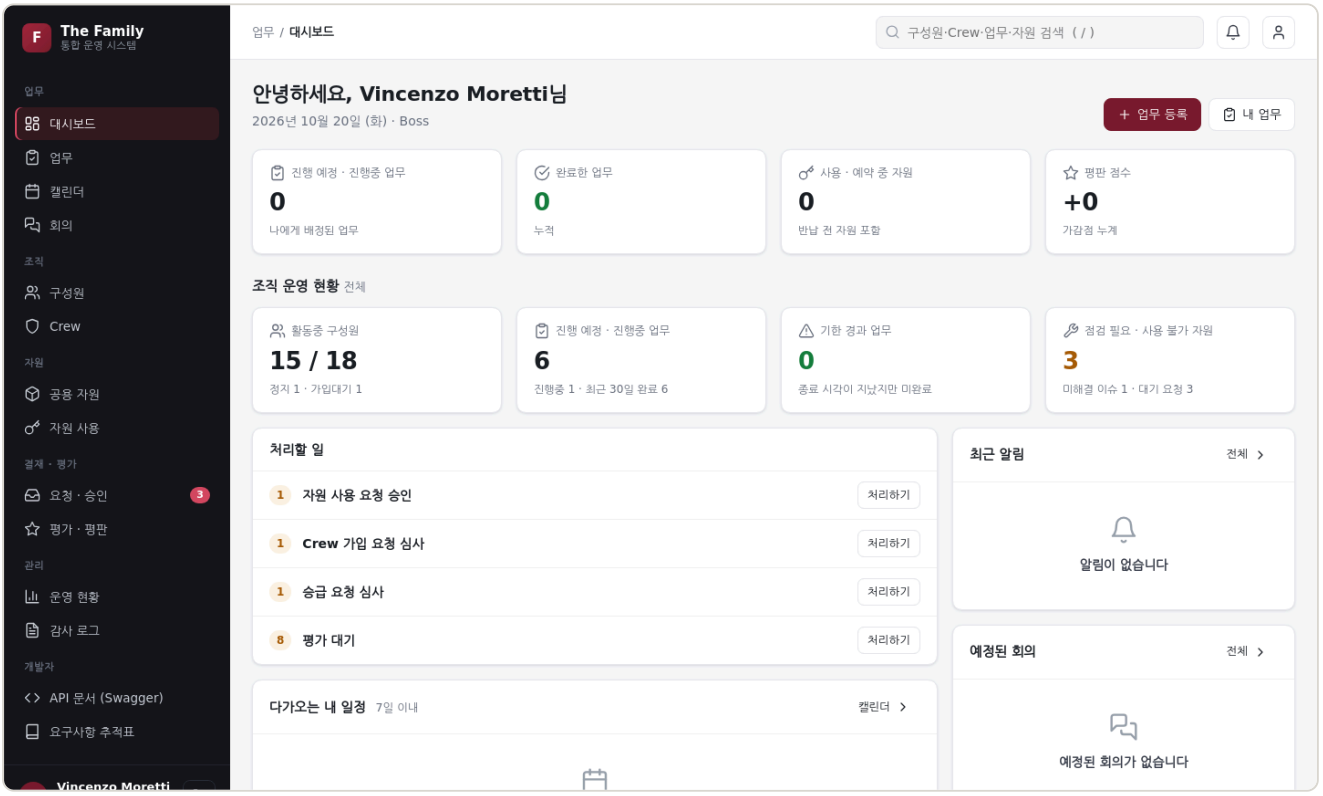
3.3 웹 화면 (EJS + REST API)

모든 업무 기능은 웹 화면에서 사용할 수 있다. 레이아웃·메뉴는 서버가 EJS 로 사용자 권한에 맞게 렌더링하고, 권한이 없는 화면은 서버에서 403 으로 차단한다. 화면의 조회·변경은 본 보고서의 REST API 를 그대로 호출하므로, 화면과 API 에 동일한 권한·업무 규칙이 적용된다(버튼 숨김은 편의 기능일 뿐 서버가 최종 판단).

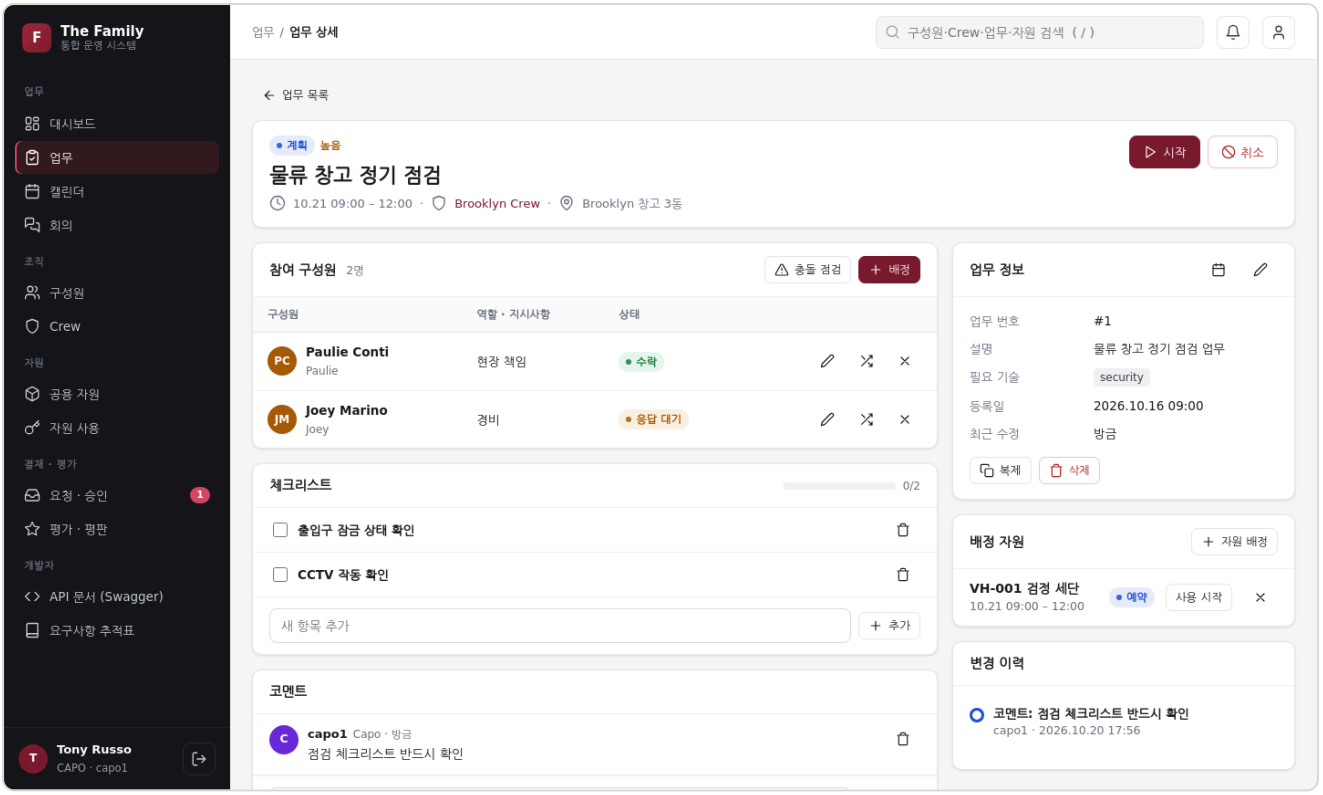
화면	경로	주요 기능	요구사항	접근
로그인	/login	로그인, 데모 계정 자동 입력, 로그인 실패 제한	FR-23, NFR-03	공개
대시보드	/dashboard	개인 KPI, 처리할 일(배정 응답·승인·평가 대기), 7일 일정, 알림, 회의, 조직 KPI·차트	FR-02, FR-09, FR-22	전체 (조직 KPI 는 stats.read)
업무	/tasks	기간·Crew·상태·중요도·담당자 탐색, 내 업무, 등록, 템플릿 등록, CSV	FR-07, FR-12, FR-21	전체 (범위 적용)
업무 상세	/tasks/:id	상태 전이, 완료(결과·참여 기록), 결과 수정·재개, 배정(후보 추천)·교체·해제, 수락/거절, 자원 배정·사용·반납, 체크리스트, 코멘트, 이력	FR-07~11, BR-02, BR-03, BR-06, BR-07	전체 (범위 적용)
캘린더	/calendar	월간 업무 일정, Crew·내 업무 필터	FR-09, FR-12	전체
회의	/meetings	회의 생성, 참석 응답, 참석자 관리, 회의록	FR-05, FR-09	전체
구성원	/members	검색·필터·정렬, 등록, CSV (범위 밖 연락처 비공개)	FR-01, FR-21	전체 (범위 적용)
구성원 상세	/members/:id	역할·상태 변경, 승급 요청, 징계, 평판 가감, 기술, 업무·일정·평가·변경·소속·징계 이력	FR-03, FR-04, FR-19, BR-04, BR-10	전체 (범위 적용)
Crew	/crews, /crews/:id	Crew 현황, 편성·제외·이동, 책임자·감독 지정, 공지, 가입 요청 심사, 운영 현황, 편성 이력	FR-05, FR-06, BR-01, BR-05	전체 (관리 기능은 범위)
공용 자원	/resources, /resources/:id	자원 검색, 시간대별 이용 가능 자원, 관리 필요 자원, 등록·상태 변경, 배정, 사용 요청, 이상 신고·해결, 예약·이력	FR-13~17, BR-07, BR-08	전체 (등급 적용)
자원 사용	/allocations	내/관리 자원 예약·사용·반납, 반납 지연, 내 사용 요청	FR-15, FR-17	전체
요청 · 승인	/approvals	자원 사용·Crew 가입·승급 요청 승인/반려/취소	FR-03, FR-06, FR-17	전체
평가 · 평판	/evaluations	평가 대기, 평가 작성·수정·무효화, 평판 순위, 평가 항목 관리	FR-18, FR-19	전체 (기능별 권한)
운영 현황	/stats	기간·Crew 별 KPI, 업무 추이, 상태 분포, Crew 비교, 자원 가동률, 업무량, 활동 히트맵, 변경 추이	FR-22	stats.read
감사 로그	/audit	행위자·행위·대상·결과·기간 조회, 집계, CSV, 상세	FR-20	audit.read
계정 관리	/admin/accounts	잠금·해제, 비밀번호 초기화, 강제 로그아웃, 로그인 이력, 실패 탐지	FR-23	account.manage
권한 관리	/admin/roles	역할별 권한 매트릭스 편집 (즉시 반영)	FR-23, BR-09	permission.manage
시스템 설정	/admin/system	운영 정책, 무결성 점검, DB 현황, 공지 발송, 데모 초기화	FR-23, DR-05	system.manage
알림 · 내 정보	/notifications, /me	알림 확인·삭제, 연락처·비밀번호 변경, 알림 수신 설정, 로그인 이력, 저장한 검색	FR-02, FR-09	전체



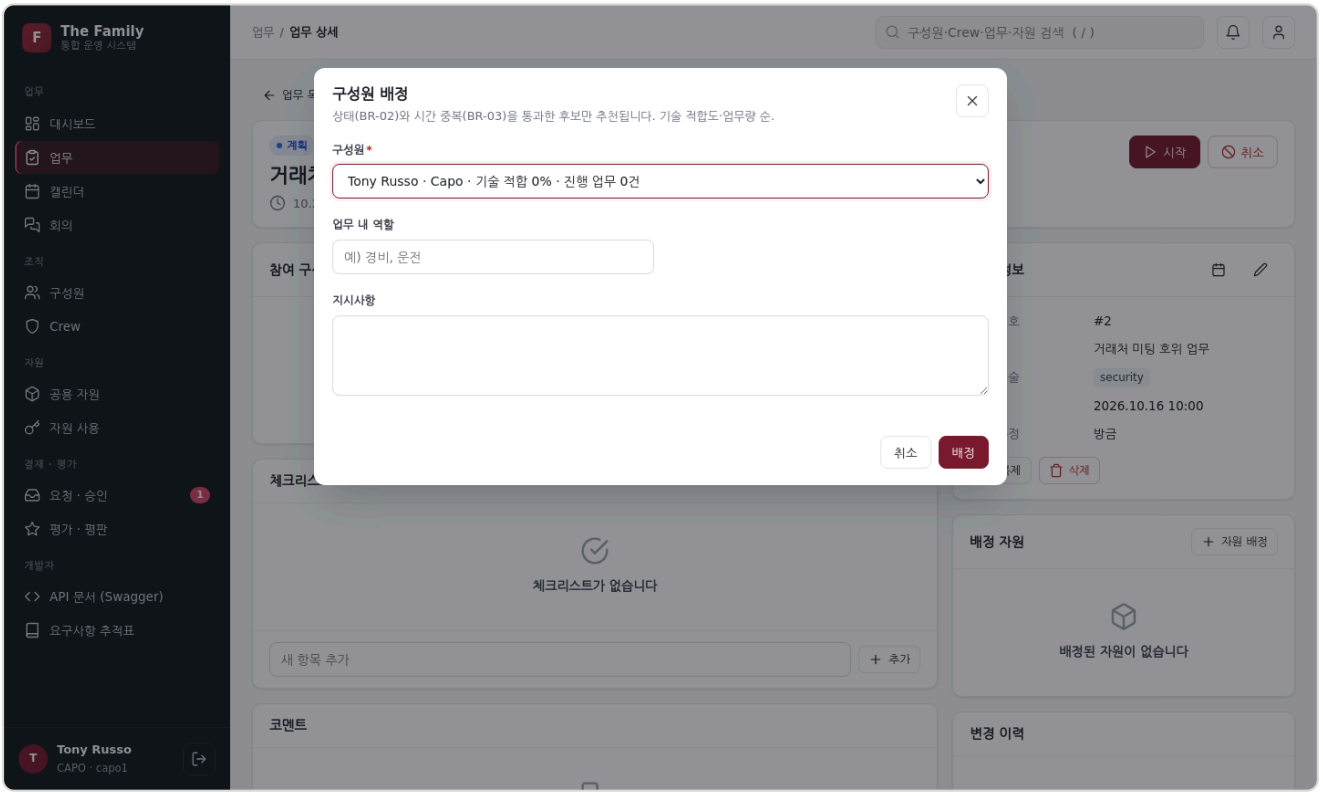
로그인 화면



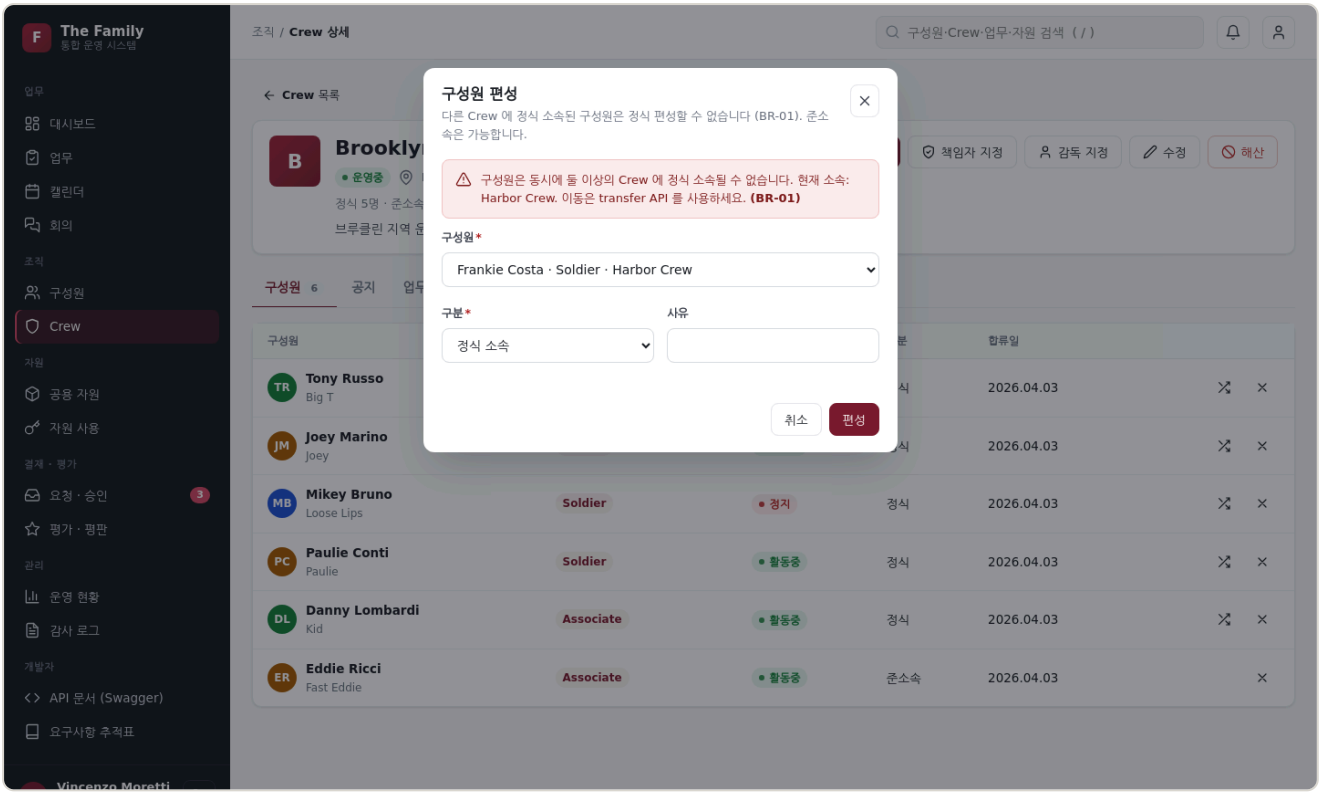
대시보드 (Boss)



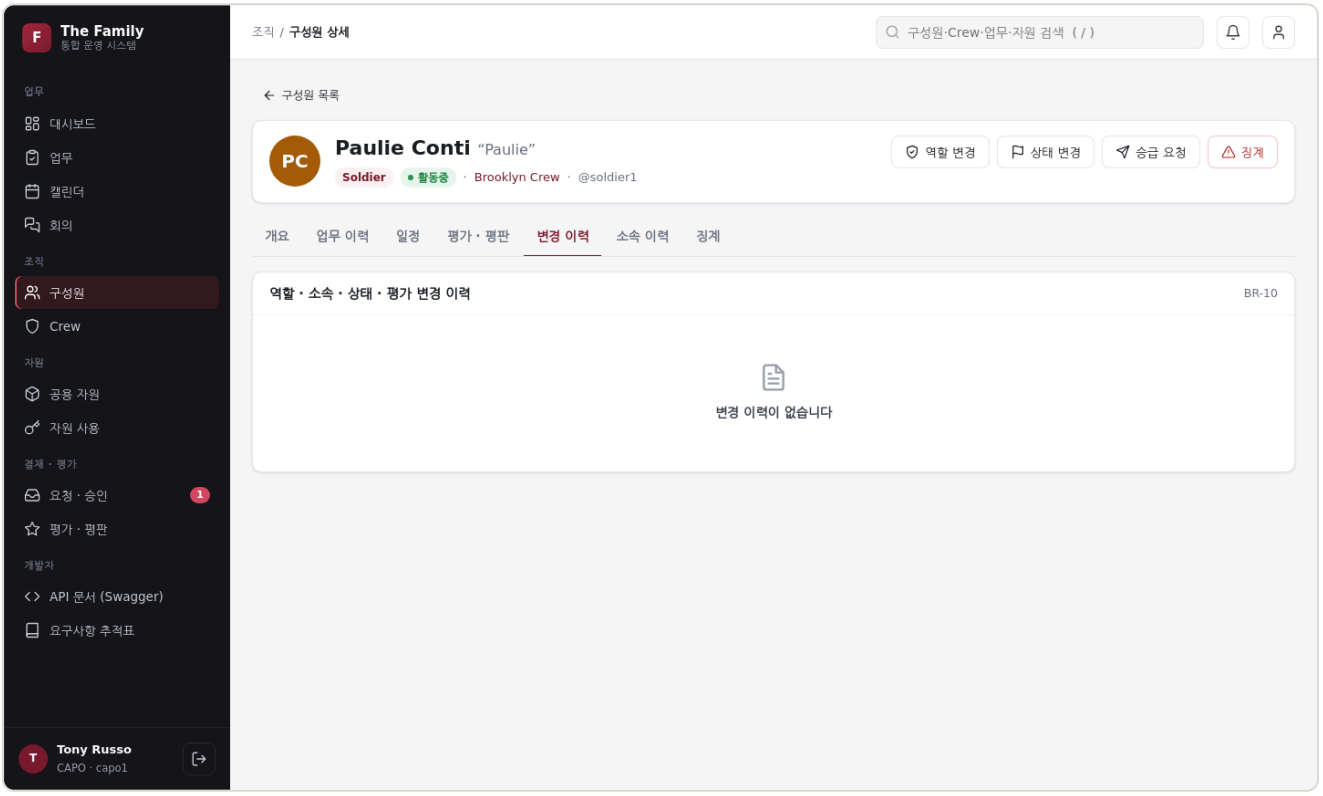
업무 상세 (Capo)



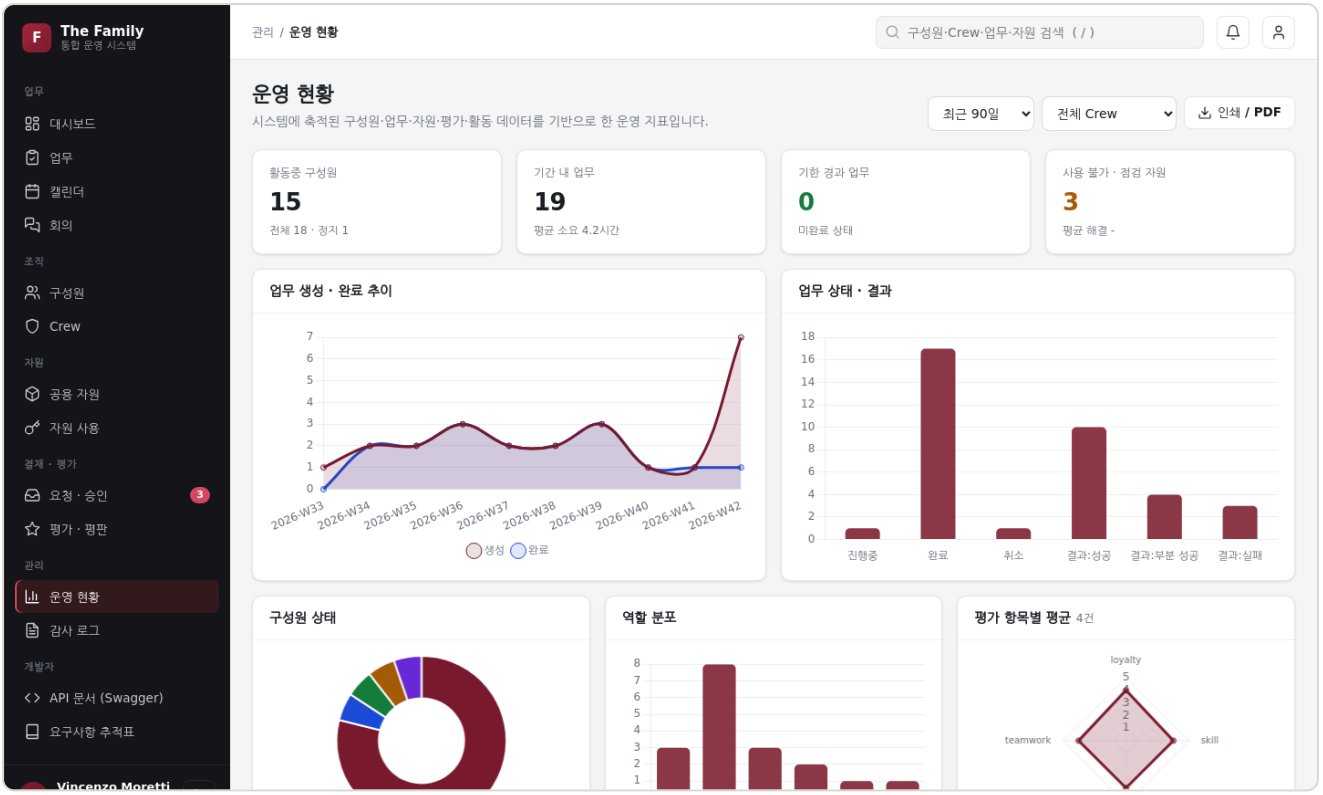
배정 후보 추천 - 정지·시간 중복 구성원 자동 제외 (BR-02, BR-03)



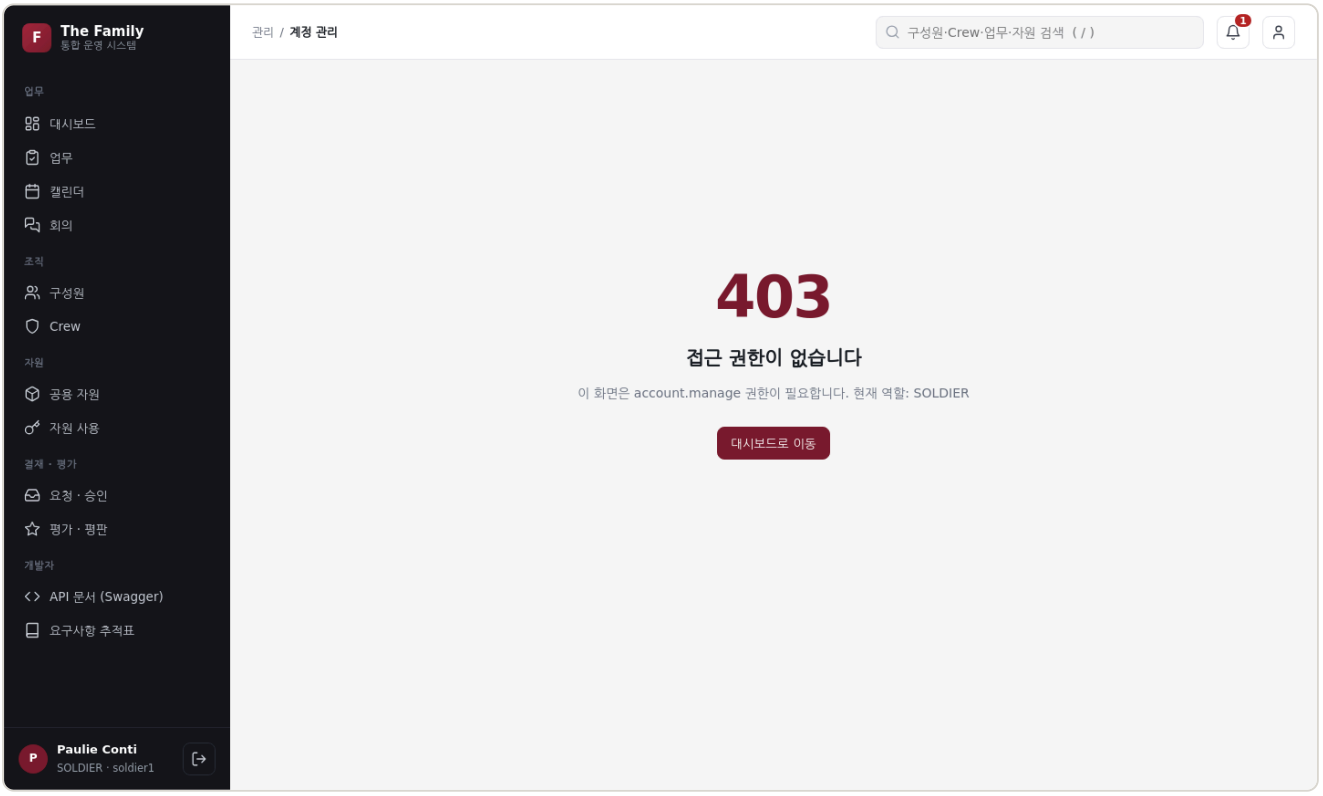
업무 규칙 위반 안내 - 정식 소속 중복 (BR-01)



구성원 변경 이력 (BR-10)



운영 현황 (FR-22)



권한 없는 화면 직접 접근 차단 (BR-09)

3.4 운영 · 보안

항목	구현
인증	JWT(8시간) + httpOnly 쿠키(화면). 매 요청마다 DB 에서 계정 상태·잠금·토큰 버전·권한을 확인하여 정지·탈퇴·잠금·강제 로그아웃이 즉시 반영됨

비밀키 관리	JWT 서명키를 소스에 두지 않고 최초 실행 시 난수로 생성해 MongoDB 에 보관 (JWT_SECRET 환경변수로 지정 가능)
비밀번호	bcrypt 해시, 8자 이상, 변경 시 다른 세션 만료, 관리자 초기화 시 임시 비밀번호
무차별 대입 방지	계정별 연속 실패 시 자동 잠금(설정값) + IP·아이디별 실패 횟수 제한(10분 20회 / IP 60회, 429)
CSRF	쿠키로 인증된 변경 요청은 X-Requested-With 헤더 필수 (다른 사이트의 form 전송 차단), SameSite=Lax 쿠키
보안 헤더	Content-Security-Policy(인라인 스크립트 금지), X-Frame-Options, X-Content-Type-Options, Referrer-Policy
입력 검증	모든 API 의 path/query/body 를 스키마로 검증·형변환, 미정의 필드 제거, SQL 은 전부 파라미터 바인딩, 화면 출력은 자동 이스케이프
권한 오남용 방지	권한 코드 + 관리 범위 + 등급 비교(BR-04)를 서버에서 검사, 거부 시도는 감사 로그(DENIED)에 기록
동시성	배정·자원 예약·소속 변경을 트랜잭션 + 행 잠금으로 처리, DB UNIQUE 제약으로 이중 보장
주기 작업	5분마다 오래된 자원 요청 자동 만료, 종료된 회의 완료 처리, 반납 지연 알림
안정성	전역 오류 처리(서버 비정상 종료 방지), 포트 사용 중이면 다음 포트로 자동 실행, SIGINT/SIGTERM 시 정상 종료, gzip 압축, 접근 로그
운영 모드	DEMO_MODE=false 로 실행하면 데모 초기화 API 비활성화

4. 역할 · 권한 설계

4.1 역할 등급

등급	역할	관리 범위 (scope)	설명
1	ASSOCIATE	본인	본인 정보와 소속 Crew 기본 정보 조회, 배정된 활동 참여
2	SOLDIER	본인 (소속 Crew 업무 조회)	배정 업무 수행, 자원 사용 요청·체크아웃·반납
3	CAPO	자신이 책임자인 Crew	담당 Crew 구성원·업무·자원 배정 관리
4	UNDERBOSS	감독(underboss_id)하는 Crew + 무소속 구성원	여러 Crew 운영, 승급 승인, 결과 수정, 통계
5	BOSS	조직 전체	조직 전반 운영·감사
6	ADMINISTRATOR	조직 전체 (기술 관리)	계정·권한·시스템 설정·감사 기록

권한 판정 = ① **권한 코드**(역할별 권한 매트릭스, DB role_permissions, 관리자가 API 로 변경 가능) **+** ② **관리 범위**(services/scope.js) **+** ③ **업무 규칙**(BR-04 등급 비교 등).

4.2 역할별 권한 매트릭스 (초기값)

권한 코드	설명	Associate	Soldier	Capo	Underboss	Boss	Admin
`member.create`	구성원 등록				●	●	●
`member.update`	구성원 기본 정보 수정				●	●	●
`member.delete`	가입대기 구성원 삭제					●	●
`member.role`	구성원 역할 변경			●	●	●	●
`member.status`	구성원 상태 변경			●	●	●	●
`member.read`	관리 범위 구성원 상세/이력 조회			●	●	●	●
`member.discipline`	징계 등록/해제			●	●	●	
`promotion.request`	승급 요청 등록			●	●	●	
`promotion.review`	승급 요청 승인/반려				●	●	
`crew.create`	Crew 생성/해산/복구					●	
`crew.update`	Crew 정보 수정				●	●	
`crew.appoint`	Crew 책임자/감독자 지정				●	●	
`crew.members`	Crew 구성원 편성 변경 및 가입 요청 심사			●	●	●	
`crew.announce`	Crew 공지 관리			●	●	●	
`region.manage`	지역 관리					●	●
`task.create`	업무 등록/복제/삭제			●	●	●	
`task.update`	업무 정보/일정 수정			●	●	●	

`task.assign`	업무 배정 관리			●	●	●	
`task.progress`	업무 진행 상태 변경 및 완료 처리			●	●	●	
`task.result.amend`	완료된 업무 결과 수정 및 재개				●	●	
`template.manage`	업무 템플릿 관리			●	●	●	
`meeting.manage`	회의 관리			●	●	●	
`resource.manage`	자원 등록/수정/상태 관리				●	●	
`resource.allocate`	자원 배정 및 사용 요청 심사			●	●	●	
`evaluation.write`	평가 작성			●	●	●	
`evaluation.read`	관리 범위 평가 이력 조회			●	●	●	●
`evaluation.void`	평가 무효화				●	●	
`evaluation.criteria`	평가 항목 관리					●	●
`reputation.adjust`	평판 가감점 부여				●	●	
`stats.read`	운영 현황/통계 조회				●	●	●
`audit.read`	감사 로그 조회					●	●
`account.manage`	계정 잠금/해제/비밀번호 초기화/강제 로그아웃						●
`permission.manage`	역할별 권한 관리						●
`system.manage`	시스템 설정/점검/데모 초기화						●

권한 코드가 필요 없는 API(본인 정보, 배정 업무, 알림 등)는 로그인한 모든 사용자가 사용하되, 데이터 범위는 서버에서 본인/소속 Crew 로 제한된다.

5. 상태 체계

5.1 구성원 상태 (FR-04)

상태	의미	허용 전이	효과
PENDING	가입대기	ACTIVE, WITHDRAWN	조회만 가능, 업무 배정 불가
ACTIVE	활동중	STANDBY, SUSPENDED, WITHDRAWN	정상
STANDBY	대기	ACTIVE, SUSPENDED, WITHDRAWN	업무 배정 가능(대기 인력)
SUSPENDED	정지	ACTIVE, STANDBY, WITHDRAWN	쓰기 작업 차단, 새 업무 배정 불가(BR-02)
WITHDRAWN	탈퇴	(Administrator 만 STANDBY 복구)	로그인 불가, 소속 종료·미완료 배정 해제·예약 취소·세션 만료

5.2 업무 상태 (FR-10)

상태	허용 전이	비고
PLANNED	IN_PROGRESS, CANCELLED	등록 직후
IN_PROGRESS	ON_HOLD, CANCELLED, (COMPLETED: /complete)	완료는 결과·참여 기록 입력 필수
ON_HOLD	IN_PROGRESS, CANCELLED	
COMPLETED	(IN_PROGRESS: /reopen, Underboss 이상)	결과 보호 (BR-06)
CANCELLED	PLANNED (참여자 일정 재검사)	취소 시 자원 예약 자동 취소

배정 상태: ASSIGNED(배정) → ACCEPTED(수락) / DECLINED(거절) / REMOVED(해제)

5.3 자원 상태 (FR-13, FR-16)

상태	구분	수동 전이
AVAILABLE	정상	NEEDS_INSPECTION, MAINTENANCE, UNAVAILABLE, RETIRED
IN_USE	정상(사용중, 체크아웃/반납으로 자동 관리)	NEEDS_INSPECTION, UNAVAILABLE
NEEDS_INSPECTION	사용 불가 계열 (이상 신고·파손 반납 시 자동)	AVAILABLE, MAINTENANCE, UNAVAILABLE, RETIRED
MAINTENANCE	사용 불가 계열	AVAILABLE, NEEDS_INSPECTION, UNAVAILABLE, RETIRED
UNAVAILABLE	사용 불가 계열 (분실 반납 시 자동)	AVAILABLE, MAINTENANCE, RETIRED
RETIRED	폐기	-

자원 배정 상태: RESERVED(예약) → IN_USE(체크아웃) → RETURNED(반납, GOOD/DAMAGED/LOST) / CANCELLED(취소)

요청 상태(승급·가입·자원 사용): PENDING → APPROVED / REJECTED / CANCELLED

6. 데이터베이스 설계

6.1 SQL / MongoDB 역할 분담 (DR-05)

DB	저장 데이터	선택 이유
MySQL family_20260000 0	계정·역할·권한, 구성원, 지역, Crew, Crew 소속, 기술, 승급/가입 요청, 징계, 업무, 업무 배정, 자원 유형/자원, 자원 배정/요청/이슈, 회의/참석자	엔터티 간 관계(FK)와 일관성이 핵심이며, 업무 규칙(정식 소속 1개, 시간 중복 금지)을 트랜잭션·행 잠금·UNIQUE/CHECK 제약으로 보장해야 함
Mongo DB family_log_20260 0000	감사 로그, 구성원/업무/자원 변경 이력, 평가·평가 항목·평판, 활동 기록, 코멘트·체크리스트·템플릿·공지, 알림, 로그인 이력, 설정, 회의록	시간에 따라 계속 증가(append-only)하고, 문서마다 구조가 다르거나 확장되는 데이터(평가 항목, 활동 상세, 이력 부가 정보)에 적합. 시간 인덱스로 조회

하나의 기능 흐름에서 두 DB 를 함께 사용한다. 예) 업무 완료: MySQL tasks/task_assignments 갱신 + MongoDB task_history·activities·notifications 기록.

6.2 MySQL 테이블 (20개)

주요 제약: `crew_memberships.active_formal_key` (UNIQUE, 생성 컬럼) → BR-01, `task_assignments.active_key` (UNIQUE) → 동일 업무 중복 배정 방지, `CHECK(end_at > start_at)`, 모든 관계 FK.

```
CREATE TABLE IF NOT EXISTS roles (
  id TINYINT UNSIGNED PRIMARY KEY,
  code VARCHAR(20) NOT NULL UNIQUE,
  name VARCHAR(50) NOT NULL,
  level TINYINT UNSIGNED NOT NULL UNIQUE,
  description VARCHAR(255)
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS permissions (
  code VARCHAR(50) PRIMARY KEY,
  category VARCHAR(30) NOT NULL,
  description VARCHAR(255) NOT NULL
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS role_permissions (
  role_id TINYINT UNSIGNED NOT NULL,
  permission_code VARCHAR(50) NOT NULL,
  PRIMARY KEY (role_id, permission_code),
  FOREIGN KEY (role_id) REFERENCES roles(id),
  FOREIGN KEY (permission_code) REFERENCES permissions(code) ON DELETE CASCADE
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS regions (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL UNIQUE,
  description VARCHAR(255),
```

```
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS members (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(30) NOT NULL UNIQUE,
  password_hash VARCHAR(100) NOT NULL,
  name VARCHAR(50) NOT NULL,
  alias VARCHAR(50),
  email VARCHAR(100) UNIQUE,
  phone VARCHAR(30),
  birth_date DATE,
  role_id TINYINT UNSIGNED NOT NULL,
  status ENUM('PENDING','ACTIVE','STANDBY','SUSPENDED','WITHDRAWN') NOT NULL DEFAULT 'PENDING',
  joined_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  notes TEXT,
  is_locked TINYINT(1) NOT NULL DEFAULT 0,
  locked_reason VARCHAR(255),
  token_version INT UNSIGNED NOT NULL DEFAULT 0,
  failed_logins INT UNSIGNED NOT NULL DEFAULT 0,
  last_login_at DATETIME,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (role_id) REFERENCES roles(id),
  INDEX idx_members_status (status),
  INDEX idx_members_role (role_id)
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS crews (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL UNIQUE,
  region_id INT UNSIGNED,
  description VARCHAR(255),
  capo_id INT UNSIGNED,
  underboss_id INT UNSIGNED,
  status ENUM('ACTIVE','DISSOLVED') NOT NULL DEFAULT 'ACTIVE',
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (region_id) REFERENCES regions(id),
  FOREIGN KEY (capo_id) REFERENCES members(id),
  FOREIGN KEY (underboss_id) REFERENCES members(id)
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS crew_memberships (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  crew_id INT UNSIGNED NOT NULL,
  member_id INT UNSIGNED NOT NULL,
  membership_type ENUM('FORMAL','AFFILIATE') NOT NULL DEFAULT 'FORMAL',
  joined_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  left_at DATETIME,
  left_reason VARCHAR(255),
  changed_by INT UNSIGNED,
  active_formal_key INT UNSIGNED GENERATED ALWAYS AS
    (IF(left_at IS NULL AND membership_type = 'FORMAL', member_id, NULL)) STORED,
  active_pair_key VARCHAR(40) GENERATED ALWAYS AS
    (IF(left_at IS NULL, CONCAT(crew_id, ':', member_id), NULL)) STORED,
  UNIQUE KEY uq_one_formal_crew (active_formal_key),
  UNIQUE KEY uq_active_pair (active_pair_key),
  FOREIGN KEY (crew_id) REFERENCES crews(id),
  FOREIGN KEY (member_id) REFERENCES members(id),
```

```
  INDEX idx_cm_member (member_id, left_at)
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS member_skills (
  member_id INT UNSIGNED NOT NULL,
  skill VARCHAR(50) NOT NULL,
  level TINYINT UNSIGNED NOT NULL DEFAULT 1,
  PRIMARY KEY (member_id, skill),
  FOREIGN KEY (member_id) REFERENCES members(id) ON DELETE CASCADE
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS promotion_requests (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  member_id INT UNSIGNED NOT NULL,
  current_role_id TINYINT UNSIGNED NOT NULL,
  target_role_id TINYINT UNSIGNED NOT NULL,
  requested_by INT UNSIGNED NOT NULL,
  reason VARCHAR(500),
  status ENUM('PENDING','APPROVED','REJECTED','CANCELLED') NOT NULL DEFAULT 'PENDING',
  reviewed_by INT UNSIGNED,
  review_comment VARCHAR(500),
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  reviewed_at DATETIME,
  FOREIGN KEY (member_id) REFERENCES members(id),
  FOREIGN KEY (target_role_id) REFERENCES roles(id),
  FOREIGN KEY (requested_by) REFERENCES members(id)
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS crew_join_requests (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  crew_id INT UNSIGNED NOT NULL,
  member_id INT UNSIGNED NOT NULL,
  membership_type ENUM('FORMAL','AFFILIATE') NOT NULL DEFAULT 'FORMAL',
  message VARCHAR(500),
  status ENUM('PENDING','APPROVED','REJECTED','CANCELLED') NOT NULL DEFAULT 'PENDING',
  reviewed_by INT UNSIGNED,
  review_comment VARCHAR(500),
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  reviewed_at DATETIME,
  FOREIGN KEY (crew_id) REFERENCES crews(id),
  FOREIGN KEY (member_id) REFERENCES members(id)
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS disciplinary_actions (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  member_id INT UNSIGNED NOT NULL,
  action_type ENUM('WARNING','SUSPENSION') NOT NULL,
  reason VARCHAR(500) NOT NULL,
  issued_by INT UNSIGNED NOT NULL,
  status ENUM('ACTIVE','REVOKED') NOT NULL DEFAULT 'ACTIVE',
  revoked_by INT UNSIGNED,
  revoke_reason VARCHAR(500),
  revoked_at DATETIME,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (member_id) REFERENCES members(id),
  FOREIGN KEY (issued_by) REFERENCES members(id)
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS tasks (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(100) NOT NULL,
  description TEXT,
  crew_id INT UNSIGNED NOT NULL,
  priority ENUM('LOW','NORMAL','HIGH','CRITICAL') NOT NULL DEFAULT 'NORMAL',
  status ENUM('PLANNED','IN_PROGRESS','ON_HOLD','COMPLETED','CANCELLED') NOT NULL DEFAULT 'PLANNED',
  start_at DATETIME NOT NULL,
  end_at DATETIME NOT NULL,
  location VARCHAR(100),
  allow_concurrent TINYINT(1) NOT NULL DEFAULT 0,
  required_skills JSON,
  result_outcome ENUM('SUCCESS','PARTIAL','FAILURE'),
  result_summary TEXT,
  result_value DECIMAL(14,2),
  completed_at DATETIME,
  completed_by INT UNSIGNED,
  created_by INT UNSIGNED NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CHECK (end_at > start_at),
  FOREIGN KEY (crew_id) REFERENCES crews(id),
  FOREIGN KEY (created_by) REFERENCES members(id),
  INDEX idx_tasks_crew_status (crew_id, status),
  INDEX idx_tasks_period (start_at, end_at)
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS task_assignments (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  task_id INT UNSIGNED NOT NULL,
  member_id INT UNSIGNED NOT NULL,
  task_role VARCHAR(50),
  instructions TEXT,
  status ENUM('ASSIGNED','ACCEPTED','DECLINED','REMOVED') NOT NULL DEFAULT 'ASSIGNED',
  assigned_by INT UNSIGNED NOT NULL,
  assigned_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  responded_at DATETIME,
  removed_at DATETIME,
  attendance ENUM('PRESENT','PARTIAL','ABSENT'),
  contribution TINYINT UNSIGNED,
  participation_note VARCHAR(500),
  active_key VARCHAR(40) GENERATED ALWAYS AS
    (IF(status IN ('ASSIGNED','ACCEPTED'), CONCAT(task_id, ':', member_id), NULL)) STORED,
  UNIQUE KEY uq_active_assignment (active_key),
  FOREIGN KEY (task_id) REFERENCES tasks(id),
  FOREIGN KEY (member_id) REFERENCES members(id),
  INDEX idx_ta_member (member_id, status)
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS resource_types (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  code VARCHAR(30) NOT NULL UNIQUE,
  name VARCHAR(50) NOT NULL,
  description VARCHAR(255),
  default_min_level TINYINT UNSIGNED NOT NULL DEFAULT 2
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS resources (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  code VARCHAR(30) NOT NULL UNIQUE,
```

```
  name VARCHAR(100) NOT NULL,
  type_id INT UNSIGNED NOT NULL,
  crew_id INT UNSIGNED,
  status ENUM('AVAILABLE','IN_USE','NEEDS_INSPECTION','MAINTENANCE','UNAVAILABLE','RETIRED') NOT NULL
DEFAULT 'AVAILABLE',
  location VARCHAR(100),
  min_role_level TINYINT UNSIGNED NOT NULL DEFAULT 2,
  description VARCHAR(500),
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (type_id) REFERENCES resource_types(id),
  FOREIGN KEY (crew_id) REFERENCES crews(id),
  INDEX idx_resources_status (status)
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS resource_allocations (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  resource_id INT UNSIGNED NOT NULL,
  task_id INT UNSIGNED,
  member_id INT UNSIGNED,
  start_at DATETIME NOT NULL,
  end_at DATETIME NOT NULL,
  status ENUM('RESERVED','IN_USE','RETURNED','CANCELLED') NOT NULL DEFAULT 'RESERVED',
  purpose VARCHAR(255),
  allocated_by INT UNSIGNED NOT NULL,
  checked_out_at DATETIME,
  returned_at DATETIME,
  return_condition ENUM('GOOD','DAMAGED','LOST'),
  return_note VARCHAR(500),
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CHECK (end_at > start_at),
  CHECK (task_id IS NOT NULL OR member_id IS NOT NULL),
  FOREIGN KEY (resource_id) REFERENCES resources(id),
  FOREIGN KEY (task_id) REFERENCES tasks(id),
  FOREIGN KEY (member_id) REFERENCES members(id),
  INDEX idx_alloc_resource (resource_id, status, start_at, end_at)
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS resource_requests (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  resource_id INT UNSIGNED NOT NULL,
  requester_id INT UNSIGNED NOT NULL,
  task_id INT UNSIGNED,
  start_at DATETIME NOT NULL,
  end_at DATETIME NOT NULL,
  purpose VARCHAR(255),
  status ENUM('PENDING','APPROVED','REJECTED','CANCELLED') NOT NULL DEFAULT 'PENDING',
  reviewed_by INT UNSIGNED,
  review_comment VARCHAR(500),
  allocation_id INT UNSIGNED,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  reviewed_at DATETIME,
  FOREIGN KEY (resource_id) REFERENCES resources(id),
  FOREIGN KEY (requester_id) REFERENCES members(id)
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS resource_issues (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  resource_id INT UNSIGNED NOT NULL,
  reported_by INT UNSIGNED NOT NULL,
```

```
  severity ENUM('LOW','MEDIUM','HIGH') NOT NULL DEFAULT 'MEDIUM',
  description VARCHAR(500) NOT NULL,
  status ENUM('OPEN','RESOLVED') NOT NULL DEFAULT 'OPEN',
  resolved_by INT UNSIGNED,
  resolution VARCHAR(500),
  resolved_at DATETIME,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (resource_id) REFERENCES resources(id),
  FOREIGN KEY (reported_by) REFERENCES members(id)
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS meetings (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(100) NOT NULL,
  agenda TEXT,
  crew_id INT UNSIGNED,
  location VARCHAR(100),
  start_at DATETIME NOT NULL,
  end_at DATETIME NOT NULL,
  status ENUM('SCHEDULED','CANCELLED','DONE') NOT NULL DEFAULT 'SCHEDULED',
  organizer_id INT UNSIGNED NOT NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CHECK (end_at > start_at),
  FOREIGN KEY (crew_id) REFERENCES crews(id),
  FOREIGN KEY (organizer_id) REFERENCES members(id)
  ) ENGINE=InnoDB
```

```
CREATE TABLE IF NOT EXISTS meeting_attendees (
  meeting_id INT UNSIGNED NOT NULL,
  member_id INT UNSIGNED NOT NULL,
  rsvp ENUM('PENDING','YES','NO','MAYBE') NOT NULL DEFAULT 'PENDING',
  responded_at DATETIME,
  PRIMARY KEY (meeting_id, member_id),
  FOREIGN KEY (meeting_id) REFERENCES meetings(id) ON DELETE CASCADE,
  FOREIGN KEY (member_id) REFERENCES members(id)
  ) ENGINE=InnoDB
```

6.3 MongoDB 컬렉션 (19개)

컬렉션	용도	주요 필드
`audit_logs`	관리 활동·중요 상태 변경·권한 거부 감사 로그 (FR-20)	at, actor{id,username,role}, action, entity{type,id}, details{}, result(SUCCESS|DENIED), ip
`member_history`	구성원 역할/소속/상태/평가/평판/징계 변경 이력 (BR-10)	memberId, type(ROLE|CREW|STATUS|EVALUATION|REPUTATION|DISCIPLINE), from, to, reason, by{}, at, (effects, vacated…)
`task_history`	업무 상태·일정·배정·결과 변경 이력 (FR-10)	taskId, event, from, to, note, by{}, at, (memberId, outcome…)
`resource_history`	자원 등록·상태 변경·배정·체크아웃·반납·이슈 이력 (FR-15)	resourceId, event, from, to, by{}, at, (allocationId, condition, reason…)
`evaluations`	평가 (FR-18) – 평가 항목이 바뀌어도 스키마 변경 불필요	memberId, taskId, evaluator{}, scores{항목코드:점수}, overall, comment, status, revisions[], createdAt
`evaluation_criteria`	평가 항목 및 가중치	code, name, description, weight, active
`reputation_ledger`	평판 가감점 원장	memberId, points, reason, by{}, at

`activities`	구성원 활동 기록 (현장 보고 등 자유 형식)	memberId, type, description, refs{taskId}, location, details{}, source, at
`task_comments`	업무 코멘트	taskId, author{}, body, createdAt, updatedAt, edited
`task_checklists`	업무 체크리스트 (가변 길이 항목 배열)	taskId(unique), items[{id,text,done,doneBy,updatedAt}], seq
`task_templates`	업무 템플릿 (구조화된 기본값 + 체크리스트)	name, description, defaults{priority,durationHours,location,allowConcurrent,requiredSkills}, checklist[]
`crew_announcements`	Crew 공지	crewId, title, body, pinned, author{}, createdAt
`notifications`	사용자 알림	memberId, type, title, message, link, read, createdAt
`login_history`	로그인/로그아웃 이력 및 실패 기록	memberId, username, event, result, reason, ip, userAgent, at
`revoked_tokens`	로그아웃된 토큰 (TTL 인덱스로 만료 시 자동 삭제)	jti(unique), memberId, expiresAt
`settings`	시스템 운영 설정	_id=system, evaluationEditWindowHours, maxActiveAssignmentsPerMember, loginLockThreshold, …
`saved_filters`	사용자별 저장된 검색 조건	memberId, name, target, query{}
`notification_prefs`	알림 수신 거부 설정	memberId(unique), mutedTypes[]
`meeting_minutes`	회의록 (자유 형식 본문 + 결정사항)	meetingId(unique), content, decisions[], updatedBy, updatedAt

7. 업무 규칙 구현

ID	규칙	서버 측 구현
BR-01	하나의 구성원은 동시에 둘 이상의 Crew에 정식 소속될 수 없다.	① 서버: 편성/가입승인/이동 시 기존 정식 소속을 FOR UPDATE 로 잠그고 검사. ② DB: crew_memberships 의 생성 컬럼 active_formal_key 에 UNIQUE 제약 → 어떤 경로로도 위반 불가. 이동은 소속 종료+신규 소속을 하나의 트랜잭션으로 처리.
BR-02	정지 또는 탈퇴 상태의 구성원은 새로운 업무에 배정될 수 없다.	배정 트랜잭션에서 구성원 행을 FOR UPDATE 로 잠근 뒤 상태 검사(SUSPENDED/WITHDRAWN/PENDING 거부). 정지 징계 시 상태 자동 전환, 탈퇴 시 기존 미완료 배정 자동 해제.
BR-03	동일 시간대에 동시에 수행할 수 없는 업무가 하나의 구성원에게 중복 배정되어서는 안 된다.	배정·인원교체·일정변경·취소복구 시 구성원의 진행 전/중 업무와 시간 구간 겹침(start<기존end AND end>기존start)을 검사. 두 업무가 모두 allowConcurrent=true 인 경우만 예외. 구성원 행 잠금으로 동시 요청에도 1건만 성공.
BR-04	구성원은 자신보다 높은 권한을 가진 사용자의 역할을 임의로 변경할 수 없다.	역할/상태/징계/계정 변경 시 대상의 현재 등급과 부여할 역할 등급이 모두 요청자보다 낮아야 함(Administrator 예외). 자기 자신 변경 금지. Capo 는 Associate↔Soldier 만 변경.
BR-05	Crew 책임자는 자신이 관리하지 않는 Crew의 구성원 편성을 임의로 변경할 수 없다.	모든 편성 API(추가/제외/이동/가입승인/책임자 지정)에서 services/scope.js 의 assertManageCrew 로 대상 Crew(이동은 출발·도착 모두)가 관리 범위인지 검사.
BR-06	완료된 업무의 핵심 결과 정보는 일반 구성원이 임의로 변경할 수 없어야 한다.	완료 업무의 기본정보·일정·상태·배정·체크리스트 변경을 서버에서 차단. 결과 수정/재개는 task.result.amend 권한(Underboss 이상)만 가능하며 사유와 변경 전 값이 이력에 남음. 기록된 참여 결과의 재수정도 동일.
BR-07	이미 사용 중인 자원은 동일 시간대에 다른 업무 또는 사용자에게 중복 배정될 수 없다.	배정·기간변경·요청승인·업무 일정변경 시 자원 행을 FOR UPDATE 로 잠그고 RESERVED/IN_USE 배정과 시간 겹침 검사. 체크아웃 시 미반납 사용자가 있으면 거부.
BR-08	사용 불가 상태의 자원은 새로운 업무에 배정할 수 없다.	NEEDS_INSPECTION / MAINTENANCE / UNAVAILABLE / RETIRED 자원은 배정·기간변경·사용요청·체크아웃 모두 거부.
BR-09	권한이 없는 사용자는 제한된 조직 정보 또는 관리 기능에 직접 URL 로 접근하더라도 작업을 수행할 수 없어야 한다.	모든 보호 API 에 인증 미들웨어 + 권한 코드 검사 + 데이터 범위 검사를 서버에서 적용(화면 표시와 무관). 매 요청마다 DB 에서 최신 역할/상태/잠금/토큰버전을 확인. 거부 시도는 감사 로그에 DENIED 로 기록.
BR-10	구성원의 역할, 소속, 상태 및 평가와 관련된 주요 변경 이력은 추후 확인할 수 있어야 한다.	역할·소속·상태·평가·평판·징계 변경을 MongoDB member_history 에 이전 값/새 값/사유/변경자/시각과 함께 기록. 소속 이력은 MySQL crew_memberships(joined_at/left_at)에도 보존.

업무 규칙 위반 응답 예: `409 { "success": false, "error": { "code": "BUSINESS_RULE_VIOLATION", "message": "...", "details": { "rule": "BR-03", "conflicts": [...] } } }`

8. API 공통 규격

8.1 응답 형식 (AR-05, NFR-05)

```
성공: { "success": true, "message": "...", "data": { ... }, "meta": { "page": 1, "size": 20, "total": 42, "totalPages": 3 } }
실패: { "success": false, "error": { "code": "VALIDATION_ERROR", "message": "입력값이 올바르지 않습니다.", "details": [ ... ] } }
```

8.2 오류 코드 (AR-04)

HTTP	code	의미
400	VALIDATION_ERROR	필수값 누락·형식 오류·잘못된 JSON·허용되지 않는 정렬 기준 (details 에 필드별 사유)
401	UNAUTHORIZED	토큰 없음/만료/로그아웃/강제 로그아웃/잠긴 계정
403	FORBIDDEN	권한 코드 없음 또는 관리 범위 밖 (details.rule: BR-09, BR-04, BR-05)
404	NOT_FOUND	대상 없음 / 존재하지 않는 API
409	BUSINESS_RULE_VIOLATION / CONFLICT	업무 규칙 위반(details.rule) / 중복 데이터
422	INVALID_STATE	현재 상태에서 허용되지 않는 전이·작업

8.3 인증 · 목록 조회

- 인증: `Authorization: Bearer {JWT}` (EJS 화면은 httpOnly 쿠키). 매 요청마다 DB 에서 최신 역할·상태·잠금·토큰 버전을 확인한다.
- 목록: `page`, `size`(최대 100), `sort`(허용 필드, `-` 접두사 = 내림차순) + 리소스별 필터. 다중 값 필터는 쉼표로 구분 (예: `status=PLANNED,IN_PROGRESS`).
- 일시: ISO-8601 (UTC). 예) `2026-10-20T09:00:00Z`

9. 요구사항 추적표

요구사항별로 구현 API, 구현 위치(소스 파일:줄), 확인 방법을 정리하였다. 확인 방법의 상세 절차는 10장에 있다.

기능 요구사항

Requirement ID	Method	API	구현 위치	확인방법

FR-01	POST	`/api/auth/register` API-001 회원 가입 신청 (Associate, 가입대기 상태로 등록)	`routes/auth.js:62`	10장 FR-01 절차 (4단계) POST /api/members → 201 / PATCH /api/members/20 → 200
	PUT	`/api/auth/password` API-005 내 비밀번호 변경 (다른 세션은 모두 만료)	`routes/auth.js:120`	
	PATCH	`/api/me` API-009 내 연락처/별칭 수정 (역할·상태·소속은 변경 불가)	`routes/me.js:60`	
	POST	`/api/members` API-035 구성원 등록 (역할/상태/소속 Crew 지정 가능)	`routes/members.js:137`	
	PATCH	`/api/members/:id` API-036 구성원 기본 정보 수정 (이름/별칭/연락처/생년월일/메모)	`routes/members.js:179`	
	DELETE	`/api/members/:id` API-037 가입대기 구성원 삭제 (활동 이력이 없는 PENDING 계정만)	`routes/members.js:203`	
	GET	`/api/members/:id/skills` API-048 구성원 보유 기술 목록	`routes/members.js:442`	
	PUT	`/api/members/:id/skills/:skill` API-049 구성원 기술 등록/수준 변경 (본인 또는 관리자)	`routes/members.js:454`	
	DELETE	`/api/members/:id/skills/:skill` API-050 구성원 기술 삭제	`routes/members.js:471`	

	POST	`/api/crews/:id/join-requests` API-083 Crew 가입 요청 (본인 신청, 가입대기 계정도 가능)	`routes/joinRequests.js:33`	
	POST	`/api/join-requests/:id/approve` API-086 가입 요청 승인 (Crew 편성 + 가입대기 계정은 활동중으로 전환)	`routes/joinRequests.js:95`	
	PATCH	`/api/admin/accounts/:id/username` API-211 로그인 아이디 변경	`routes/admin.js:122`	

FR-02	GET	`/api/auth/login-history` API-006 내 로그인/로그아웃 이력 조회	`routes/auth.js:136`	10장 FR-02 절차 (4단계) GET /api/me → 200 / GET /api/auth/permissions → 200
	GET	`/api/auth/permissions` API-007 내 역할, 권한 목록, 관리 범위(Crew) 조회	`routes/auth.js:153`	
	GET	`/api/me` API-008 내 프로필 조회 (현재 소속, 역할, 상태, 활동 요약)	`routes/me.js:37`	
	PATCH	`/api/me` API-009 내 연락처/별칭 수정 (역할·상태·소속은 변경 불가)	`routes/me.js:60`	
	GET	`/api/me/dashboard` API-014 내 대시보드 (7일 내 일정, 미읽은 알림, 대기 중 요청, 사용 중 자원)	`routes/me.js:191`	
	GET	`/api/members/:id` API-034 구성원 상세 조회 (소속 Crew, 기술 포함)	`routes/members.js:118`	
	GET	`/api/crews/:id/members` API-071 Crew 구성원 목록 (정식/준소속 구분)	`routes/crews.js:256`	
	GET	`/api/crews/:id/announcements` API-089 Crew 공지 목록 (고정 공지 우선)	`routes/announcements.js:22`	
	GET	`/api/activities` API-186 활동 기록 조회 (본인 또는 관리 범위 구성원)	`routes/activities.js:14`	

FR-03	PATCH	`/api/members/:id/role` API-038 구성원 역할 변경 (자신보다 높은 권한자 변경 불가)	`routes/members.js:225`	10장 FR-03 절차 (4단계) PATCH /api/members/14/role → 200 / PATCH /api/members/9/status → 200
	PATCH	`/api/members/:id/status` API-039 구성원 상태 변경 (활동중/대기/정지/탈퇴)	`routes/members.js:244`	
	POST	`/api/promotion-requests` API-051 승급 요청 등록	`routes/promotions.js:41`	
	GET	`/api/promotion-requests` API-052 승급 요청 목록 (내가 요청했거나 심사 가능한 요청)	`routes/promotions.js:68`	
	GET	`/api/promotion-requests/:id` API-053 승급 요청 상세	`routes/promotions.js:95`	
	POST	`/api/promotion-requests/:id/approve` API-054 승급 요청 승인 (역할 변경 실행 + 이력 기록)	`routes/promotions.js:107`	
	POST	`/api/promotion-requests/:id/reject` API-055 승급 요청 반려 (사유 필수)	`routes/promotions.js:130`	
	DELETE	`/api/promotion-requests/:id` API-056 승급 요청 취소 (요청자 본인, 대기 상태만)	`routes/promotions.js:145`	

	POST	`/api/members/:id/disciplinary-actions` API-057 구성원 징계 등록 (SUSPENSION 은 상태를 정지로 변경)	`routes/discipline.js:25`	
	POST	`/api/crews/:id/members/:memberId/transfer` API-074 구성원을 다른 Crew 로 이동 (정식 소속 원자적 변경)	`routes/crews.js:320`	

FR-04	POST	`/api/auth/register` API-001 회원 가입 신청 (Associate, 가입대기 상태로 등록)	`routes/auth.js:62`	10장 FR-04 절차 (4단계) GET /api/members/8/status-transitions → 200 / PATCH /api/members/18/status → 422
	POST	`/api/members` API-035 구성원 등록 (역할/상태/소속 Crew 지정 가능)	`routes/members.js:137`	
	DELETE	`/api/members/:id` API-037 가입대기 구성원 삭제 (활동 이력이 없는 PENDING 계정만)	`routes/members.js:203`	
	PATCH	`/api/members/:id/status` API-039 구성원 상태 변경 (활동중/대기/정지/탈퇴)	`routes/members.js:244`	
	GET	`/api/members/:id/status-transitions` API-040 구성원의 현재 상태에서 요청자가 수행 가능한 상태 전이 목록	`routes/members.js:266`	
	POST	`/api/members/:id/disciplinary-actions` API-057 구성원 징계 등록 (SUSPENSION 은 상태를 정지로 변경)	`routes/discipline.js:25`	
	POST	`/api/disciplinary-actions/:id/revoke` API-060 징계 해제 (정지 징계 해제 시 다른 정지 징계가 없으면 상태를 ACTIVE 로 복구)	`routes/discipline.js:98`	

	POST	`/api/join-requests/:id/approve` API-086 가입 요청 승인 (Crew 편성 + 가입대기 계정은 활동중으로 전환)	`routes/joinRequests.js:95`	
	GET	`/api/stats/members` API-192 구성원 현황 (상태별·역할별·Crew별 분포)	`routes/stats.js:74`	
	GET	`/api/stats/status-changes` API-201 구성원 변경 추이 (역할·상태·소속·평가 변경 이력의 주별 집계)	`routes/stats.js:287`	

FR-05	GET	`/api/members` API-032 구성원 검색/목록 (권한 범위 내, 필터·정렬·페이지)	`routes/members.js:80`	10장 FR-05 절차 (5단계) GET /api/crews → 200 / GET /api/crews/1 → 200
	GET	`/api/members/:id` API-034 구성원 상세 조회 (소속 Crew, 기술 포함)	`routes/members.js:118`	
	GET	`/api/crews` API-061 Crew 목록/검색 (지역·상태·책임자 필터)	`routes/crews.js:58`	
	GET	`/api/crews/:id` API-063 Crew 상세 (책임자, 감독자, 인원, 진행 업무 수)	`routes/crews.js:93`	
	POST	`/api/crews` API-064 Crew 생성	`routes/crews.js:109`	
	PATCH	`/api/crews/:id` API-065 Crew 정보 수정 (이름/지역/설명)	`routes/crews.js:131`	
	DELETE	`/api/crews/:id` API-066 Crew 해산 (정식 구성원이 없고 진행 업무가 없을 때만)	`routes/crews.js:152`	
	POST	`/api/crews/:id/restore` API-067 해산된 Crew 복구	`routes/crews.js:174`	
	PUT	`/api/crews/:id/capo` API-068 Crew 책임자(Capo) 지정	`routes/crews.js:188`	
	DELETE	`/api/crews/:id/capo` API-069 Crew 책임자 해제 (공석 처리)	`routes/crews.js:221`	

PUT	`/api/crews/:id/underboss` API-070 Crew 감독 Underboss 지정/변경	`routes/crews.js:238`
GET	`/api/crews/:id/members` API-071 Crew 구성원 목록 (정식/준 소속 구분)	`routes/crews.js:256`
GET	`/api/crews/:id/stats` API-076 Crew 운영 현황 (구성원 상태 분포, 업무 상태 분포, 완료율, 평균 기여도)	`routes/crews.js:375`
GET	`/api/regions` API-077 지역 목록 (활성 Crew 수 포함)	`routes/regions.js:19`
GET	`/api/regions/:id` API-078 지역 상세 (소속 Crew 목록 포함)	`routes/regions.js:26`
POST	`/api/regions` API-079 지역 등록	`routes/regions.js:38`
PATCH	`/api/regions/:id` API-080 지역 정보 수정	`routes/regions.js:53`
DELETE	`/api/regions/:id` API-081 지역 삭제 (연결된 Crew 가 없을 때만)	`routes/regions.js:68`
GET	`/api/crews/:id/announcements` API-089 Crew 공지 목록 (고정 공지 우선)	`routes/announcements.js:22`

GET	`/api/crews/:id/announcements/:annId` API-090 Crew 공지 상세	`routes/announcements.js:46`	
POST	`/api/crews/:id/announcements` API-091 Crew 공지 등록 (소속 구성원에게 알림)	`routes/announcements.js:57`	
PATCH	`/api/crews/:id/announcements/:annId` API-092 Crew 공지 수정 / 고정 설정	`routes/announcements.js:78`	
DELETE	`/api/crews/:id/announcements/:annId` API-093 Crew 공지 삭제	`routes/announcements.js:96`	
POST	`/api/meetings` API-135 회의 생성 (참석자 초대 및 알림)	`routes/meetings.js:83`	

FR-06	GET	`/api/me/inbox` API-015 내 처리 대기함 (내가 응답·승인해야 할 항목 집계)	`routes/me.js:211`	10장 FR-06 절차 (4단계) DELETE /api/crews/1/members/14?reason=편성 조정 → 200 / GET /api/crews/1/membership-history → 200
	GET	`/api/crews/managed` API-062 내가 관리할 수 있는 Crew 목록	`routes/crews.js:82`	
	PUT	`/api/crews/:id/underboss` API-070 Crew 감독 Underboss 지정/변경	`routes/crews.js:238`	
	POST	`/api/crews/:id/members` API-072 Crew 에 구성원 편성 (정식/준소속)	`routes/crews.js:278`	
	DELETE	`/api/crews/:id/members/:memberId` API-073 Crew 에서 구성원 제외	`routes/crews.js:301`	
	POST	`/api/crews/:id/members/:memberId/transfer` API-074 구성원을 다른 Crew 로 이동 (정식 소속 원자적 변경)	`routes/crews.js:320`	
	GET	`/api/crews/:id/membership-history` API-075 Crew 구성원 편성 변경 이력 (가입/제외/이동)	`routes/crews.js:350`	
	POST	`/api/crews/:id/join-requests` API-083 Crew 가입 요청 (본인 신청, 가입대기 계정도 가능)	`routes/joinRequests.js:33`	

GET	`/api/join-requests` API-084 가입 요청 목록 (내 요청 또는 관리 Crew 로 들어온 요청)	`routes/joinRequests.js:60`	
GET	`/api/join-requests/:id` API-085 가입 요청 상세	`routes/joinRequests.js:83`	
POST	`/api/join-requests/:id/approve` API-086 가입 요청 승인 (Crew 편성 + 가입대기 계정은 활동중으로 전환)	`routes/joinRequests.js:95`	
POST	`/api/join-requests/:id/reject` API-087 가입 요청 반려 (사유 필수)	`routes/joinRequests.js:117`	
DELETE	`/api/join-requests/:id` API-088 내 가입 요청 취소 (대기 상태만)	`routes/joinRequests.js:131`	
POST	`/api/crews/:id/announcements` API-091 Crew 공지 등록 (소속 구성원에게 알림)	`routes/announcements.js:57`	

FR-07	**GET**	`/api/tasks/:id` API-097 업무 상세 (참여 구성원, 배정 자원 포함)	`routes/tasks.js:112`	10장 FR-07 절차 (5단계) POST /api/tasks → 201 / PATCH /api/tasks/25 → 200
	POST	`/api/tasks` API-098 업무 등록 (기본 정보, 일정, 중요도)	`routes/tasks.js:127`	
	PATCH	`/api/tasks/:id` API-099 업무 기본 정보 수정 (제목/설명/중요도/장소/필요 기술)	`routes/tasks.js:151`	
	PATCH	`/api/tasks/:id/schedule` API-100 업무 일정 변경 (참여자 시간 중복·자원 중복 재검사)	`routes/tasks.js:176`	
	PATCH	`/api/tasks/:id/status` API-101 업무 진행 상태 변경 (전이 규칙 검사, 이력 기록)	`routes/tasks.js:217`	
	POST	`/api/tasks/:id/duplicate` API-108 업무 복제 (새 일정으로 PLANNED 업무 생성, 체크리스트 포함)	`routes/tasks.js:398`	
	DELETE	`/api/tasks/:id` API-109 업무 삭제 (배정·자원 이력이 없는 PLANNED/CANCELLED 업무만)	`routes/tasks.js:421`	
	POST	`/api/tasks/:id/checklist` API-124 체크리스트 항목 추가 (업무 관리자)	`routes/taskCollab.js:100`	

DELETE	`/api/tasks/:id/checklist/:itemId` API-126 체크리스트 항목 삭제 (업무 관리자)	`routes/taskCollab.js:143`	
GET	`/api/task-templates` API-127 업무 템플릿 목록	`routes/templates.js:36`	
GET	`/api/task-templates/:id` API-128 업무 템플릿 상세	`routes/templates.js:46`	
POST	`/api/task-templates` API-129 업무 템플릿 등록	`routes/templates.js:54`	
PATCH	`/api/task-templates/:id` API-130 업무 템플릿 수정	`routes/templates.js:66`	
DELETE	`/api/task-templates/:id` API-131 업무 템플릿 삭제	`routes/templates.js:80`	
POST	`/api/task-templates/:id/instantiate` API-132 템플릿으로 업무 생성 (기본값 + 체크리스트 자동 적용)	`routes/templates.js:92`	

FR-08	GET	`/api/me/tasks` API-010 나에게 배정된 업무 목록 (확인 필요 사항 포함)	`routes/me.js:76`	10장 FR-08 절차 (3단계) GET /api/tasks/2/assignments → 200
	GET	`/api/me/tasks/:taskId` API-011 내 업무 브리핑 (내 역할/지시사항, 동료, 자원, 체크리스트, 최근 코멘트)	`routes/me.js:126`	
	POST	`/api/me/assignments/:assignmentId/accept` API-012 배정된 업무 수락	`routes/me.js:160`	
	POST	`/api/me/assignments/:assignmentId/decline` API-013 배정된 업무 거절 (사유 필수, Crew 책임자에게 알림)	`routes/me.js:175`	
	GET	`/api/members/:id/availability` API-044 특정 시간대 배정 가능 여부 사전 점검 (상태·시간 중복)	`routes/members.js:354`	
	GET	`/api/members/:id/skills` API-048 구성원 보유 기술 목록	`routes/members.js:442`	
	GET	`/api/tasks/:id` API-097 업무 상세 (참여 구성원, 배정 자원 포함)	`routes/tasks.js:112`	
	GET	`/api/tasks/:id/candidates` API-111 배정 후보 추천 (상태·시간 충돌 없는 구성원, 기술 적합도·실무 인력·업무량 순)	`routes/tasks.js:460`	

GET	`/api/tasks/:id/assignments` API-113 업무 참여 구성원 목록 (배정 상태 포함)	`routes/assignments.js:41`	
POST	`/api/tasks/:id/assignments` API-114 업무에 구성원 배정	`routes/assignments.js:58`	
PATCH	`/api/tasks/:id/assignments/:memberId` API-115 배정 정보 수정 (업무 내 역할, 지시사항)	`routes/assignments.js:83`	
DELETE	`/api/tasks/:id/assignments/:memberId` API-116 배정 해제	`routes/assignments.js:106`	
POST	`/api/tasks/:id/assignments/:memberId/replace` API-117 배정 인원 교체 (기존 해제 + 신규 배정을 하나의 트랜잭션으로)	`routes/assignments.js:125`	
GET	`/api/stats/members/workload` API-196 구성원별 업무량 (기간 내 배정 건수·배정 시간·진행중 업무)	`routes/stats.js:174`	

FR-09	GET	`/api/me/tasks` API-010 나에게 배정된 업무 목록 (확인 필요 사항 포함)	`routes/me.js:76`	10장 FR-09 절차 (5단계) GET /api/me/tasks → 200 / GET /api/me/tasks/1 → 200
	GET	`/api/me/tasks/:taskId` API-011 내 업무 브리핑 (내 역할/지시사항, 동료, 자원, 체크리스트, 최근 코멘트)	`routes/me.js:126`	
	POST	`/api/me/assignments/:assignmentId/accept` API-012 배정된 업무 수락	`routes/me.js:160`	
	POST	`/api/me/assignments/:assignmentId/decline` API-013 배정된 업무 거절 (사유 필수, Crew 책임자에게 알림)	`routes/me.js:175`	
	GET	`/api/me/dashboard` API-014 내 대시보드 (7일 내 일정, 미읽은 알림, 대기 중 요청, 사용 중 자원)	`routes/me.js:191`	
	GET	`/api/me/inbox` API-015 내 처리 대기함 (내가 응답·승인해야 할 항목 집계)	`routes/me.js:211`	
	GET	`/api/me/notifications` API-016 내 알림 목록	`routes/me.js:262`	
	PATCH	`/api/me/notifications/read-all` API-017 내 알림 모두 읽음 처리	`routes/me.js:277`	

PATCH	`/api/me/notifications/:id/read` API-018 알림 1건 읽음 처리	`routes/me.js:286`
DELETE	`/api/me/notifications/:id` API-019 알림 1건 삭제	`routes/me.js:298`
GET	`/api/me/notification-settings` API-020 내 알림 수신 설정 조회	`routes/me.js:310`
PUT	`/api/me/notification-settings` API-021 내 알림 수신 설정 변경 (수신 거부할 알림 유형 지정)	`routes/me.js:319`
GET	`/api/members/:id/schedule` API-043 구성원 일정 타임라인 (업무·자원 사용·회의)	`routes/members.js:324`
GET	`/api/tasks/calendar` API-096 업무 캘린더 (일자별 그룹)	`routes/tasks.js:88`
PATCH	`/api/tasks/:id/assignments/:memberId` API-115 배정 정보 수정 (업무 내 역할, 지시사항)	`routes/assignments.js:83`
GET	`/api/tasks/:id/comments` API-119 업무 코멘트 목록	`routes/taskCollab.js:28`
POST	`/api/tasks/:id/comments` API-120 업무 코멘트 작성 (업무 조회 권한자)	`routes/taskCollab.js:42`

PATCH	`/api/tasks/:id/comments/:commentId` API-121 내 코멘트 수정 (작성자 본인)	`routes/taskCollab.js:56`
DELETE	`/api/tasks/:id/comments/:commentId` API-122 코멘트 삭제 (작성자 또는 업무 관리자)	`routes/taskCollab.js:71`
GET	`/api/tasks/:id/checklist` API-123 업무 체크리스트 조회 (진행률 포함)	`routes/taskCollab.js:89`
POST	`/api/tasks/:id/checklist` API-124 체크리스트 항목 추가 (업무 관리자)	`routes/taskCollab.js:100`
PATCH	`/api/tasks/:id/checklist/:itemId` API-125 체크리스트 항목 완료 체크/수정 (참여자: 체크만, 관리자: 문구 수정)	`routes/taskCollab.js:118`
GET	`/api/meetings` API-133 회의 목록 (내가 참석/주관하거나 관리 Crew 회의)	`routes/meetings.js:41`
GET	`/api/meetings/:id` API-134 회의 상세 (참석자 응답 현황, 회의록 포함)	`routes/meetings.js:67`
POST	`/api/meetings` API-135 회의 생성 (참석자 초대 및 알림)	`routes/meetings.js:83`
PATCH	`/api/meetings/:id` API-136 회의 정보/일정 수정	`routes/meetings.js:116`

	DELETE	`/api/meetings/:id` API-137 회의 취소	`routes/meetings.js:142`	
	POST	`/api/meetings/:id/attendees` API-138 회의 참석자 추가	`routes/meetings.js:158`	
	DELETE	`/api/meetings/:id/attendees/:memberId` API-139 회의 참석자 제외	`routes/meetings.js:177`	
	POST	`/api/meetings/:id/rsvp` API-140 회의 참석 여부 응답 (초대된 본인)	`routes/meetings.js:193`	

FR-10	PATCH	`/api/tasks/:id/status` API-101 업무 진행 상태 변경 (전이 규칙 검사, 이력 기록)	`routes/tasks.js:217`	10장 FR-10 절차 (4단계) GET /api/tasks/9/transitions → 200 / PATCH /api/tasks/9/status → 422
	GET	`/api/tasks/:id/transitions` API-102 현재 상태에서 가능한 다음 상태 목록	`routes/tasks.js:257`	
	GET	`/api/tasks/:id/status-history` API-103 업무 상태·배정 변경 이력 (MongoDB)	`routes/tasks.js:271`	
	POST	`/api/tasks/:id/complete` API-104 업무 완료 처리 (결과 및 참여 기록 입력)	`routes/tasks.js:288`	
	POST	`/api/tasks/:id/reopen` API-107 완료된 업무 재개 (Underboss 이상, COMPLETED → IN_PROGRESS)	`routes/tasks.js:382`	
	GET	`/api/tasks/:id/timeline` API-110 업무 통합 타임라인 (상태 이력 + 코멘트 + 자원 사용 이벤트)	`routes/tasks.js:439`	
	GET	`/api/tasks/:id/checklist` API-123 업무 체크리스트 조회 (진행률 포함)	`routes/taskCollab.js:89`	
	PATCH	`/api/tasks/:id/checklist/:itemId` API-125 체크리스트 항목 완료 체크/수정 (참여자: 체크만, 관리자: 문구 수정)	`routes/taskCollab.js:118`	

FR-11	GET	`/api/me/tasks/:taskId` API-011 내 업무 브리핑 (내 역할/지시사항, 동료, 자원, 체크리스트, 최근 코멘트)	`routes/me.js:126`	10장 FR-11 절차 (5단계) POST /api/tasks/3/complete → 200 / GET /api/tasks/3/result → 200
	GET	`/api/members/:id/assignments` API-042 구성원의 업무 참여 이력 (참여 기록·기여도 포함)	`routes/members.js:298`	
	GET	`/api/crews/:id/stats` API-076 Crew 운영 현황 (구성원 상태 분포, 업무 상태 분포, 완료율, 평균 기여도)	`routes/crews.js:375`	
	POST	`/api/tasks/:id/complete` API-104 업무 완료 처리 (결과 및 참여 기록 입력)	`routes/tasks.js:288`	
	GET	`/api/tasks/:id/result` API-105 업무 결과 및 참여 기록 조회	`routes/tasks.js:332`	
	PATCH	`/api/tasks/:id/result` API-106 완료된 업무 결과 수정 (Underboss 이상, 사유 필수, 수정 전 값 이력 보존)	`routes/tasks.js:355`	
	PUT	`/api/tasks/:id/assignments/:memberId/participation` API-118 완료 업무의 참여 기록(출석·기여도) 입력	`routes/assignments.js:149`	

GET	`/api/evaluations/pending` API-175 평가 대기 목록 (관리 범위의 최근 완료 업무 참여자 중 미평가)	`routes/evaluations.js:101`	
POST	`/api/evaluations` API-177 평가 등록 (업무 결과 기반, 항목별 1~5점, 가중 평균 자동 계산)	`routes/evaluations.js:129`	
GET	`/api/stats/crews` API-193 Crew 별 운영 비교 (인원, 업무 완료율, 성공률, 평균 기여도, 성과값)	`routes/stats.js:100`	
GET	`/api/stats/tasks` API-194 업무 현황 (기간 내 상태별·우선순위별·결과별 분포)	`routes/stats.js:131`	

FR-12	GET	`/api/tasks` API-094 업무 탐색 (기간·Crew·상태·우선순위·담당자·검색어, 정렬, 페이지)	`routes/tasks.js:57`	10장 FR-12 절차 (5단계) GET /api/tasks?crewId=1&status=PLANNED,IN_PROGRESS,ON_HOLD&sort=startAt → 200 / GET /api/tasks?memberId=8 → 200
	GET	`/api/tasks/export` API-095 업무 목록 CSV 내보내기 (탐색 조건 동일 적용)	`routes/tasks.js:75`	
	GET	`/api/tasks/calendar` API-096 업무 캘린더 (일자별 그룹)	`routes/tasks.js:88`	
	GET	`/api/tasks/:id/conflicts` API-112 업무 충돌 진단 (참여자 일정 중복 BR-03, 자원 중복 BR-07)	`routes/tasks.js:497`	
	GET	`/api/stats/tasks` API-194 업무 현황 (기간 내 상태별·우선순위별·결과별 분포)	`routes/stats.js:131`	

FR-13	GET	`/api/resources` API-142 자원 목록/검색 (유형·상태·Crew·검색어)	`routes/resources.js:41`	10장 FR-13 절차 (4단계) POST /api/resources → 201 / PATCH /api/resources/10/status → 200
	GET	`/api/resources/attention` API-144 관리 필요 자원 현황 (점검 필요/정비중/사용 불가 + 미해결 이슈 + 반납 지연)	`routes/resources.js:95`	
	GET	`/api/resources/:id` API-145 자원 상세 (현재 사용 현황, 다음 예약, 미해결 이슈)	`routes/resources.js:117`	
	POST	`/api/resources` API-146 자원 등록	`routes/resources.js:133`	
	PATCH	`/api/resources/:id` API-147 자원 정보 수정 (이름/위치/소유 Crew/이용 등급/설명)	`routes/resources.js:160`	
	DELETE	`/api/resources/:id` API-148 자원 삭제 (사용 이력이 전혀 없는 경우만, 그 외는 RETIRED 상태로 관리)	`routes/resources.js:184`	
	PATCH	`/api/resources/:id/status` API-149 자원 상태 변경 (정상/점검 필요/정비중/사용 불가/폐기)	`routes/resources.js:200`	
	GET	`/api/resource-types` API-152 자원 유형 목록 (유형별 보유/가용 수량)	`routes/resourceTypes.js:20`	

GET	`/api/resource-types/:id` API-153 자원 유형 상세	`routes/resourceTypes.js:26`	
POST	`/api/resource-types` API-154 자원 유형 등록	`routes/resourceTypes.js:34`	
PATCH	`/api/resource-types/:id` API-155 자원 유형 수정	`routes/resourceTypes.js:52`	
DELETE	`/api/resource-types/:id` API-156 자원 유형 삭제 (해당 유형 자원이 없을 때만)	`routes/resourceTypes.js:71`	
POST	`/api/resource-issues/:id/resolve` API-173 자원 이슈 해결 처리 (다른 미해결 이슈가 없으면 자원 상태 복구)	`routes/resourceIssues.js:83`	

FR-14	**GET**	`/api/me/dashboard` API-014 내 대시보드 (7일 내 일정, 미읽은 알림, 대기 중 요청, 사용 중 자원)	`routes/me.js:191`	10장 FR-14 절차 (4단계) GET /api/resources → 200 / GET /api/resources/available?startAt=2030-01-01T00:00:00Z&endAt=2030-01-01T02:00:00Z → 200
	GET	`/api/resources` API-142 자원 목록/검색 (유형·상태·Crew·검색어)	`routes/resources.js:41`	
	GET	`/api/resources/available` API-143 지정 시간대에 내가 이용 가능한 자원 (권한·상태·기존 배정 고려)	`routes/resources.js:71`	
	GET	`/api/resources/:id/availability` API-151 자원의 기간 내 사용 가능/불가 구간 (예약 현황)	`routes/resources.js:255`	
	GET	`/api/resource-types` API-152 자원 유형 목록 (유형별 보유/가용 수량)	`routes/resourceTypes.js:20`	
	POST	`/api/resource-requests` API-164 자원 사용 요청 (권한 등급·상태·시간 중복 사전 검사)	`routes/resourceRequests.js:34`	
	GET	`/api/resource-requests` API-165 자원 사용 요청 목록 (내 요청 또는 심사 가능한 요청)	`routes/resourceRequests.js:63`	
	GET	`/api/resource-requests/:id` API-166 자원 사용 요청 상세	`routes/resourceRequests.js:87`	

	DELETE	`/api/resource-requests/:id` API-169 내 자원 사용 요청 취소 (대기 상태만)	`routes/resourceRequests.js:135`	

FR-15	GET	`/api/tasks/:id/timeline` API-110 업무 통합 타임라인 (상태 이력 + 코멘트 + 자원 사용 이벤트)	`routes/tasks.js:439`	10장 FR-15 절차 (5단계) POST /api/allocations → 201 / POST /api/allocations/4/checkout → 200
	PATCH	`/api/resources/:id/status` API-149 자원 상태 변경 (정상/점검 필요/정비중/사용 불가/폐기)	`routes/resources.js:200`	
	GET	`/api/resources/:id/history` API-150 자원 사용·반납·상태 변경 이력 (MongoDB)	`routes/resources.js:237`	
	GET	`/api/allocations` API-157 자원 배정/사용 기록 조회 (자원·업무·구성원·상태·기간)	`routes/allocations.js:42`	
	GET	`/api/allocations/:id` API-158 자원 배정 상세	`routes/allocations.js:82`	
	DELETE	`/api/allocations/:id` API-161 자원 예약 취소 (RESERVED 상태만)	`routes/allocations.js:149`	
	POST	`/api/allocations/:id/checkout` API-162 자원 사용 시작 (체크아웃: RESERVED → IN_USE)	`routes/allocations.js:164`	
	POST	`/api/allocations/:id/return` API-163 자원 반납 (상태 기록, 파손 시 점검 필요 / 분실 시 사용 불가 자동 전환)	`routes/allocations.js:189`	

	POST	`/api/resources/:id/issues`	`routes/resourceIssues.js:25`	API-170 자원 이상 신고 (정상 자원은 점검 필요 상태로 자동 전환)
	GET	`/api/resource-issues`	`routes/resourceIssues.js:52`	API-171 자원 이슈 목록 (상태·심각도·자원)
	POST	`/api/resource-issues/:id/resolve`	`routes/resourceIssues.js:83`	API-173 자원 이슈 해결 처리 (다른 미해결 이슈가 없으면 자원 상태 복구)
	GET	`/api/stats/resources/utilization`	`routes/stats.js:196`	API-197 자원 가동률 (기간 대비 배정·사용 시간 비율)

FR-16	GET	`/api/resources` API-142 자원 목록/검색 (유형·상태·Crew·검색어)	`routes/resources.js:41`	10장 FR-16 절차 (5단계) GET /api/resources/attention → 200 / GET /api/resources?usable=false → 200
	GET	`/api/resources/attention` API-144 관리 필요 자원 현황 (점검 필요/정비중/사용 불가 + 미해결 이슈 + 반납 지연)	`routes/resources.js:95`	
	PATCH	`/api/resources/:id/status` API-149 자원 상태 변경 (정상/점검 필요/정비중/사용 불가/폐기)	`routes/resources.js:200`	
	POST	`/api/allocations/:id/return` API-163 자원 반납 (상태 기록, 파손 시 점검 필요 / 분실 시 사용 불가 자동 전환)	`routes/allocations.js:189`	
	POST	`/api/resources/:id/issues` API-170 자원 이상 신고 (정상 자원은 점검 필요 상태로 자동 전환)	`routes/resourceIssues.js:25`	
	GET	`/api/resource-issues` API-171 자원 이슈 목록 (상태·심각도·자원)	`routes/resourceIssues.js:52`	
	GET	`/api/resource-issues/:id` API-172 자원 이슈 상세	`routes/resourceIssues.js:71`	

	POST	`/api/resource-issues/:id/resolve` API-173 자원 이슈 해결 처리 (다른 미해결 이슈가 없으면 자원 상태 복구)	`routes/resourceIssues.js:83`	
	GET	`/api/stats/resources/issues` API-198 자원 이슈 통계 (심각도·상태·유형별, 평균 해결 시간)	`routes/stats.js:215`	

FR-17	GET	`/api/me/inbox` API-015 내 처리 대기함 (내가 응답·승인해야 할 항목 집계)	`routes/me.js:211`	10장 FR-17 절차 (3단계) POST /api/allocations → 201 / POST /api/resource-requests/1/approve → 200
	GET	`/api/allocations` API-157 자원 배정/사용 기록 조회 (자원·업무·구성원·상태·기간)	`routes/allocations.js:42`	
	GET	`/api/allocations/:id` API-158 자원 배정 상세	`routes/allocations.js:82`	
	POST	`/api/allocations` API-159 자원을 업무 또는 구성원에게 배정 (예약)	`routes/allocations.js:94`	
	PATCH	`/api/allocations/:id` API-160 예약 기간 변경 (중복 재검사)	`routes/allocations.js:122`	
	DELETE	`/api/allocations/:id` API-161 자원 예약 취소 (RESERVED 상태만)	`routes/allocations.js:149`	
	POST	`/api/resource-requests` API-164 자원 사용 요청 (권한 등급·상태·시간 중복 사전 검사)	`routes/resourceRequests.js:34`	
	GET	`/api/resource-requests` API-165 자원 사용 요청 목록 (내 요청 또는 심사 가능한 요청)	`routes/resourceRequests.js:63`	

	POST	`/api/resource-requests/:id/approve` API-167 자원 사용 요청 승인 (배정 생성 시 BR-07/BR-08 재검사)	`routes/resourceRequests.js:99`	
	POST	`/api/resource-requests/:id/reject` API-168 자원 사용 요청 반려 (사유 필수)	`routes/resourceRequests.js:121`	

FR-18	GET	`/api/me/inbox` API-015 내 처리 대기함 (내가 응답·승인해야 할 항목 집계)	`routes/me.js:211`	10장 FR-18 절차 (4단계) POST /api/evaluations → 201 / GET /api/evaluations → 200
	GET	`/api/members/:id/reputation` API-045 구성원 평판 점수 (평가 평균, 가감점, 출석률)	`routes/members.js:375`	
	POST	`/api/members/:id/reputation-adjustments` API-046 평판 가감점 부여 (공로/과실)	`routes/members.js:392`	
	PUT	`/api/tasks/:id/assignments/:memberId/participation` API-118 완료 업무의 참여 기록(출석·기여도) 입력	`routes/assignments.js:149`	
	GET	`/api/evaluations` API-174 평가 이력 조회 (구성원·업무·평가자·기간)	`routes/evaluations.js:59`	
	GET	`/api/evaluations/pending` API-175 평가 대기 목록 (관리 범위의 최근 완료 업무 참여자 중 미평가)	`routes/evaluations.js:101`	
	POST	`/api/evaluations` API-177 평가 등록 (업무 결과 기반, 항목별 1~5점, 가중평균 자동 계산)	`routes/evaluations.js:129`	
	PATCH	`/api/evaluations/:id` API-178 평가 수정 (작성자: 설정된 기간 내 / Underboss 이상: 언제나, 수정 전 내용 보존)	`routes/evaluations.js:164`	

DELETE	`/api/evaluations/:id` API-179 평가 무효화 (삭제하지 않고 VOIDED 처리, 사유 기록)	`routes/evaluations.js:195`	
GET	`/api/evaluation-criteria` API-181 평가 항목 목록 (코드, 가중치)	`routes/evaluations.js:228`	
POST	`/api/evaluation-criteria` API-182 평가 항목 추가	`routes/evaluations.js:235`	
PATCH	`/api/evaluation-criteria/:code` API-183 평가 항목 수정 (이름/설명/가중치/활성 여부)	`routes/evaluations.js:254`	
DELETE	`/api/evaluation-criteria/:code` API-184 평가 항목 삭제 (평가에 사용된 적 없는 항목만, 그 외는 비활성화)	`routes/evaluations.js:271`	
GET	`/api/reputation/leaderboard` API-185 평판 순위 (조회 가능한 Crew 구성원 대상)	`routes/evaluations.js:289`	
POST	`/api/activities` API-188 내 활동 보고 등록 (현장 보고·순찰·훈련 등, 유연한 상세 정보)	`routes/activities.js:60`	
GET	`/api/stats/evaluations` API-199 평가 통계 (항목별 평균, 종합 점수 분포, 월별 평균)	`routes/stats.js:233`	

FR-19	GET	`/api/members/:id/crew-history` API-041 구성원의 Crew 소속 이력 (가입/이동/탈퇴)	`routes/members.js:282`	10장 FR-19 절차 (5단계) GET /api/evaluations?memberId=8 → 200 / GET /api/members/8/reputation → 200
	GET	`/api/members/:id/assignments` API-042 구성원의 업무 참여 이력 (참여 기록·기여도 포함)	`routes/members.js:298`	
	GET	`/api/members/:id/reputation` API-045 구성원 평판 점수 (평가 평균, 가감점, 출석률)	`routes/members.js:375`	
	GET	`/api/members/:id/history` API-047 구성원 주요 변경 이력 (역할·소속·상태·평가·징계)	`routes/members.js:416`	
	GET	`/api/promotion-requests` API-052 승급 요청 목록 (내가 요청했거나 심사 가능한 요청)	`routes/promotions.js:68`	
	GET	`/api/disciplinary-actions` API-058 징계 목록 (관리 범위 내)	`routes/discipline.js:56`	
	GET	`/api/disciplinary-actions/:id` API-059 징계 상세	`routes/discipline.js:83`	
	GET	`/api/crews/:id/membership-history` API-075 Crew 구성원 편성 변경 이력 (가입/제외/이동)	`routes/crews.js:350`	

GET	`/api/tasks/:id/result` API-105 업무 결과 및 참여 기록 조회	`routes/tasks.js:332`	
GET	`/api/evaluations` API-174 평가 이력 조회 (구성원·업무·평가자·기간)	`routes/evaluations.js:59`	
GET	`/api/evaluations/:id` API-176 평가 상세	`routes/evaluations.js:121`	
GET	`/api/evaluations/:id/revisions` API-180 평가 수정 이력	`routes/evaluations.js:212`	
GET	`/api/reputation/leaderboard` API-185 평판 순위 (조회 가능한 Crew 구성원 대상)	`routes/evaluations.js:289`	
GET	`/api/activities` API-186 활동 기록 조회 (본인 또는 관리 범위 구성원)	`routes/activities.js:14`	
GET	`/api/activities/:id` API-187 활동 기록 상세	`routes/activities.js:44`	
GET	`/api/stats/evaluations` API-199 평가 통계 (항목별 평균, 종합 점수 분포, 월별 평균)	`routes/stats.js:233`	

FR-20	**GET**	`/api/auth/login-history` API-006 내 로그인/로그아웃 이력 조회	`routes/auth.js:136`	10장 FR-20 절차 (5단계) GET /api/audit-logs?action=MEMBER_ROLE_CHANGED → 200 / GET /api/audit-logs?result=DENIED → 200
	PUT	`/api/roles/:code/permissions` API-028 역할의 권한 집합 전체 교체	`routes/roles.js:67`	
	POST	`/api/roles/:code/permissions` API-029 역할에 권한 1개 부여	`routes/roles.js:91`	
	DELETE	`/api/roles/:code/permissions/:permission` API-030 역할에서 권한 1개 회수	`routes/roles.js:108`	
	PATCH	`/api/members/:id/status` API-039 구성원 상태 변경 (활동중/대기/정지/탈퇴)	`routes/members.js:244`	
	POST	`/api/members/:id/reputation-adjustments` API-046 평판 가감점 부여 (공로/과실)	`routes/members.js:392`	
	GET	`/api/members/:id/history` API-047 구성원 주요 변경 이력 (역할·소속·상태·평가·징계)	`routes/members.js:416`	
	POST	`/api/disciplinary-actions/:id/revoke` API-060 징계 해제 (정지 징계 해제 시 다른 정지 징계가 없으면 상태를 ACTIVE 로 복구)	`routes/discipline.js:98`	
	POST	`/api/crews` API-064 Crew 생성	`routes/crews.js:109`	

DELETE	`/api/crews/:id` API-066 Crew 해산 (정식 구성원이 없고 진행 업무가 없을 때만)	`routes/crews.js:152`
POST	`/api/crews/:id/restore` API-067 해산된 Crew 복구	`routes/crews.js:174`
POST	`/api/regions` API-079 지역 등록	`routes/regions.js:38`
PATCH	`/api/tasks/:id/status` API-101 업무 진행 상태 변경 (전이 규칙 검사, 이력 기록)	`routes/tasks.js:217`
GET	`/api/tasks/:id/status-history` API-103 업무 상태·배정 변경 이력 (MongoDB)	`routes/tasks.js:271`
POST	`/api/tasks/:id/complete` API-104 업무 완료 처리 (결과 및 참여 기록 입력)	`routes/tasks.js:288`
PATCH	`/api/tasks/:id/result` API-106 완료된 업무 결과 수정 (Underboss 이상, 사유 필수, 수정 전 값 이력 보존)	`routes/tasks.js:355`
GET	`/api/tasks/:id/timeline` API-110 업무 통합 타임라인 (상태 이력 + 코멘트 + 자원 사용 이벤트)	`routes/tasks.js:439`

PUT	`/api/meetings/:id/minutes` API-141 회의록 작성/갱신 (MongoDB, 결정사항 포함, 시작 이후 작성 시 DONE 처리)	`routes/meetings.js:208`
POST	`/api/resources` API-146 자원 등록	`routes/resources.js:133`
DELETE	`/api/evaluations/:id` API-179 평가 무효화 (삭제하지 않고 VOIDED 처리, 사유 기록)	`routes/evaluations.js:195`
GET	`/api/audit-logs` API-202 감사 로그 조회 (행위자·행위·대상·결과·기간)	`routes/audit.js:35`
GET	`/api/audit-logs/summary` API-203 감사 로그 집계 (행위별·결과별·일자별 건수, 상위 행위자)	`routes/audit.js:50`
GET	`/api/audit-logs/export` API-204 감사 로그 CSV 내보내기 (최대 10,000건)	`routes/audit.js:74`
GET	`/api/audit-logs/:id` API-205 감사 로그 상세	`routes/audit.js:88`
POST	`/api/admin/accounts/:id/lock` API-207 계정 잠금 (로그인 차단 + 기존 세션 즉시 무효)	`routes/admin.js:62`

	POST	`/api/admin/accounts/:id/unlock` API-208 계정 잠금 해제 (로그인 실패 횟수 초기화)	`routes/admin.js:78`	
	POST	`/api/admin/accounts/:id/reset-password` API-209 비밀번호 초기화 (임시 비밀번호 발급, 기존 세션 만료)	`routes/admin.js:92`	
	GET	`/api/admin/accounts/:id/login-history` API-212 특정 계정의 로그인 이력 (성공/실패, IP)	`routes/admin.js:137`	
	PATCH	`/api/admin/settings` API-215 시스템 운영 설정 변경 (평가 수정 가능 시간, 1인 최대 동시 업무 수, 로그인 잠금 임계값)	`routes/admin.js:181`	

FR-21	GET	`/api/me/saved-filters` API-022 내가 저장한 검색 조건 목록	`routes/me.js:330`	10장 FR-21 절차 (5단계) GET /api/search?q=Brooklyn → 200 / GET /api/members?q=Paulie&sort=name → 200
	POST	`/api/me/saved-filters` API-023 검색 조건 저장 (자주 쓰는 업무/구성원/자원 검색 조건)	`routes/me.js:341`	
	DELETE	`/api/me/saved-filters/:id` API-024 저장한 검색 조건 삭제	`routes/me.js:359`	
	GET	`/api/members` API-032 구성원 검색/목록 (권한 범위 내, 필터·정렬·페이지)	`routes/members.js:80`	
	GET	`/api/members/export` API-033 구성원 목록 CSV 내보내기 (검색 조건 동일 적용)	`routes/members.js:100`	
	GET	`/api/crews` API-061 Crew 목록/검색 (지역·상태·책임자 필터)	`routes/crews.js:58`	
	GET	`/api/regions` API-077 지역 목록 (활성 Crew 수 포함)	`routes/regions.js:19`	
	GET	`/api/join-requests` API-084 가입 요청 목록 (내 요청 또는 관리 Crew 로 들어온 요청)	`routes/joinRequests.js:60`	
	GET	`/api/tasks` API-094 업무 탐색 (기간·Crew·상태·우선순위·담당자·검색어, 정렬, 페이지)	`routes/tasks.js:57`	

GET	`/api/meetings` API-133 회의 목록 (내가 참석/주관하거나 관리 Crew 회의)	`routes/meetings.js:41`	
GET	`/api/resources` API-142 자원 목록/검색 (유형·상태·Crew·검색어)	`routes/resources.js:41`	
GET	`/api/allocations` API-157 자원 배정/사용 기록 조회 (자원·업무·구성원·상태·기간)	`routes/allocations.js:42`	
GET	`/api/resource-requests` API-165 자원 사용 요청 목록 (내 요청 또는 심사 가능한 요청)	`routes/resourceRequests.js:63`	
GET	`/api/search` API-189 통합 검색 (구성원·Crew·업무·자원, 권한 범위 내)	`routes/search.js:40`	
GET	`/api/search/suggest` API-190 검색어 자동완성 (이름 접두어 일치, 최대 10건)	`routes/search.js:54`	
GET	`/api/admin/accounts` API-206 계정 목록 (잠금 여부, 로그인 실패 횟수, 최근 로그인)	`routes/admin.js:34`	

FR-22	GET	`/api/members/export` API-033 구성원 목록 CSV 내보내기 (검색 조건 동일 적용)	`routes/members.js:100`	10장 FR-22 절차 (7단계) GET /api/stats/overview → 200 / GET /api/stats/crews?from=2026-01-01T00:00:00Z → 200
	GET	`/api/crews/:id/stats` API-076 Crew 운영 현황 (구성원 상태 분포, 업무 상태 분포, 완료율, 평균 기여도)	`routes/crews.js:375`	
	GET	`/api/regions/:id/stats` API-082 지역별 운영 현황 (구성원·업무·자원 집계)	`routes/regions.js:83`	
	GET	`/api/tasks/export` API-095 업무 목록 CSV 내보내기 (탐색 조건 동일 적용)	`routes/tasks.js:75`	
	GET	`/api/reputation/leaderboard` API-185 평판 순위 (조회 가능한 Crew 구성원 대상)	`routes/evaluations.js:289`	
	GET	`/api/stats/overview` API-191 운영 현황 요약 (KPI 대시보드)	`routes/stats.js:35`	
	GET	`/api/stats/members` API-192 구성원 현황 (상태별·역할별·Crew별 분포)	`routes/stats.js:74`	
	GET	`/api/stats/crews` API-193 Crew 별 운영 비교 (인원, 업무 완료율, 성공률, 평균 기여도, 성과값)	`routes/stats.js:100`	

GET	`/api/stats/tasks` API-194 업무 현황 (기간 내 상태별·우선순위별·결과별 분포)	`routes/stats.js:131`
GET	`/api/stats/tasks/trend` API-195 업무 추이 (일/주/월 단위 생성·완료 건수 시계열)	`routes/stats.js:154`
GET	`/api/stats/members/workload` API-196 구성원별 업무량 (기간 내 배정 건수·배정 시간·진행중 업무)	`routes/stats.js:174`
GET	`/api/stats/resources/utilization` API-197 자원 가동률 (기간 대비 배정·사용 시간 비율)	`routes/stats.js:196`
GET	`/api/stats/resources/issues` API-198 자원 이슈 통계 (심각도·상태·유형별, 평균 해결 시간)	`routes/stats.js:215`
GET	`/api/stats/evaluations` API-199 평가 통계 (항목별 평균, 종합 점수 분포, 월별 평균)	`routes/stats.js:233`
GET	`/api/stats/activities/heatmap` API-200 활동 히트맵 (요일 × 시간대 활동 건수, MongoDB 집계)	`routes/stats.js:262`

	GET	`/api/stats/status-changes` API-201 구성원 변경 추이 (역할·상태·소속·평가 변경 이력의 주별 집계)	`routes/stats.js:287`	
	GET	`/api/audit-logs/summary` API-203 감사 로그 집계 (행위별·결과별·일자별 건수, 상위 행위자)	`routes/audit.js:50`	
	GET	`/api/admin/security/failed-logins` API-213 로그인 실패 집계 (계정·IP 별, 의심 계정 탐지)	`routes/admin.js:154`	

FR-23	POST	`/api/auth/login` API-002 로그인 (JWT 토큰 발급)	`routes/auth.js:87`	10장 FR-23 절차 (8단계) POST /api/admin/accounts/9/lock → 200 / POST /api/auth/login → 401
	PUT	`/api/auth/password` API-005 내 비밀번호 변경 (다른 세션은 모두 만료)	`routes/auth.js:120`	
	GET	`/api/roles` API-025 역할 목록 (권한 등급 및 역할별 인원 수)	`routes/roles.js:32`	
	GET	`/api/roles/:code` API-026 역할 상세 (상태별 인원 분포 포함)	`routes/roles.js:42`	
	GET	`/api/roles/:code/permissions` API-027 역할의 권한 매트릭스 (전체 권한 중 부여 여부)	`routes/roles.js:54`	
	PUT	`/api/roles/:code/permissions` API-028 역할의 권한 집합 전체 교체	`routes/roles.js:67`	
	POST	`/api/roles/:code/permissions` API-029 역할에 권한 1개 부여	`routes/roles.js:91`	
	DELETE	`/api/roles/:code/permissions/:permission` API-030 역할에서 권한 1개 회수	`routes/roles.js:108`	
	GET	`/api/permissions` API-031 전체 권한 코드 목록 (각 권한을 보유한 역할 포함)	`routes/roles.js:125`	

GET	`/api/admin/accounts` API-206 계정 목록 (잠금 여부, 로그인 실패 횟수, 최근 로그인)	`routes/admin.js:34`
POST	`/api/admin/accounts/:id/lock` API-207 계정 잠금 (로그인 차단 + 기존 세션 즉시 무효)	`routes/admin.js:62`
POST	`/api/admin/accounts/:id/unlock` API-208 계정 잠금 해제 (로그인 실패 횟수 초기화)	`routes/admin.js:78`
POST	`/api/admin/accounts/:id/reset-password` API-209 비밀번호 초기화 (임시 비밀번호 발급, 기존 세션 만료)	`routes/admin.js:92`
POST	`/api/admin/accounts/:id/force-logout` API-210 강제 로그아웃 (해당 계정의 모든 발급 토큰 무효화)	`routes/admin.js:108`
PATCH	`/api/admin/accounts/:id/username` API-211 로그인 아이디 변경	`routes/admin.js:122`
GET	`/api/admin/accounts/:id/login-history` API-212 특정 계정의 로그인 이력 (성공/실패, IP)	`routes/admin.js:137`

	GET	`/api/admin/security/failed-logins` API-213 로그인 실패 집계 (계정·IP 별, 의심 계정 탐지)	`routes/admin.js:154`	
	GET	`/api/admin/settings` API-214 시스템 운영 설정 조회	`routes/admin.js:175`	
	PATCH	`/api/admin/settings` API-215 시스템 운영 설정 변경 (평가 수정 가능 시간, 1인 최대 동시 업무 수, 로그인 잠금 임계값)	`routes/admin.js:181`	
	GET	`/api/admin/system/info` API-216 시스템 정보 (DB 이름, 버전, 테이블/컬렉션별 데이터 건수)	`routes/admin.js:198`	
	GET	`/api/admin/integrity-check` API-217 업무 규칙 무결성 점검 (BR-01/03/07/08 위반 데이터 탐지 + 운영 참고 정보)	`routes/admin.js:218`	
	POST	`/api/admin/system/reset-demo` API-218 데모 데이터 초기화 (테스트 재현용: 모든 데이터를 초기 상태로)	`routes/admin.js:249`	
	POST	`/api/admin/broadcast` API-219 전체/대상 공지 알림 발송 (역할·Crew 필터)	`routes/admin.js:263`	

업무 규칙

Requirement ID	Method	API	구현 위치	확인방법

BR-01	POST	`/api/members` API-035 구성원 등록 (역할/상태/소속 Crew 지정 가능)	`routes/members.js:137`	10장 BR-01 절차 (4단계) POST /api/crews/1/members → 409 BR-01 / POST /api/crews/1/members → 201
	PUT	`/api/crews/:id/capo` API-068 Crew 책임자(Capo) 지정	`routes/crews.js:188`	
	POST	`/api/crews/:id/members` API-072 Crew 에 구성원 편성 (정식/준소속)	`routes/crews.js:278`	
	POST	`/api/crews/:id/members/:memberId/transfer` API-074 구성원을 다른 Crew 로 이동 (정식 소속 원자적 변경)	`routes/crews.js:320`	
	POST	`/api/crews/:id/join-requests` API-083 Crew 가입 요청 (본인 신청, 가입대기 계정도 가능)	`routes/joinRequests.js:33`	
	POST	`/api/join-requests/:id/approve` API-086 가입 요청 승인 (Crew 편성 + 가입대기 계정은 활동중으로 전환)	`routes/joinRequests.js:95`	
	GET	`/api/admin/integrity-check` API-217 업무 규칙 무결성 점검 (BR-01/03/07/08 위반 데이터 탐지 + 운영 참고 정보)	`routes/admin.js:218`	

BR-02	GET	`/api/members/:id/availability` API-044 특정 시간대 배정 가능 여부 사전 점검 (상태·시간 중복)	`routes/members.js:354`	10장 BR-02 절차 (5단계) POST /api/tasks/2/assignments → 409 BR-02 / POST /api/tasks/5/assignments → 409 BR-02
	POST	`/api/members/:id/disciplinary-actions` API-057 구성원 징계 등록 (SUSPENSION은 상태를 정지로 변경)	`routes/discipline.js:25`	
	GET	`/api/tasks/:id/candidates` API-111 배정 후보 추천 (상태·시간 충돌 없는 구성원, 기술 적합도·실무 인력·업무량 순)	`routes/tasks.js:460`	
	POST	`/api/tasks/:id/assignments` API-114 업무에 구성원 배정	`routes/assignments.js:58`	
	POST	`/api/tasks/:id/assignments/:memberId/replace` API-117 배정 인원 교체 (기존 해제 + 신규 배정을 하나의 트랜잭션으로)	`routes/assignments.js:125`	

BR-03	GET	`/api/members/:id/schedule` API-043 구성원 일정 타임라인 (업무·자원 사용·회의)	`routes/members.js:324`	10장 BR-03 절차 (8단계) GET /api/tasks/1 → 200 / POST /api/tasks/2/assignments → 409 BR-03
	GET	`/api/members/:id/availability` API-044 특정 시간대 배정 가능 여부 사전 점검 (상태·시간 중복)	`routes/members.js:354`	
	PATCH	`/api/tasks/:id/schedule` API-100 업무 일정 변경 (참여자 시간 중복·자원 중복 재검사)	`routes/tasks.js:176`	
	GET	`/api/tasks/:id/candidates` API-111 배정 후보 추천 (상태·시간 충돌 없는 구성원, 기술 적합도·실무 인력·업무량 순)	`routes/tasks.js:460`	
	GET	`/api/tasks/:id/conflicts` API-112 업무 충돌 진단 (참여자 일정 중복 BR-03, 자원 중복 BR-07)	`routes/tasks.js:497`	
	POST	`/api/tasks/:id/assignments` API-114 업무에 구성원 배정	`routes/assignments.js:58`	
	POST	`/api/tasks/:id/assignments/:memberId/replace` API-117 배정 인원 교체 (기존 해제 + 신규 배정을 하나의 트랜잭션으로)	`routes/assignments.js:125`	
	GET	`/api/stats/members/workload` API-196 구성원별 업무량 (기간 내 배정 건수·배정 시간·진행중 업무)	`routes/stats.js:174`	

	GET	`/api/admin/integrity-check` API-217 업무 규칙 무결성 점검 (BR-01/03/07/08 위반 데이터 탐지 + 운영 참고 정보)	`routes/admin.js:218`	
BR-04	POST	`/api/members` API-035 구성원 등록 (역할/상태/소속 Crew 지정 가능)	`routes/members.js:137`	10장 BR-04 절차 (5단계) PATCH /api/members/3/role → 403 BR-04 / PATCH /api/members/14/role → 403 BR-04
	PATCH	`/api/members/:id` API-036 구성원 기본 정보 수정 (이름/별칭/연락처/생년월일/메모)	`routes/members.js:179`	
	PATCH	`/api/members/:id/role` API-038 구성원 역할 변경 (자신보다 높은 권한자 변경 불가)	`routes/members.js:225`	
	PATCH	`/api/members/:id/status` API-039 구성원 상태 변경 (활동중/대기/정지/탈퇴)	`routes/members.js:244`	
	POST	`/api/promotion-requests` API-051 승급 요청 등록	`routes/promotions.js:41`	
	POST	`/api/promotion-requests/:id/approve` API-054 승급 요청 승인 (역할 변경 실행 + 이력 기록)	`routes/promotions.js:107`	
	POST	`/api/members/:id/disciplinary-actions` API-057 구성원 징계 등록 (SUSPENSION은 상태를 정지로 변경)	`routes/discipline.js:25`	

BR-05	PATCH	`/api/members/:id/role` API-038 구성원 역할 변경 (자신보다 높은 권한자 변경 불가)	`routes/members.js:225`	10장 BR-05 절차 (4단계) POST /api/crews/2/members → 403 BR-05 / DELETE /api/crews/2/members/11 → 403 BR-05
	GET	`/api/crews/managed` API-062 내가 관리할 수 있는 Crew 목록	`routes/crews.js:82`	
	PATCH	`/api/crews/:id` API-065 Crew 정보 수정 (이름/지역/설명)	`routes/crews.js:131`	
	POST	`/api/crews/:id/members` API-072 Crew 에 구성원 편성 (정식/준소속)	`routes/crews.js:278`	
	DELETE	`/api/crews/:id/members/:memberId` API-073 Crew 에서 구성원 제외	`routes/crews.js:301`	
	POST	`/api/crews/:id/members/:memberId/transfer` API-074 구성원을 다른 Crew 로 이동 (정식 소속 원자적 변경)	`routes/crews.js:320`	
	POST	`/api/join-requests/:id/approve` API-086 가입 요청 승인 (Crew 편성 + 가입대기 계정은 활동중으로 전환)	`routes/joinRequests.js:95`	
	POST	`/api/tasks/:id/assignments` API-114 업무에 구성원 배정	`routes/assignments.js:58`	

BR-06	**PATCH**	`/api/tasks/:id` API-099 업무 기본 정보 수정 (제목/설명/중요도/장소/필요 기술)	`routes/tasks.js:151`	10장 BR-06 절차 (7단계) PATCH /api/tasks/3/result → 403 / PATCH /api/tasks/3/result → 403
	PATCH	`/api/tasks/:id/result` API-106 완료된 업무 결과 수정 (Underboss 이상, 사유 필수, 수정 전 값 이력 보존)	`routes/tasks.js:355`	
	POST	`/api/tasks/:id/reopen` API-107 완료된 업무 재개 (Underboss 이상, COMPLETED → IN_PROGRESS)	`routes/tasks.js:382`	
	PATCH	`/api/tasks/:id/assignments/:memberId` API-115 배정 정보 수정 (업무 내 역할, 지시사항)	`routes/assignments.js:83`	
	DELETE	`/api/tasks/:id/assignments/:memberId` API-116 배정 해제	`routes/assignments.js:106`	
	PUT	`/api/tasks/:id/assignments/:memberId/participation` API-118 완료 업무의 참여 기록(출석·기여도) 입력	`routes/assignments.js:149`	
	PATCH	`/api/tasks/:id/checklist/:itemId` API-125 체크리스트 항목 완료 체크/수정 (참여자: 체크만, 관리자: 문구 수정)	`routes/taskCollab.js:118`	

BR-07	PATCH	`/api/tasks/:id/schedule` API-100 업무 일정 변경 (참여자 시간 중복·자원 중복 재검사)	`routes/tasks.js:176`	10장 BR-07 절차 (4단계) POST /api/allocations → 409 BR-07 / POST /api/allocations → 409 BR-07
	GET	`/api/tasks/:id/conflicts` API-112 업무 충돌 진단 (참여자 일정 중복 BR-03, 자원 중복 BR-07)	`routes/tasks.js:497`	
	GET	`/api/resources/available` API-143 지정 시간대에 내가 이용 가능한 자원 (권한·상태·기존 배정 고려)	`routes/resources.js:71`	
	GET	`/api/resources/:id/availability` API-151 자원의 기간 내 사용 가능/불가 구간 (예약 현황)	`routes/resources.js:255`	
	POST	`/api/allocations` API-159 자원을 업무 또는 구성원에게 배정 (예약)	`routes/allocations.js:94`	
	PATCH	`/api/allocations/:id` API-160 예약 기간 변경 (중복 재검사)	`routes/allocations.js:122`	
	POST	`/api/allocations/:id/checkout` API-162 자원 사용 시작 (체크아웃: RESERVED → IN_USE)	`routes/allocations.js:164`	
	POST	`/api/resource-requests` API-164 자원 사용 요청 (권한 등급·상태·시간 중복 사전 검사)	`routes/resourceRequests.js:34`	

	POST	`/api/resource-requests/:id/approve` API-167 자원 사용 요청 승인 (배정 생성 시 BR-07/BR-08 재검사)	`routes/resourceRequests.js:99`	
	GET	`/api/admin/integrity-check` API-217 업무 규칙 무결성 점검 (BR-01/03/07/08 위반 데이터 탐지 + 운영 참고 정보)	`routes/admin.js:218`	

BR-08	GET	`/api/resources/available` API-143 지정 시간대에 내가 이용 가능한 자원 (권한·상태·기존 배정 고려)	`routes/resources.js:71`	10장 BR-08 절차 (5단계) POST /api/allocations → 409 BR-08 / POST /api/allocations → 409 BR-08
	PATCH	`/api/resources/:id/status` API-149 자원 상태 변경 (정상/점검 필요/정비중/사용 불가/폐기)	`routes/resources.js:200`	
	POST	`/api/allocations` API-159 자원을 업무 또는 구성원에게 배정 (예약)	`routes/allocations.js:94`	
	PATCH	`/api/allocations/:id` API-160 예약 기간 변경 (중복 재검사)	`routes/allocations.js:122`	
	POST	`/api/allocations/:id/checkout` API-162 자원 사용 시작 (체크아웃: RESERVED → IN_USE)	`routes/allocations.js:164`	
	POST	`/api/resource-requests` API-164 자원 사용 요청 (권한 등급·상태·시간 중복 사전 검사)	`routes/resourceRequests.js:34`	
	POST	`/api/resource-requests/:id/approve` API-167 자원 사용 요청 승인 (배정 생성 시 BR-07/BR-08 재검사)	`routes/resourceRequests.js:99`	
	POST	`/api/resources/:id/issues` API-170 자원 이상 신고 (정상 자원은 점검 필요 상태로 자동 전환)	`routes/resourceIssues.js:25`	

	GET	`/api/admin/integrity-check` API-217 업무 규칙 무결성 점검 (BR-01/03/07/08 위반 데이터 탐지 + 운영 참고 정보)	`routes/admin.js:218`	
BR-09	GET	`/api/roles/:code/permissions` API-027 역할의 권한 매트릭스 (전체 권한 중 부여 여부)	`routes/roles.js:54`	10장 BR-09 절차 (6단계) GET /api/members → 401 / GET /api/audit-logs → 403 BR-09
	GET	`/api/permissions` API-031 전체 권한 코드 목록 (각 권한을 보유한 역할 포함)	`routes/roles.js:125`	
	GET	`/api/members` API-032 구성원 검색/목록 (권한 범위 내, 필터·정렬·페이지)	`routes/members.js:80`	
	GET	`/api/members/:id` API-034 구성원 상세 조회 (소속 Crew, 기술 포함)	`routes/members.js:118`	
	POST	`/api/tasks` API-098 업무 등록 (기본 정보, 일정, 중요도)	`routes/tasks.js:127`	
	GET	`/api/search` API-189 통합 검색 (구성원·Crew·업무·자원, 권한 범위 내)	`routes/search.js:40`	
	GET	`/api/audit-logs` API-202 감사 로그 조회 (행위자·행위·대상·결과·기간)	`routes/audit.js:35`	

BR-10	PATCH	`/api/members/:id/role` API-038 구성원 역할 변경 (자신보다 높은 권한자 변경 불가)	`routes/members.js:225`	10장 BR-10 절차 (5단계) GET /api/members/14/history → 200 / GET /api/stats/status-changes → 200
	PATCH	`/api/members/:id/status` API-039 구성원 상태 변경 (활동중/대기/정지/탈퇴)	`routes/members.js:244`	
	GET	`/api/members/:id/crew-history` API-041 구성원의 Crew 소속 이력 (가입/이동/탈퇴)	`routes/members.js:282`	
	POST	`/api/members/:id/reputation-adjustments` API-046 평판 가감점 부여 (공로/과실)	`routes/members.js:392`	
	GET	`/api/members/:id/history` API-047 구성원 주요 변경 이력 (역할·소속·상태·평가·징계)	`routes/members.js:416`	
	POST	`/api/promotion-requests/:id/approve` API-054 승급 요청 승인 (역할 변경 실행 + 이력 기록)	`routes/promotions.js:107`	
	POST	`/api/members/:id/disciplinary-actions` API-057 구성원 징계 등록 (SUSPENSION은 상태를 정지로 변경)	`routes/discipline.js:25`	
	GET	`/api/disciplinary-actions` API-058 징계 목록 (관리 범위 내)	`routes/discipline.js:56`	

POST	`/api/disciplinary-actions/:id/revoke` API-060 징계 해제 (정지 징계 해제 시 다른 정지 징계가 없으면 상태를 ACTIVE 로 복구)	`routes/discipline.js:98`
PUT	`/api/crews/:id/capo` API-068 Crew 책임자(Capo) 지정	`routes/crews.js:188`
DELETE	`/api/crews/:id/capo` API-069 Crew 책임자 해제 (공석 처리)	`routes/crews.js:221`
POST	`/api/crews/:id/members` API-072 Crew 에 구성원 편성 (정식/준소속)	`routes/crews.js:278`
DELETE	`/api/crews/:id/members/:memberId` API-073 Crew 에서 구성원 제외	`routes/crews.js:301`
POST	`/api/crews/:id/members/:memberId/transfer` API-074 구성원을 다른 Crew 로 이동 (정식 소속 원자적 변경)	`routes/crews.js:320`
GET	`/api/crews/:id/membership-history` API-075 Crew 구성원 편성 변경 이력 (가입/제외/이동)	`routes/crews.js:350`
POST	`/api/join-requests/:id/approve` API-086 가입 요청 승인 (Crew 편성 + 가입대기 계정은 활동중으로 전환)	`routes/joinRequests.js:95`

	POST	`/api/evaluations` API-177 평가 등록 (업무 결과 기반, 항목별 1~5점, 가중 평균 자동 계산)	`routes/evaluations.js:129`	
	PATCH	`/api/evaluations/:id` API-178 평가 수정 (작성자: 설정된 기간 내 / Underboss 이상: 언제나, 수정 전 내용 보존)	`routes/evaluations.js:164`	
	DELETE	`/api/evaluations/:id` API-179 평가 무효화 (삭제하지 않고 VOIDED 처리, 사유 기록)	`routes/evaluations.js:195`	
	GET	`/api/evaluations/:id/revisions` API-180 평가 수정 이력	`routes/evaluations.js:212`	
	GET	`/api/stats/status-changes` API-201 구성원 변경 추이 (역할·상태·소속·평가 변경 이력의 주별 집계)	`routes/stats.js:287`	
	GET	`/api/audit-logs` API-202 감사 로그 조회 (행위자·행위·대상·결과·기간)	`routes/audit.js:35`	

데이터 요구사항

Requirement ID	Method	API	구현 위치	확인방법

DR-01	**GET**	`/api/roles` API-025 역할 목록 (권한 등급 및 역할별 인원 수)	`routes/roles.js:32`	10장 DR-01 절차 (4단계) GET /api/admin/integrity-check → 200 / DELETE /api/regions/1 → 422
	GET	`/api/roles/:code` API-026 역할 상세 (상태별 인원 분포 포함)	`routes/roles.js:42`	
	DELETE	`/api/members/:id` API-037 가입대기 구성원 삭제 (활동 이력이 없는 PENDING 계정만)	`routes/members.js:203`	
	GET	`/api/members/:id/crew-history` API-041 구성원의 Crew 소속 이력 (가입/이동/탈퇴)	`routes/members.js:282`	
	DELETE	`/api/regions/:id` API-081 지역 삭제 (연결된 Crew 가 없을 때만)	`routes/regions.js:68`	
	DELETE	`/api/resource-types/:id` API-156 자원 유형 삭제 (해당 유형 자원이 없을 때만)	`routes/resourceTypes.js:71`	
	GET	`/api/admin/integrity-check` API-217 업무 규칙 무결성 점검 (BR-01/03/07/08 위반 데이터 탐지 + 운영 참고 정보)	`routes/admin.js:218`	

DR-02	**GET**	`/api/members/:id/assignments` API-042 구성원의 업무 참여 이력 (참여 기록·기여도 포함)	`routes/members.js:298`	10장 DR-02 절차 (3단계) GET /api/tasks/1 → 200 / GET /api/members/8/assignments → 200
	GET	`/api/tasks/:id` API-097 업무 상세 (참여 구성원, 배정 자원 포함)	`routes/tasks.js:112`	
	DELETE	`/api/tasks/:id` API-109 업무 삭제 (배정·자원 이력이 없는 PLANNED/CANCELLED 업무만)	`routes/tasks.js:421`	
	GET	`/api/tasks/:id/assignments` API-113 업무 참여 구성원 목록 (배정 상태 포함)	`routes/assignments.js:41`	

DR-03	**GET**	`/api/resources/:id` API-145 자원 상세 (현재 사용 현황, 다음 예약, 미해결 이슈)	`routes/resources.js:117`	10장 DR-03 절차 (3단계) GET /api/resources/2 → 200 / GET /api/resources/2/history → 200
	DELETE	`/api/resources/:id` API-148 자원 삭제 (사용 이력이 전혀 없는 경우만, 그 외는 RETIRED 상태로 관리)	`routes/resources.js:184`	
	GET	`/api/resources/:id/history` API-150 자원 사용·반납·상태 변경 이력 (MongoDB)	`routes/resources.js:237`	
	GET	`/api/allocations` API-157 자원 배정/사용 기록 조회 (자원·업무·구성원·상태·기간)	`routes/allocations.js:42`	
	POST	`/api/allocations` API-159 자원을 업무 또는 구성원에게 배정 (예약)	`routes/allocations.js:94`	
	POST	`/api/allocations/:id/checkout` API-162 자원 사용 시작 (체크아웃: RESERVED → IN_USE)	`routes/allocations.js:164`	
	POST	`/api/allocations/:id/return` API-163 자원 반납 (상태 기록, 파손 시 점검 필요 / 분실 시 사용 불가 자동 전환)	`routes/allocations.js:189`	

DR-04	GET	`/api/tasks/:id/comments` API-119 업무 코멘트 목록	`routes/taskCollab.js:28`	10장 DR-04 절차 (4단계) GET /api/activities?size=5 → 200 / POST /api/activities → 201
	POST	`/api/tasks/:id/comments` API-120 업무 코멘트 작성 (업무 조회 권한자)	`routes/taskCollab.js:42`	
	PUT	`/api/meetings/:id/minutes` API-141 회의록 작성/갱신 (MongoDB, 결정사항 포함, 시작 이후 작성 시 DONE 처리)	`routes/meetings.js:208`	
	GET	`/api/evaluations` API-174 평가 이력 조회 (구성원·업무·평가자·기간)	`routes/evaluations.js:59`	
	POST	`/api/evaluations` API-177 평가 등록 (업무 결과 기반, 항목별 1~5점, 가중 평균 자동 계산)	`routes/evaluations.js:129`	
	POST	`/api/evaluation-criteria` API-182 평가 항목 추가	`routes/evaluations.js:235`	
	DELETE	`/api/evaluation-criteria/:code` API-184 평가 항목 삭제 (평가에 사용된 적 없는 항목만, 그 외는 비활성화)	`routes/evaluations.js:271`	
	GET	`/api/activities` API-186 활동 기록 조회 (본인 또는 관리 범위 구성원)	`routes/activities.js:14`	
	POST	`/api/activities` API-188 내 활동 보고 등록 (현장 보고·순찰·훈련 등, 유연한 상세 정보)	`routes/activities.js:60`	
	GET	`/api/stats/activities/heatmap` API-200 활동 히트맵 (요일 × 시간대 활동 건수, MongoDB 집계)	`routes/stats.js:262`	

DR-05	GET	`/api/admin/system/info` API-216 시스템 정보 (DB 이름, 버전, 테이블/컬렉션별 데이터 건수)	`routes/admin.js:198`	10장 DR-05 절차 (1단계) GET /api/admin/system/info → 200
	POST	`/api/admin/system/reset-demo` API-218 데모 데이터 초기화 (테스트 재현용: 모든 데이터를 초기 상태로)	`routes/admin.js:249`	
DR-06	GET	`/api/admin/system/info` API-216 시스템 정보 (DB 이름, 버전, 테이블/컬렉션별 데이터 건수)	`routes/admin.js:198`	10장 DR-06 절차 (1단계) GET /api/admin/system/info → 200

REST API 요구사항

Requirement ID	Method	API	구현 위치	확인방법
AR-01	-	조회 GET, 생성 POST(201), 전체 교체 PUT, 부분 수정 PATCH, 삭제/취소 DELETE, 상태 전이 동작(승인·완료·체크아웃 등)은 하위 리소스 POST.		10장 AR-01 절차 (1단계) GET /api/api-catalog?tag=Tasks → 200
AR-02	-	식별자=Path, 필터·정렬·페이지=Query, 인증 토큰=Authorization Header, 생성/변경 데이터=JSON Body.		10장 AR-02 절차 (3단계) GET /api/tasks/1 → 200 / GET /api/tasks?status=PLANNED&sort=-startAt → 200
AR-03	GET	`/api/tasks` API-094 업무 탐색 (기간·Crew·상태·우선순위·담당자·검색어, 정렬, 페이지)	`routes/tasks.js:57`	10장 AR-03 절차 (2단계) GET /api/members?q=o&status=ACTIVE&sort=-joinedAt&page=2&size=5 → 200 / GET /api/tasks?sort=unknown → 400
AR-04	-	400 VALIDATION_ERROR(필드별 details) / 401 / 403 FORBIDDEN(rule) / 404 NOT_FOUND / 409 BUSINESS_RULE_VIOLATION(rule) / 422 INVALID_STATE.		10장 AR-04 절차 (5단계) POST /api/tasks → 400 / POST /api/tasks/2/assignments → 409 BR-03

AR-05	-	성공 { success:true, message, data, meta? } / 실패 { success:false, error: { code, message, details } } 를 lib/api.js 와 lib/errorHandler.js 에서 일괄 생성.		10장 AR-05 절차 (2단계) GET /api/crews/1 → 200 / GET /api/crews/999 → 404
AR-06	GET	`/api/api-catalog` API-221 API 카탈로그 (전체 API 와 연결된 요구사항 - 요구사항 추적용, 공개)	`routes/admin.js:297`	10장 AR-06 절차 (1단계) GET /api/api-catalog → 200
AR-07	-	각 API 는 서로 다른 업무 동작을 수행하도록 설계(예: 상태 변경 vs 완료 처리(결과 입력) vs 결과 수정(권한·이력) vs 재개). 단순 필터 차이는 하나의 목록 API 의 Query Parameter 로 통합.		10장 AR-07 절차 (1단계) GET /api/api-catalog → 200
AR-08	GET	`/api/api-catalog` API-221 API 카탈로그 (전체 API 와 연결된 요구사항 - 요구사항 추적용, 공개)	`routes/admin.js:297`	10장 AR-08 절차 (1단계) GET /api/api-catalog?req=BR-03 → 200

비기능 요구사항

Requirement ID	Method	API	구현 위치	확인방법
NFR-01	GET	`/api/me/notifications` API-016 내 알림 목록	`routes/me.js:262`	10장 NFR-01 절차 (2단계) PATCH /api/tasks/1/status → 200 / PATCH /api/tasks/1/status → 422
	GET	`/api/health` API-220 서버 및 DB 연결 상태 확인 (공개)	`routes/admin.js:287`	
NFR-02	GET	`/api/health` API-220 서버 및 DB 연결 상태 확인 (공개)	`routes/admin.js:287`	10장 NFR-02 절차 (4단계) POST /api/tasks → 400 / GET /api/members?q=' OR 1=1 -- → 200

NFR-03	**POST**	`/api/auth/login` API-002 로그인 (JWT 토큰 발급)	`routes/auth.js:87`	10장 NFR-03 절차 (2단계) PATCH /api/members/8/role → 403
	POST	`/api/auth/logout` API-003 로그아웃 (현재 토큰 폐기)	`routes/auth.js:100`	
	POST	`/api/auth/refresh` API-004 토큰 재발급 (기존 토큰은 폐기)	`routes/auth.js:111`	
	GET	`/api/auth/permissions` API-007 내 역할, 권한 목록, 관리 범위(Crew) 조회	`routes/auth.js:153`	
	POST	`/api/admin/accounts/:id/lock` API-207 계정 잠금 (로그인 차단 + 기존 세션 즉시 무효)	`routes/admin.js:62`	
	POST	`/api/admin/accounts/:id/force-logout` API-210 강제 로그아웃 (해당 계정의 모든 발급 토큰 무효화)	`routes/admin.js:108`	

NFR-04	GET	`/api/me/saved-filters` API-022 내가 저장한 검색 조건 목록	`routes/me.js:330`	10장 NFR-04 절차 (3단계) POST /api/me/saved-filters → 201
	POST	`/api/me/saved-filters` API-023 검색 조건 저장 (자주 쓰는 업무/구성원/자원 검색 조건)	`routes/me.js:341`	
	GET	`/api/members` API-032 구성원 검색/목록 (권한 범위 내, 필터·정렬·페이지)	`routes/members.js:80`	
	GET	`/api/crews` API-061 Crew 목록/검색 (지역·상태·책임자 필터)	`routes/crews.js:58`	
	GET	`/api/tasks` API-094 업무 탐색 (기간·Crew·상태·우선순위·담당자·검색어, 정렬, 페이지)	`routes/tasks.js:57`	
	GET	`/api/resources` API-142 자원 목록/검색 (유형·상태·Crew·검색어)	`routes/resources.js:41`	
	GET	`/api/activities` API-186 활동 기록 조회 (본인 또는 관리 범위 구성원)	`routes/activities.js:14`	
	GET	`/api/search` API-189 통합 검색 (구성원·Crew·업무·자원, 권한 범위 내)	`routes/search.js:40`	
	GET	`/api/search/suggest` API-190 검색어 자동완성 (이름 접두어 일치, 최대 10건)	`routes/search.js:54`	
	GET	`/api/audit-logs` API-202 감사 로그 조회 (행위자·행위·대상·결과·기간)	`routes/audit.js:35`	
NFR-05	-	AR-05 와 동일. Swagger 의 모든 응답 예시도 동일한 SuccessResponse / ErrorResponse 스키마 사용.		10장 NFR-05 절차 (2단계)

기술 제약조건

Requirement ID	Method	API	구현 위치	확인방법
CR-01	-	Express 4 기반 (package.json). app.js 진입점.	app.js, package.json	10장 CR-01 절차 (1단계)
CR-02	-	views/layout.ejs(공통 레이아웃·권한별 메뉴), login.ejs, error.ejs, catalog.ejs. 21개 업무 화면을 서버가 EJS 로 렌더링하며, 권한이 없는 화면은 서버에서 403 으로 차단한다. 화면의 데이터 처리는 동일한 REST API 를 호출한다 (public/js/pages/*.js).	routes/pages.js, views/*.ejs	10장 CR-02 절차 (수동 확인)
CR-03	-	mysql2 + mongodb 드라이버 (db/mysql.js, db/mongo.js). DR-05 참고.		10장 CR-03 절차 (1단계)
CR-04	GET	`/api/api-catalog` API-221 API 카탈로그 (전체 API 와 연결된 요구사항 - 요구사항 추적용, 공개)	`routes/admin.js:297`	10장 CR-04 절차 (수동 확인)
CR-05	-	config / db(연결·스키마·시드) / lib(API 레지스트리·검증·인증·보안·오류·기록) / services(업무 규칙·주기 작업) / routes(기능별 API 26개 모듈 + 화면 라우터) / views(EJS) / public(화면 스크립트·스타일) 로 분리.	전체 디렉터리 구조	10장 CR-05 절차 (수동 확인)
CR-06	GET	`/api/health` API-220 서버 및 DB 연결 상태 확인 (공개)	`routes/admin.js:287`	10장 CR-06 절차 (수동 확인)

10. 기능 확인 방법 (테스트 절차)

공통 준비

1. `node app.js` 로 서버 실행 → `http://{서버주소}:3000/api-docs` 접속
2. **각 요구사항(아래 소제목) 확인을 시작할 때마다** admin 으로 로그인하여 `POST /api/admin/system/reset-demo` Body `{"confirm":"RESET"}` 실행 → 데모 데이터 초기화
3. 각 단계의 **[계정]** 으로 `POST /api/auth/login` (비밀번호 `pass1234`) → 받은 token 을 Authorize 에 입력. **[인증 없음]** 은 Authorize 를 Logout 한 상태로 실행
4. 표의 Method / URL / Body 로 실행하고 **기대 결과**(HTTP 상태 코드, 업무 규칙 ID = 응답의 `error.details.rule`, 확인 내용)를 확인

모든 절차는 `node scripts/verify-report.js` 로 동일하게 자동 실행·검증되었다 (총 207단계 전체 통과).

기능 요구사항

FR-01 시스템 구성원의 기본 정보를 등록하고 관리할 수 있어야 한다.

구현: MySQL members 테이블에 계정·기본 정보를 저장. 관리자 등록(POST /api/members), 본인 가입 신청(POST /api/auth/register, PENDING 상태), 정보 수정(PATCH /api/members/{id}, PATCH /api/me), 기술 관리.

#	계정	요청	기대 결과
1	**boss**	**POST** `/api/members` `{"username":"soldier7","password":"pass1234","name":"Gino Vitale","role":"SOLDIER","crewId":1}`	**201** data.id = 20, role = SOLDIER
2	**boss**	**PATCH** `/api/members/20` `{"phone":"010-9999-0000","alias":"Gino"}`	**200** data.phone = 010-9999-0000
3	인증 없음	**POST** `/api/auth/register` `{"username":"newcomer1","password":"pass1234","name":"홍길동"}`	**201** data.status = PENDING (가입 대기)
4	**boss**	**GET** `/api/members?status=PENDING`	**200** newcomer1, pending1 이 조회됨

FR-02 구성원은 자신의 현재 소속, 역할, 상태 및 허용된 범위의 활동 정보를 확인할 수 있어야 한다.

구현: GET /api/me 가 소속 Crew·역할·상태·활동 요약을, /api/me/dashboard 가 일정·알림·요청을, /api/auth/permissions 가 권한과 관리 범위를 반환.

#	계정	요청	기대 결과

1	**soldier1**	GET `/api/me`	**200** role = SOLDIER, statusLabel = 활동중, crews[0].crewName = Brooklyn Crew
2	**soldier1**	GET `/api/auth/permissions`	**200** scope = SELF (관리 범위 없음)
3	**soldier1**	GET `/api/members/8/crew-history`	**200** 본인 소속 이력 1건
4	**soldier1**	GET `/api/me/dashboard`	**200** upcoming 에 1번 업무(물류 창고 정기 점검)

FR-03 권한을 가진 사용자는 구성원의 역할, 상태 또는 소속 정보를 변경할 수 있어야 한다.

구현: 역할 변경 PATCH /api/members/{id}/role, 상태 변경 PATCH /api/members/{id}/status, 소속 이동 POST /api/crews/{id}/members/{memberId}/transfer. 모두 권한(member.role / member.status / crew.members)과 관리 범위를 서버에서 검사.

#	계정	요청	기대 결과
1	**capo1**	PATCH `/api/members/14/role` `{"role":"SOLDIER","reason":"승급"}`	**200** from = ASSOCIATE, to = SOLDIER
2	**capo1**	PATCH `/api/members/9/status` `{"status":"STANDBY","reason":"휴가"}`	**200** from = ACTIVE, to = STANDBY
3	**underboss 1**	POST `/api/crews/1/members/9/transfer` `{"targetCrewId":2,"reason":"인력 보강"}`	**200** fromCrewId = 1, toCrewId = 2
4	**underboss 1**	GET `/api/members/9/crew-history`	**200** Brooklyn(leftAt 기록) → Harbor 이력 2건

FR-04 구성원은 활동중, 대기, 정지, 탈퇴 등 운영상 필요한 상태로 구분하여 관리할 수 있어야 한다.

구현: 상태 체계: PENDING(가입대기) / ACTIVE(활동중) / STANDBY(대기) / SUSPENDED(정지) / WITHDRAWN(탈퇴). 허용 전이표를 서버에서 검사하며, 탈퇴 시 소속 종료·배정 해제·예약 취소·세션 만료를 함께 처리.

#	계정	요청	기대 결과
1	**boss**	GET `/api/members/8/status-transitions`	**200** allowed = [STANDBY, SUSPENDED, WITHDRAWN]
2	**boss**	PATCH `/api/members/8/status` `{"status":"SUSPENDED","reason":"조사"}`	**200**
3	**boss**	PATCH `/api/members/18/status` `{"status":"ACTIVE"}`	**422** WITHDRAWN → ACTIVE 전이 불가 (INVALID_STATE)

4	**boss**	GET `/api/members?status=SUSPENDED`	**200** soldier1(8), suspended1(16) 2명

FR-05 시스템은 하나 이상의 Crew를 관리할 수 있어야 하며, 각 Crew의 책임자와 구성원을 확인할 수 있어야 한다.

구현: MySQL crews(책임자 capo_id, 감독 underboss_id, 지역 region_id) + crew_memberships. Crew 생성/조회/구성원 목록/책임자 지정 API 제공.

#	계정	요청	기대 결과
1	**soldier1**	GET `/api/crews`	**200** Crew 3개
2	**soldier1**	GET `/api/crews/1`	**200** capoName = Tony Russo, underbossName = Marco Bellini
3	**soldier1**	GET `/api/crews/1/members`	**200** capo1(isCapo=true) 포함 구성원 목록
4	**boss**	POST `/api/crews` `{"name":"Queens Crew","regionId":1,"underbossId":4}`	**201**
5	**boss**	PUT `/api/crews/4/capo` `{"memberId":7}`	**409** capo3 은 이미 Uptown Crew 책임자 (CONFLICT)

FR-06 권한을 가진 사용자는 자신이 관리할 수 있는 범위에서 Crew의 구성원 편성을 변경할 수 있어야 한다.

구현: Crew 편성 추가/제외/이동 API. 요청자의 관리 범위(Capo=담당 Crew, Underboss=감독 Crew, Boss=전체)를 services/scope.js 에서 판정.

#	계정	요청	기대 결과
1	**capo1**	DELETE `/api/crews/1/members/14?reason=편성 조정`	**200** removed = true
2	**capo1**	POST `/api/crews/1/members` `{"memberId":14,"membershipType":"FORMAL"}`	**201**
3	**capo1**	GET `/api/crews/1/membership-history`	**200** associate1 의 제외(leftAt)·재편성 이력
4	**capo1**	POST `/api/crews/2/members` `{"memberId":17}`	**403** BR-05 담당하지 않는 Harbor Crew 편성 거부

FR-07 권한을 가진 사용자는 새로운 업무를 등록하고 업무의 기본 정보와 일정, 중요도, 상태를 관리할

수 있어야 한다.

구현: MySQL tasks. 등록·기본정보 수정·일정 변경(충돌 재검사)·상태 변경을 별도 API 로 분리. 담당 Crew 범위 밖 업무 등록은 403.

#	계정	요청	기대 결과
1	**capo1**	**POST** `/api/tasks` `{"title":"신규 거래처 실사","crewId":1,"priority":"HIGH","startAt":"2030-03-02T09:00:00Z","endAt":"2030-03-02T12:00:00Z","location":"Brooklyn 5번가"}`	**201** data.id = 25, status = PLANNED
2	**capo1**	**PATCH** `/api/tasks/25` `{"priority":"CRITICAL","description":"우선 처리"}`	**200** priority = CRITICAL
3	**capo1**	**PATCH** `/api/tasks/25/schedule` `{"startAt":"2030-03-03T09:00:00Z","endAt":"2030-03-03T11:00:00Z"}`	**200**
4	**capo1**	**PATCH** `/api/tasks/25/status` `{"status":"IN_PROGRESS"}`	**200** from = PLANNED, to = IN_PROGRESS
5	**capo1**	**POST** `/api/tasks` `{"title":"타 Crew 업무","crewId":2,"startAt":"2030-03-02T09:00:00Z","endAt":"2030-03-02T12:00:00Z"}`	**403** 관리하지 않는 Harbor Crew 업무 등록 거부

FR-08 하나의 업무에는 한 명 이상의 구성원이 참여할 수 있어야 한다.

구현: MySQL task_assignments (업무 N : 구성원 M). 업무 내 역할·지시사항·응답 상태·참여 기록 관리.

#	계정	요청	기대 결과
1	**capo1**	**POST** `/api/tasks/2/assignments` `{"memberId":14,"taskRole":"운전"}`	**201**
2	**capo1**	**POST** `/api/tasks/2/assignments` `{"memberId":15,"taskRole":"경비"}`	**201**
3	**capo1**	**GET** `/api/tasks/2/assignments`	**200** 참여 구성원 2명

FR-09 구성원은 자신에게 배정된 업무와 해당 업무에서 자신이 확인해야 할 정보를 조회할 수 있어야 한다.

구현: GET /api/me/tasks (수락 필요·체크리스트 미완료 등 actionRequired), GET /api/me/tasks/{taskId} (역할·지시사항·동료·자원·체크리스트·코멘트 브리핑), 배정 수락/거절, 알림.

#	계정	요청	기대 결과
1	**soldier2**	**GET** `/api/me/tasks`	**200** 1번 업무, actionRequired 에 "배정 수락/거절 응답 필요"

2	**soldier2**	**GET** `/api/me/tasks/1`	**200** myAssignment.taskRole = 경비, resources 에 VH-001, checklist 2건
3	**soldier2**	**POST** `/api/me/assignments/2/accept`	**200** status = ACCEPTED
4	**soldier2**	**GET** `/api/me/notifications`	**200** TASK_ASSIGNED 알림
5	**soldier2**	**GET** `/api/me/tasks/3`	**403** 배정되지 않은 업무 브리핑 조회 거부

FR-10 업무의 진행 상태는 수행 과정에 따라 변경될 수 있어야 하며, 변경 이력을 추후 확인할 수 있어야 한다.

구현: 상태 전이표(PLANNED→IN_PROGRESS→ON_HOLD/COMPLETED…)를 서버에서 검사. 모든 변경은 MongoDB task_history 에 기록되고 status-history / timeline API 로 조회.

#	계정	요청	기대 결과
1	**capo2**	**GET** `/api/tasks/9/transitions`	**200** current = ON_HOLD, allowed = [IN_PROGRESS, CANCELLED]
2	**capo2**	**PATCH** `/api/tasks/9/status` `{"status":"IN_PROGRESS","reason":"부품 도착, 작업 재개"}`	**200**
3	**capo2**	**PATCH** `/api/tasks/9/status` `{"status":"PLANNED"}`	**422** IN_PROGRESS → PLANNED 전이 불가
4	**capo2**	**GET** `/api/tasks/9/status-history`	**200** STATUS_CHANGED 이력 2건 (ON_HOLD→IN_PROGRESS 포함)

FR-11 완료된 업무의 결과와 참여 기록은 이후 평가 및 운영 현황 확인에 활용할 수 있어야 한다.

구현: POST /api/tasks/{id}/complete 로 결과(성공/부분/실패, 요약, 성과값)와 참여자별 출석·기여도를 저장. 결과 조회·구성원 참여 이력·평가 대기 목록·통계에 활용.

#	계정	요청	기대 결과
1	**capo2**	**POST** `/api/tasks/3/complete` `{"outcome":"SUCCESS","summary":"화물 인수 완료","value":15000,"participation":[{"memberId":10,"attendance":"PRESENT","contribution":90},{"memberId":11,"attendance":"PRESENT","contribution":70}]}`	**200** status = COMPLETED, resultOutcome = SUCCESS

2	**capo2**	GET `/api/tasks/3/result`	**200** 참여자 2명의 attendance / contribution
3	**capo2**	GET `/api/members/10/assignments?taskStatus=COMPLETED`	**200** 3번 업무 contribution = 90
4	**capo2**	GET `/api/evaluations/pending`	**200** 3번 업무 참여자(10, 11)가 평가 대기 목록에 표시
5	**boss**	GET `/api/stats/crews`	**200** Crew 별 완료율·성공률·성과값(totalValue) 반영

FR-12 권한을 가진 사용자는 특정 기간, Crew, 상태, 담당 구성원 등을 기준으로 업무를 탐색할 수 있어야 한다.

구현: GET /api/tasks 가 from/to(기간), crewId, status(다중), priority(다중), memberId(담당자), q(검색어), overdue, unassigned 필터와 정렬·페이지를 지원. 역할별 조회 범위 자동 적용.

#	계정	요청	기대 결과
1	**boss**	GET `/api/tasks?crewId=1&status=PLANNED,IN_PROGRESS,ON_HOLD&sort=startAt`	**200** 모든 결과 crewId = 1
2	**boss**	GET `/api/tasks?memberId=8`	**200** soldier1 이 배정된 1번 업무
3	**boss**	GET `/api/tasks?from=2020-01-01T00:00:00Z&to=2035-12-31T00:00:00Z&priority=HIGH,CRITICAL&sort=-priority`	**200** HIGH/CRITICAL 업무만
4	**boss**	GET `/api/tasks?q=창고`	**200** 제목/설명/장소에 "창고" 포함 업무
5	**capo1**	GET `/api/tasks?crewId=2`	**200** 관리 범위 밖 Crew → 0건

FR-13 조직에서 공동으로 사용하는 자원을 등록하고 현재 상태를 관리할 수 있어야 한다.

구현: MySQL resources / resource_types. 자원 등록·수정·상태 변경(정상/사용중/점검필요/정비중/사용불가/폐기). 상태 변경은 MongoDB resource_history 에 기록.

#	계정	요청	기대 결과
1	**underboss1**	POST `/api/resources` `{"code":"VH-005","name":"소형 승합차","typeCode":"VEHICLE","crewId":1,"location":"Brooklyn 차고"}`	**201** data.id = 10, status = AVAILABLE
2	**underboss1**	PATCH `/api/resources/10` `{"location":"Harbor 차고"}`	**200**

3	**underboss1**	**PATCH** `/api/resources/10/status` `{"status":"MAINTENANCE","reason":"정기 정비"}`	**200** from = AVAILABLE, to = MAINTENANCE
4	**underboss1**	**GET** `/api/resources?status=MAINTENANCE`	**200** VH-003, VH-005

FR-14 사용자는 자신의 권한과 자원의 상태에 따라 이용 가능한 자원을 확인할 수 있어야 한다.

구현: 자원마다 이용 최소 역할 등급(minRoleLevel)과 소유 Crew 를 두고, 조회 시 서버에서 필터링. /api/resources/available 은 상태·기존 배정까지 고려.

#	계정	요청	기대 결과
1	**soldier1**	**GET** `/api/resources`	**200** 등급 3 자원(창고/은신처)과 타 Crew 자원 제외
2	**soldier1**	**GET** `/api/resources/available?startAt=2030-01-01T00:00:00Z&endAt=2030-01-01T02:00:00Z`	**200** 사용 가능한(usable=true) 자원만
3	**soldier1**	**GET** `/api/resources/4`	**403** 등급 3 창고(WH-001) 접근 거부
4	**boss**	**GET** `/api/resources/4`	**200**

FR-15 자원의 사용, 반환 및 상태 변경 이력은 추후 확인할 수 있어야 한다.

구현: 배정(예약)→체크아웃(사용)→반납 흐름을 MySQL resource_allocations 에 저장하고, 모든 이벤트를 MongoDB resource_history 에 기록.

#	계정	요청	기대 결과
1	**boss**	**POST** `/api/allocations` `{"resourceId":5,"memberId":8,"startAt":"2030-01-10T09:00:00Z","endAt":"2030-01-10T18:00:00Z","purpose":"연락용"}`	**201** data.id = 4, status = RESERVED
2	**soldier1**	**POST** `/api/allocations/4/checkout`	**200** status = IN_USE
3	**soldier1**	**POST** `/api/allocations/4/return` `{"condition":"GOOD","note":"이상 없음"}`	**200** status = RETURNED, resourceStatus = AVAILABLE
4	**boss**	**GET** `/api/resources/5/history`	**200** ALLOCATED, CHECKED_OUT, RETURNED 이벤트
5	**boss**	**GET** `/api/allocations?resourceId=5`	**200** 사용 기록 2건(과거 1 + 신규 1)

FR-16 사용 불가 또는 점검이 필요한 자원은 정상 자원과 구분하여 관리되어야 한다.

구현: NEEDS_INSPECTION / MAINTENANCE / UNAVAILABLE / RETIRED 상태를 정상 자원과 구분(usable=false). 이상 신고 시 자동으로 점검 필요 전환, /api/resources/attention 에서 별도 관리, 해결 처리 시 복구.

#	계정	요청	기대 결과

1	**boss**	GET /api/resources/attention	**200** blocked 에 VH-003(정비중), CM-002(사용불가), EQ-001(점검필요)
2	**boss**	GET /api/resources?usable=false	**200** 사용 불가 계열 자원 3건
3	**soldier1**	POST /api/resources/1/issues `{"severity":"MEDIUM","description":"타이어 경고등 점등"}`	**201** resourceStatus = NEEDS_INSPECTION
4	**capo1**	POST /api/allocations `{"resourceId":1,"memberId":8,"startAt":"2030-01-10T09:00:00Z","endAt":"2030-01-10T18:00:00Z"}`	**409** BR-08 점검 필요 자원 배정 거부
5	**underboss 1**	POST /api/resource-issues/2/resolve `{"resolution":"점검 완료"}`	**200** status = RESOLVED, resourceStatus = AVAILABLE

FR-17 권한을 가진 사용자는 업무 수행에 필요한 자원을 업무 또는 구성원에게 배정할 수 있어야 한다.

구현: POST /api/allocations (업무 또는 구성원 지정, 업무 배정 시 일정 자동 적용). 구성원은 사용 요청(POST /api/resource-requests)을 하고 관리자가 승인하면 배정 생성.

#	계정	요청	기대 결과
1	**capo3**	POST /api/allocations `{"resourceId":9,"taskId":5,"purpose":"답사 이동"}`	**201** 5번 업무 일정으로 VH-004 예약
2	**capo1**	POST /api/resource-requests/1/approve `{"comment":"승인"}`	**200** status = APPROVED, allocationId 생성
3	**soldier2**	GET /api/me/dashboard	**200** allocations 에 CM-001(무전기 세트 1)

FR-18 구성원의 활동 결과를 바탕으로 평가 또는 평판 정보를 기록할 수 있어야 한다.

구현: MongoDB evaluations(항목별 점수·가중 평균·수정 이력), evaluation_criteria(평가 항목/가중치), reputation_ledger(평판 가감점). 완료 업무 참여자만 업무 기반 평가 가능.

#	계정	요청	기대 결과
1	**capo2**	POST /api/tasks/3/complete `{"outcome":"SUCCESS","summary":"화물 인수 완료"}`	**200**
2	**capo2**	POST /api/evaluations `{"memberId":10,"taskId":3,"scores":{"loyalty":4,"skill":5,"discretion":4,"teamwork":4},"comment":"화물 인수 책임 완수"}`	**201** overall(가중 평균) 자동 계산

3	**underboss1**	POST `/api/members/8/reputation-adjustments` `{"points":5,"reason":"위기 상황 신속 대응"}`	**201**
4	**soldier3**	GET `/api/evaluations`	**200** 본인 평가 조회 (평가자 이름은 익명)

FR-19 권한을 가진 사용자는 필요한 범위에서 구성원의 활동 및 평가 이력을 확인할 수 있어야 한다.

구현: evaluation.read / member.read 권한 + 관리 범위 내 구성원에 대해서만 평가·평판·참여 이력·활동 기록 조회 허용.

#	계정	요청	기대 결과
1	**capo1**	GET `/api/evaluations?memberId=8`	**200** soldier1 평가 1건
2	**capo1**	GET `/api/members/8/reputation`	**200** evaluationAverage, ledgerPoints, attendanceRate, score
3	**capo1**	GET `/api/activities?memberId=8`	**200** soldier1 활동 기록
4	**capo1**	GET `/api/members/10/reputation`	**403** 관리 범위 밖(Harbor) 구성원 평판 조회 거부
5	**soldier1**	GET `/api/members/9/reputation`	**403** 권한 없는 사용자 거부

FR-20 시스템은 중요 상태 변경과 관리 활동을 추후 확인할 수 있도록 기록해야 한다.

구현: MongoDB audit_logs 에 행위자·행위·대상·상세·결과(SUCCESS/DENIED)·IP 기록. 권한 거부 시도도 자동 기록. 조회·집계·CSV 내보내기 제공.

#	계정	요청	기대 결과
1	**capo1**	PATCH `/api/members/14/role` `{"role":"SOLDIER","reason":"승급"}`	**200**
2	**soldier1**	GET `/api/admin/accounts`	**403**
3	**admin**	GET `/api/audit-logs?action=MEMBER_ROLE_CHANGED`	**200** capo1 의 역할 변경 기록
4	**admin**	GET `/api/audit-logs?result=DENIED`	**200** soldier1 의 거부된 접근 기록
5	**admin**	GET `/api/audit-logs/summary`	**200** 행위별/결과별/일자별 집계

FR-21 사용자는 권한 범위 내에서 구성원, Crew, 업무, 자원 등의 정보를 검색하거나 조건에 따라 조회할 수 있어야 한다.

구현: 통합 검색(/api/search), 자동완성, 대상별 목록 API 의 필터·정렬·페이지. 모든 검색에 역할별 조회 범위가 SQL 조

건으로 적용.

#	계정	요청	기대 결과
1	**soldier1**	GET `/api/search?q=Brooklyn`	**200** crews/tasks/members/resources 별 결과
2	**capo1**	GET `/api/members?q=Paulie&sort=name`	**200** soldier1(Paulie Conti)
3	**boss**	GET `/api/resources?q=세단`	**200** VH-001 검정 세단
4	**boss**	GET `/api/crews?q=Harbor`	**200**
5	**soldier1**	GET `/api/members?crewId=3`	**200** 권한 범위 밖(Uptown) 구성원 0건

FR-22 관리자는 기간, Crew, 구성원 상태, 업무 상태 등 시스템에 축적된 정보를 기반으로 운영 현황을 확인할 수 있어야 한다.

구현: /api/stats/* 11종 (KPI 요약, 구성원 분포, Crew 비교, 업무 분포/추이, 업무량, 자원 가동률/이슈, 평가, 활동 히트맵, 상태 변경 추이). Underboss 는 감독 Crew 범위로 자동 제한.

#	계정	요청	기대 결과
1	**admin**	GET `/api/stats/overview`	**200** members / tasks / resources KPI
2	**boss**	GET `/api/stats/crews?from=2026-01-01T00:00:00Z`	**200** Crew 3개 비교
3	**boss**	GET `/api/stats/tasks?crewId=1`	**200** byStatus / byPriority / byOutcome
4	**boss**	GET `/api/stats/members`	**200** 상태별·역할별·Crew별 구성원 분포
5	**boss**	GET `/api/stats/tasks/trend?interval=month`	**200**
6	**underboss2**	GET `/api/stats/overview`	**200** scope = [3] (감독 Crew 만)
7	**soldier1**	GET `/api/stats/overview`	**403**

FR-23 관리자는 계정 및 권한과 관련된 필요한 관리 기능을 수행할 수 있어야 한다.

구현: 계정 목록·잠금/해제·비밀번호 초기화·강제 로그아웃·아이디 변경·로그인 이력·실패 탐지, 역할별 권한 매트릭스 관리(즉시 반영), 시스템 설정, 무결성 점검, 데모 초기화.

#	계정	요청	기대 결과
1	**admin**	GET `/api/admin/accounts?sort=-lastLoginAt`	**200**

2	**admin**	**POST** `/api/admin/accounts/9/lock` `{"reason":"보안 조사"}`	**200** isLocked = true
3	인증 없음	**POST** `/api/auth/login` `{"username":"soldier2","password":"pass1234"}`	**401** 잠긴 계정 로그인 차단
4	**admin**	**POST** `/api/admin/accounts/9/unlock`	**200**
5	**capo1**	**GET** `/api/stats/overview`	**403** Capo 는 기본적으로 통계 권한 없음
6	**admin**	**POST** `/api/roles/CAPO/permissions` `{"permission":"stats.read"}`	**201**
7	**capo1**	**GET** `/api/stats/overview`	**200** 권한 부여가 즉시 반영됨
8	**admin**	**PATCH** `/api/admin/settings` `{"maxActiveAssignmentsPerMember":3}`	**200**

업무 규칙

BR-01 하나의 구성원은 동시에 둘 이상의 Crew에 정식 소속될 수 없다.

구현: ① 서버: 편성/가입승인/이동 시 기존 정식 소속을 FOR UPDATE 로 잠그고 검사. ② DB: crew_memberships 의 생성 컬럼 active_formal_key 에 UNIQUE 제약 → 어떤 경로로도 위반 불가. 이동은 소속 종료+신규 소속을 하나의 트랜잭션으로 처리.

#	계정	요청	기대 결과
1	**capo1**	**POST** `/api/crews/1/members` `{"memberId":10}`	**409** BR-01 Harbor Crew 정식 소속인 soldier3 의 정식 편성 거부
2	**capo1**	**POST** `/api/crews/1/members` `{"memberId":10,"membershipType":"AFFILIATE"}`	**201** 정식이 아닌 준소속(AFFILIATE)은 허용
3	**underboss1**	**POST** `/api/crews/2/members/10/transfer` `{"targetCrewId":1,"reason":"정식 이동"}`	**200** 이동(transfer)으로는 정식 소속 변경 가능
4	**admin**	**GET** `/api/admin/integrity-check`	**200** checks['BR-01 multipleFormalCrews'] = [] (위반 0건)

BR-02 정지 또는 탈퇴 상태의 구성원은 새로운 업무에 배정될 수 없다.

구현: 배정 트랜잭션에서 구성원 행을 FOR UPDATE 로 잠근 뒤 상태 검사(SUSPENDED/WITHDRAWN/PENDING 거부). 정지 징계 시 상태 자동 전환, 탈퇴 시 기존 미완료 배정 자동 해제.

#	계정	요청	기대 결과

1	**capo1**	**POST** `/api/tasks/2/assignments` `{"memberId":16}`	**409** BR-02 정지(SUSPENDED) 구성원 suspended1 배정 거부
2	**boss**	**POST** `/api/tasks/5/assignments` `{"memberId":18}`	**409** BR-02 탈퇴(WITHDRAWN) 구성원 배정 거부
3	**capo1**	**POST** `/api/members/9/disciplinary-actions` `{"actionType":"SUSPENSION","reason":"보고 누락"}`	**201** statusChange.to = SUSPENDED
4	**capo1**	**POST** `/api/tasks/2/assignments` `{"memberId":9}`	**409** BR-02 방금 정지된 soldier2 배정 거부
5	**capo1**	**POST** `/api/tasks/2/assignments` `{"memberId":14}`	**201** 활동중 구성원은 정상 배정

BR-03 동일 시간대에 동시에 수행할 수 없는 업무가 하나의 구성원에게 중복 배정되어서는 안 된다.

구현: 배정·인원교체·일정변경·취소복구 시 구성원의 진행 전/중 업무와 시간 구간 겹침(start<기존end AND end>기존start)을 검사. 두 업무가 모두 allowConcurrent=true 인 경우만 예외. 구성원 행 잠금으로 동시 요청에도 1건만 성공.

#	계정	요청	기대 결과
1	**capo1**	**GET** `/api/tasks/1`	**200** 1번 업무의 startAt / endAt 확인 (soldier1 배정됨) (응답 값을 다음 단계에서 사용)
2	**capo1**	**POST** `/api/tasks/2/assignments` `{"memberId":8}`	**409** BR-03 1번 업무와 시간이 겹치는 2번 업무에 soldier1 배정 거부 (details.conflicts 에 1번 업무)
3	**capo1**	**GET** `/api/tasks/2/candidates`	**200** 후보 목록에 soldier1(8) 제외
4	**capo1**	**POST** `/api/tasks` `{"title":"중복 시간 업무","crewId":1,"startAt":"(앞 단계 응답의 startAt 값)","endAt":"(앞 단계 응답의 endAt 값)"}`	**201** 1번 업무와 같은 시간의 새 업무 (id = 25)
5	**capo1**	**POST** `/api/tasks/25/assignments` `{"memberId":8}`	**409** BR-03
6	**capo1**	**POST** `/api/tasks/25/assignments` `{"memberId":14}`	**201** 일정이 비어 있는 associate1 은 배정 가능
7	**capo1**	**PATCH** `/api/tasks/2/schedule` `{"startAt":"2030-04-01T09:00:00Z","endAt":"2030-04-01T10:00:00Z"}`	**200**

8	**capo1**	**POST** `/api/tasks/2/assignments` `{"memberId":8}`	**201** 일정을 옮긴 뒤에는 배정 가능

BR-04 구성원은 자신보다 높은 권한을 가진 사용자의 역할을 임의로 변경할 수 없다.

구현: 역할/상태/징계/계정 변경 시 대상의 현재 등급과 부여할 역할 등급이 모두 요청자보다 낮아야 함(Administrator 예외). 자기 자신 변경 금지. Capo 는 Associate↔Soldier 만 변경.

#	계정	요청	기대 결과
1	**capo1**	**PATCH** `/api/members/3/role` `{"role":"SOLDIER"}`	**403** BR-04 Capo → Underboss 역할 변경 거부
2	**capo1**	**PATCH** `/api/members/14/role` `{"role":"CAPO"}`	**403** BR-04 자신과 같은 등급 부여 거부
3	**capo1**	**PATCH** `/api/members/5/role` `{"role":"SOLDIER"}`	**403** BR-04 자기 자신 역할 변경 거부
4	**underboss1**	**PATCH** `/api/members/2/status` `{"status":"SUSPENDED"}`	**403** BR-04 Underboss → Boss 상태 변경 거부
5	**capo1**	**PATCH** `/api/members/14/role` `{"role":"SOLDIER","reason":"승급"}`	**200** 하위 구성원은 변경 가능

BR-05 Crew 책임자는 자신이 관리하지 않는 Crew의 구성원 편성을 임의로 변경할 수 없다.

구현: 모든 편성 API(추가/제외/이동/가입승인/책임자 지정)에서 services/scope.js 의 assertManageCrew 로 대상 Crew(이동은 출발·도착 모두)가 관리 범위인지 검사.

#	계정	요청	기대 결과
1	**capo1**	**POST** `/api/crews/2/members` `{"memberId":17}`	**403** BR-05 Harbor Crew 편성 거부
2	**capo1**	**DELETE** `/api/crews/2/members/11`	**403** BR-05 Harbor Crew 구성원 제외 거부
3	**capo1**	**POST** `/api/crews/1/members/9/transfer` `{"targetCrewId":2}`	**403** BR-05 도착 Crew 가 관리 범위 밖이므로 이동 거부
4	**capo2**	**DELETE** `/api/crews/2/members/17`	**200** 자신의 Crew 는 편성 변경 가능

BR-06 완료된 업무의 핵심 결과 정보는 일반 구성원이 임의로 변경할 수 없어야 한다.

구현: 완료 업무의 기본정보·일정·상태·배정·체크리스트 변경을 서버에서 차단. 결과 수정/재개는 task.result.amend 권한(Underboss 이상)만 가능하며 사유와 변경 전 값이 이력에 남음. 기록된 참여 결과의 재수정도 동일.

#	계정	요청	기대 결과

1	**capo2**	**POST** `/api/tasks/3/complete` `{"outcome":"SUCCESS","summary":"화물 인수 완료","participation":[{"memberId":11,"attendance":"PRESENT","contribution":90}]}`	**200**
2	**soldier3**	**PATCH** `/api/tasks/3/result` `{"outcome":"FAILURE","reason":"변경 시도"}`	**403** 일반 구성원 결과 수정 거부
3	**capo2**	**PATCH** `/api/tasks/3/result` `{"outcome":"FAILURE","reason":"변경 시도"}`	**403** Capo 도 결과 수정 권한 없음
4	**capo2**	**PATCH** `/api/tasks/3` `{"title":"제목 변경"}`	**409** BR-06 완료 업무 정보 변경 거부
5	**capo2**	**PUT** `/api/tasks/3/assignments/11/participation` `{"attendance":"ABSENT","contribution":0}`	**409** BR-06 기록된 참여 결과 재수정 거부
6	**underboss1**	**PATCH** `/api/tasks/3/result` `{"value":14000,"reason":"정산 반영"}`	**200** Underboss 는 사유와 함께 수정 가능
7	**underboss1**	**GET** `/api/tasks/3/status-history?event=RESULT_AMENDED`	**200** from(변경 전 값) / to / note(사유) 기록

BR-07 이미 사용 중인 자원은 동일 시간대에 다른 업무 또는 사용자에게 중복 배정될 수 없다.

구현: 배정·기간변경·요청승인·업무 일정변경 시 자원 행을 FOR UPDATE 로 잠그고 RESERVED/IN_USE 배정과 시간 겹침 검사. 체크아웃 시 미반납 사용자가 있으면 거부.

#	계정	요청	기대 결과
1	**capo1**	**POST** `/api/allocations` `{"resourceId":1,"taskId":2}`	**409** BR-07 1번 업무에 예약된 VH-001 을 시간이 겹치는 2번 업무에 배정 거부
2	**boss**	**POST** `/api/allocations` `{"resourceId":9,"memberId":12,"startAt":"2030-02-01T10:00:00Z","endAt":"2030-02-01T12:00:00Z"}`	**201**
3	**boss**	**POST** `/api/allocations` `{"resourceId":9,"memberId":13,"startAt":"2030-02-01T11:00:00Z","endAt":"2030-02-01T13:00:00Z"}`	**409** BR-07 11~12시 겹침
4	**boss**	**POST** `/api/allocations` `{"resourceId":9,"memberId":13,"startAt":"2030-02-01T12:00:00Z","endAt":"2030-02-01T14:00:00Z"}`	**201** 겹치지 않는 연속 시간은 허용

BR-08 사용 불가 상태의 자원은 새로운 업무에 배정할 수 없다.

구현: NEEDS_INSPECTION / MAINTENANCE / UNAVAILABLE / RETIRED 자원은 배정·기간변경·사용요청·체크아웃 모두 거부.

#	계정	요청	기대 결과
1	**boss**	POST /api/allocations `{"resourceId":3,"taskId":5}`	**409** BR-08 정비중(MAINTENANCE) VH-003 배정 거부
2	**boss**	POST /api/allocations `{"resourceId":6,"memberId":8,"startAt":"2030-01-10T09:00:00Z","endAt":"2030-01-10T18:00:00Z"}`	**409** BR-08 사용 불가(UNAVAILABLE) CM-002
3	**soldier1**	POST /api/resource-requests `{"resourceId":8,"startAt":"2030-01-10T09:00:00Z","endAt":"2030-01-10T18:00:00Z","purpose":"금고 작업"}`	**409** BR-08 점검 필요(NEEDS_INSPECTION) EQ-001 사용 요청 거부
4	**underboss1**	PATCH /api/resources/3/status `{"status":"AVAILABLE","reason":"정비 완료"}`	**200**
5	**boss**	POST /api/allocations `{"resourceId":3,"taskId":5}`	**201** 정상 상태로 복구 후 배정 가능

BR-09 권한이 없는 사용자는 제한된 조직 정보 또는 관리 기능에 직접 URL로 접근하더라도 작업을 수행할 수 없어야 한다.

구현: 모든 보호 API 에 인증 미들웨어 + 권한 코드 검사 + 데이터 범위 검사를 서버에서 적용(화면 표시와 무관). 매 요청마다 DB 에서 최신 역할/상태/잠금/토큰버전을 확인. 거부 시도는 감사 로그에 DENIED 로 기록.

#	계정	요청	기대 결과
1	인증 없음	GET /api/members	**401** 토큰 없이 접근 거부
2	**soldier1**	GET /api/audit-logs	**403** BR-09
3	**soldier1**	DELETE /api/members/19	**403** BR-09 관리 기능 URL 직접 호출 거부
4	**associate1**	PATCH /api/admin/settings `{"loginLockThreshold":3}`	**403** BR-09
5	**capo1**	GET /api/tasks/3	**403** BR-09 다른 Crew 업무 상세 조회 거부
6	**admin**	GET /api/audit-logs?result=DENIED	**200** 위 거부 시도들이 DENIED 로 기록됨

BR-10 구성원의 역할, 소속, 상태 및 평가와 관련된 주요 변경 이력은 추후 확인할 수 있어야 한다.

구현: 역할·소속·상태·평가·평판·징계 변경을 MongoDB member_history 에 이전 값/새 값/사유/변경자/시각과 함께 기

록. 소속 이력은 MySQL crew_memberships(joined_at/left_at)에도 보존.

#	계정	요청	기대 결과
1	**capo1**	**PATCH** `/api/members/14/role` `{"role":"SOLDIER","reason":"승급"}`	**200**
2	**underboss 1**	**POST** `/api/crews/1/members/14/transfer` `{"targetCrewId":2,"reason":"항만 지원"}`	**200**
3	**capo2**	**PATCH** `/api/members/14/status` `{"status":"STANDBY","reason":"대기 발령"}`	**200**
4	**capo2**	**GET** `/api/members/14/history`	**200** ROLE / CREW / STATUS 이력 (from, to, reason, by, at)
5	**boss**	**GET** `/api/stats/status-changes`	**200** 변경 유형별 집계

데이터 요구사항

DR-01 계정, 권한, 구성원, Crew 소속 및 상태 사이의 관계가 일관성 있게 유지되어야 한다.

구현: MySQL InnoDB 의 FK, UNIQUE(생성 컬럼), CHECK, ENUM 제약과 트랜잭션으로 일관성 보장. roles–role_permissions–permissions, members–crew_memberships–crews 관계.

#	계정	요청	기대 결과
1	**admin**	**GET** `/api/admin/integrity-check`	**200** ok = true
2	**boss**	**DELETE** `/api/regions/1`	**422** Crew 가 참조 중인 지역 삭제 거부
3	**boss**	**DELETE** `/api/members/8`	**422** 이력이 있는 구성원은 삭제 대신 WITHDRAWN 처리
4	**boss**	**GET** `/api/roles/CAPO/permissions`	**200** 역할-권한 매트릭스

DR-02 업무와 참여 구성원 사이의 관계를 확인할 수 있어야 한다.

구현: MySQL task_assignments (task_id, member_id FK, 활성 배정 중복 방지 UNIQUE).

#	계정	요청	기대 결과
1	**boss**	**GET** `/api/tasks/1`	**200** assignees 2명
2	**boss**	**GET** `/api/tasks/1/assignments`	**200**
3	**boss**	**GET** `/api/members/8/assignments`	**200** soldier1 이 참여한 업무 목록

DR-03 공용 자원의 현재 상태와 사용 이력을 모두 확인할 수 있어야 한다.

구현: 현재 상태: MySQL resources.status + 활성 배정. 사용 이력: MySQL resource_allocations(배정/사용/반납 레코드) + MongoDB resource_history(이벤트 로그).

#	계정	요청	기대 결과
1	**boss**	GET `/api/resources/2`	**200** status = IN_USE, currentUse.allocationId = 2
2	**boss**	GET `/api/resources/2/history`	**200** CHECKED_OUT 이벤트
3	**boss**	GET `/api/allocations?resourceId=2`	**200**

DR-04 시간에 따라 지속적으로 생성되는 활동, 상태 변경, 기록 및 평가 정보를 저장할 수 있어야 한다.

구현: MongoDB 컬렉션: activities, member_history, task_history, resource_history, evaluations, audit_logs, notifications, login_history 등 (스키마 유연, 시간 인덱스).

#	계정	요청	기대 결과
1	**boss**	GET `/api/activities?size=5`	**200** 활동 기록 60건 이상
2	**soldier1**	POST `/api/activities` `{"type":"PATROL","description":"창고 주변 순찰","details":{"checkpoints":4,"incidents":0}}`	**201** 자유 형식 details 저장
3	**boss**	GET `/api/members/16/history`	**200** ACTIVE → SUSPENDED 이력
4	**boss**	GET `/api/evaluations`	**200**

DR-05 데이터의 특성에 따라 SQL와 MongoDB를 적절히 사용해야 하며, 두 DB를 모두 실제 기능에서 사용해야 한다.

구현: MySQL: 관계·일관성이 중요한 20개 테이블. MongoDB: 계속 증가하거나 구조가 유연한 19개 컬렉션. 대부분의 기능 흐름(예: 업무 완료)이 두 DB 를 함께 사용.

#	계정	요청	기대 결과
1	**admin**	GET `/api/admin/system/info`	**200** mysql.tables (20개) 와 mongo.collections 데이터 건수

DR-06 각 데이터베이스의 이름에는 본인의 학번이 포함되어야 하며, 필요한 Table 혹은 Collection의 구조를 직접 설계해야 한다.

구현: MySQL DB: family_{학번}, MongoDB DB: family_log_{학번} (config/index.js 의 STUDENT_ID).

#	계정	요청	기대 결과
1	**admin**	GET `/api/admin/system/info`	**200** mysql.database, mongo.database 에 학번 포함

REST API 요구사항

AR-01 각 API는 담당 기능의 목적에 적합한 HTTP Method를 사용해야 한다.

구현: 조회 GET, 생성 POST(201), 전체 교체 PUT, 부분 수정 PATCH, 삭제/취소 DELETE, 상태 전이 동작(승인·완료·체크아웃 등)은 하위 리소스 POST.

#	계정	요청	기대 결과
1	인증 없음	GET `/api/api-catalog?tag=Tasks`	**200** 업무 API 의 Method 구성 확인

AR-02 Path Parameter, Query Parameter, Header, Request Body는 데이터의 의미와 요청 목적에 맞게 사용해야 한다.

구현: 식별자=Path, 필터·정렬·페이지=Query, 인증 토큰=Authorization Header, 생성/변경 데이터=JSON Body.

#	계정	요청	기대 결과
1	**boss**	GET `/api/tasks/1`	**200** Path Parameter
2	**boss**	GET `/api/tasks?status=PLANNED&sort=-startAt`	**200** Query Parameter
3	**capo1**	POST `/api/tasks/1/comments` `{"body":"현장 확인 완료"}`	**201** Request Body + Authorization Header

AR-03 검색과 조회 기능은 필요한 경우 필터링, 정렬, 검색 및 페이지 단위 조회 등의 방법을 제공해야 한다.

구현: 목록 API 공통: page, size(최대 100), sort(허용 필드, -접두사 내림차순) + 리소스별 필터. 응답 meta 에 page/size/total/totalPages.

#	계정	요청	기대 결과
1	**boss**	GET `/api/members?q=o&status=ACTIVE&sort=-joinedAt&page=2&size=5`	**200** meta.page = 2, data 5건 이하
2	**boss**	GET `/api/tasks?sort=unknown`	**400** 허용되지 않은 정렬 기준

AR-04 API는 정상 요청뿐 아니라 잘못된 입력, 존재하지 않는 대상, 권한 부족 및 업무 규칙 위반을 적절한 방식으로 처리해야 한다.

구현: 400 VALIDATION_ERROR(필드별 details) / 401 / 403 FORBIDDEN(rule) / 404 NOT_FOUND / 409 BUSINESS_RULE_VIOLATION(rule) / 422 INVALID_STATE.

#	계정	요청	기대 결과
1	**capo1**	POST `/api/tasks` `{"title":"x"}`	**400** details 에 crewId/startAt/endAt 누락
2	**boss**	GET `/api/tasks/99999`	**404**
3	**soldier1**	GET `/api/audit-logs`	**403**
4	**capo1**	POST `/api/tasks/2/assignments` `{"memberId":8}`	**409** BR-03

5	**capo1**	PATCH `/api/tasks/1/status` `{"status":"ON_HOLD"}`	**422**

AR-05 API 요청과 응답의 형식과 오류 표현은 서비스 전반에서 일관된 방식으로 설계되어야 한다.

구현: 성공 { success:true, message, data, meta? } / 실패 { success:false, error:{ code, message, details } } 를 lib/api.js 와 lib/errorHandler.js 에서 일괄 생성.

#	계정	요청	기대 결과
1	**boss**	GET `/api/crews/1`	**200** success = true, data
2	**boss**	GET `/api/crews/999`	**404** success = false, error.code = NOT_FOUND

AR-06 전체 시스템에는 200개 이상의 유효한 REST API가 구현되어야 한다.

구현: lib/api.js 레지스트리에 등록된 API 수가 Swagger 와 카탈로그에 그대로 표시된다.

#	계정	요청	기대 결과
1	인증 없음	GET `/api/api-catalog`	**200** total ≥ 200

AR-07 동일한 동작을 URL, 이름 또는 Parameter만 바꾸어 반복 구현한 경우 서로 다른 API로 인정하지 않는다.

구현: 각 API 는 서로 다른 업무 동작을 수행하도록 설계(예: 상태 변경 vs 완료 처리(결과 입력) vs 결과 수정(권한·이력) vs 재개). 단순 필터 차이는 하나의 목록 API 의 Query Parameter 로 통합.

#	계정	요청	기대 결과
1	인증 없음	GET `/api/api-catalog`	**200** 각 API 의 summary 와 연결 요구사항 확인

AR-08 구현된 각 API는 최소 하나 이상의 시스템 요구사항과 연결하여 설명할 수 있어야 한다.

구현: 모든 API 정의에 reqs 배열이 필수로 있으며 Swagger 설명·카탈로그·본 보고서 추적표에 자동 반영.

#	계정	요청	기대 결과
1	인증 없음	GET `/api/api-catalog?req=BR-03`	**200** BR-03 과 연결된 API 목록

비기능 요구사항

NFR-01 사용자가 수행한 주요 작업의 성공 또는 실패 여부를 명확히 확인할 수 있어야 한다.

구현: 모든 응답에 success 플래그와 message, HTTP 상태 코드. 실패 시 error.code / message / details(위반 규칙 포함).

#	계정	요청	기대 결과

1	**capo1**	**PATCH** `/api/tasks/1/status` `{"status":"IN_PROGRESS"}`	**200** success = true, message
2	**capo1**	**PATCH** `/api/tasks/1/status` `{"status":"IN_PROGRESS"}`	**422** success = false, error.message 에 허용되지 않는 전이 안내

NFR-02 잘못된 입력 또는 비정상 요청으로 인해 서버 프로세스가 비정상적으로 종료되어서는 안 된다.

구현: 모든 핸들러 예외를 전역 오류 처리기로 전달, JSON 파싱 오류·DB 오류를 4xx 로 변환, unhandledRejection/uncaughtException 기록 후 계속 실행.

#	계정	요청	기대 결과
1	**capo1**	`curl -X POST http://localhost:3000/api/tasks -H "Authorization: Bearer {capo1 토큰}" -H "Content-Type: application/json" -d '{"title": "broken'`	**400** 잘못된 JSON → 400 (curl 로 확인)
2	**boss**	**GET** `/api/tasks/abc`	**400**
3	**boss**	**GET** `/api/members?q=' OR 1=1 --`	**200** SQL 인젝션 문자열도 안전하게 처리(0건)
4	인증 없음	**GET** `/api/health`	**200** status = UP (서버 정상 동작 유지)

NFR-03 권한 검사는 화면 표시 여부와 무관하게 서버 측에서도 적용되어야 한다.

구현: BR-09 와 동일. 웹 화면은 메뉴·버튼을 권한별로 숨기지만(편의), 화면 접근과 모든 데이터 처리는 서버가 다시 검사한다. 쿠키 인증 요청은 CSRF 방어 헤더가 없으면 거부된다.

#	계정	요청	기대 결과
1	**soldier1**	**POST** `/api/members` `{"username":"hacker","password":"pass1234","name":"x"}`	**403**
2	**soldier1**	**PATCH** `/api/members/8/role` `{"role":"BOSS"}`	**403** 자기 권한 상승 시도 거부

NFR-04 데이터 수가 증가하더라도 사용자가 필요한 정보를 탐색할 수 있는 방법을 제공해야 한다.

구현: 페이지 단위 조회, 필터·정렬, 통합 검색·자동완성, 검색 조건 저장, MySQL/MongoDB 인덱스.

#	계정	요청	기대 결과
1	**boss**	**GET** `/api/activities?page=3&size=10&sort=-at`	**200**
2	**boss**	**GET** `/api/search/suggest?q=Pa`	**200**

3	**capo1**	**POST** `/api/me/saved-filters` `{"name":"긴급 업무","target":"tasks","query":{"priority":"HIGH,CRITICAL","status":"PLANNED"}}`	**201** url 에 저장된 검색 조건

NFR-05 서비스 전반의 API 응답과 오류 메시지는 가능한 일관된 형태를 가져야 한다.

구현: AR-05 와 동일. Swagger 의 모든 응답 예시도 동일한 SuccessResponse / ErrorResponse 스키마 사용.

#	계정	요청	기대 결과
1	**boss**	**GET** `/api/resources/999`	**404**
2	**boss**	**GET** `/api/evaluations/abc`	**400**

기술 제약조건

CR-01 서버는 Node.js와 Express를 기반으로 구현한다.

구현: Express 4 기반 (package.json). app.js 진입점.

#	계정	요청	기대 결과
1	인증 없음	**GET** `/api/health`	**200**

CR-02 서버 사이드 rendering이 필요한 화면은 EJS 템플릿 엔진을 사용한다.

구현: views/layout.ejs(공통 레이아웃·권한별 메뉴), login.ejs, error.ejs, catalog.ejs. 21개 업무 화면을 서버가 EJS 로 렌더링하며, 권한이 없는 화면은 서버에서 403 으로 차단한다. 화면의 데이터 처리는 동일한 REST API 를 호출한다 (public/js/pages/*.js).

1. 브라우저에서 http://{서버주소}:3000/ 접속 → 로그인 화면 (데모 계정 버튼 클릭 시 자동 입력)
2. capo1 / pass1234 로그인 → 대시보드 (개인 KPI·처리할 일·일정·알림)
3. 좌측 "업무" → 업무 2번 → "배정" → 시간 충돌 없는 후보만 추천됨 (BR-02, BR-03)
4. 좌측 "Crew" → Brooklyn Crew → 구성원·공지·가입 요청·운영 현황 탭
5. soldier1 로 로그인 후 주소창에 /admin/accounts 직접 입력 → "접근 권한이 없습니다" (서버 403, BR-09)
6. boss 로 로그인 → Crew → Brooklyn Crew → 구성원 편성 → Frankie Costa(Harbor) 정식 편성 → BR-01 위반 메시지 표시

CR-03 SQL 및 Mongo DB를 모두 사용한다.

구현: mysql2 + mongodb 드라이버 (db/mysql.js, db/mongo.js). DR-05 참고.

#	계정	요청	기대 결과
1	**admin**	**GET** `/api/admin/system/info`	**200**

CR-04 개발된 모든 API는 Swagger로 문서화한다.

구현: API 정의로부터 OpenAPI 3.0 문서를 자동 생성 (lib/api.js), swagger-ui-express 로 /api-docs 제공. 각 API 에 목적·Parameter·Body·응답 예시·오류 예시 포함.

1. 브라우저에서 http://{서버주소}:3000/api-docs 접속
2. POST /api/auth/login 실행 → token 복사 → Authorize 에 입력
3. 임의의 API → Try it out → Execute

CR-05 프로젝트 코드는 기능별 책임을 고려하여 적절히 분리한다.

구현: config / db(연결·스키마·시드) / lib(API 레지스트리·검증·인증·보안·오류·기록) / services(업무 규칙·주기 작업) / routes(기능별 API 26개 모듈 + 화면 라우터) / views(EJS) / public(화면 스크립트·스타일) 로 분리.

CR-06 제출된 프로젝트는 지정된 Ubuntu 환경에서 'node app.js' 명령 하나로 실행될 수 있어야 한다.

구현: node_modules 포함 제출. 최초 실행 시 DB·테이블·인덱스 자동 생성 및 데모 데이터 자동 입력.

1. cd ~/midterm
2. node app.js
3. 콘솔에 "등록된 REST API : N개" 출력 확인

11. 전체 API 명세 (221개)

모든 API 는 Swagger UI(/api-docs)에서 동일한 내용으로 확인·실행할 수 있다. 성공 응답은 `{ success, message, data, meta? }` 로 감싸진다.

01. Auth

API-001 POST /api/auth/register

회원 가입 신청 (Associate, 가입대기 상태로 등록)

요구사항 FR-01, FR-04 · **권한** 공개 · **성공** 201 · **구현 위치** `routes/auth.js:62`

Body 필드	타입	필수	설명 / 제약
`username`	string	Y	로그인 ID · 최대 30자
`password`	string	Y	최대 64자
`name`	string	Y	최대 50자
`alias`	string		최대 50자
`email`	email		
`phone`	string		최대 30자

Request 예시

```
{
  "username": "newcomer1",
  "password": "pass1234",
  "name": "홍길동",
  "alias": "Gil",
  "email": "newcomer1@family.local",
  "phone": "010-1234-5678"
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": 20,
    "username": "newcomer1",
    "role": "ASSOCIATE",
    "status": "PENDING"
  }
}
```

오류 응답: 400, 409 — 409: 이미 존재하는 값입니다(중복).

API-002 POST /api/auth/login

로그인 (JWT 토큰 발급)

로그인 성공 시 JWT 토큰을 발급한다. 연속 실패 횟수가 설정값 이상이면 계정이 자동 잠금된다. 데모 계정 비밀번호는 모두 pass1234.

요구사항 NFR-03, FR-23 · **권한** 공개 · **성공** 200 · **구현 위치** `routes/auth.js:87`

Body 필드	타입	필수	설명 / 제약
`username`	string	Y	로그인 ID · 최대 30자
`password`	string	Y	최대 64자

Request 예시

Response 예시 (200)

```
{
  "username": "boss",
  "password": "pass1234"
}
```

```
{
  "success": true,
  "data": {
    "token": "eyJhbGciOi...",
    "tokenType": "Bearer",
    "expiresAt": "2026-10-
19T02:00:00.000Z",
    "member": {
      "id": 2,
      "username": "boss",
      "name": "Vincenzo Moretti",
      "role": "BOSS",
      "status": "ACTIVE"
    }
  }
}
```

오류 응답: 400, 401, 429 — 429: 로그인 실패가 너무 많습니다. 10분 후 다시 시도하세요.

API-003 POST /api/auth/logout

로그아웃 (현재 토큰 폐기)

요구사항 NFR-03 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/auth.js:100`

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "revoked": true
  }
}
```

오류 응답: 401

API-004 POST /api/auth/refresh

토큰 재발급 (기존 토큰은 폐기)

요구사항 NFR-03 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/auth.js:111`

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "token": "eyJhbGciOi...",
    "tokenType": "Bearer",
    "expiresAt": "2026-10-19T02:00:00.000Z"
  }
}
```

오류 응답: 401

API-005 PUT `/api/auth/password`

내 비밀번호 변경 (다른 세션은 모두 만료)

요구사항 FR-01, FR-23 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/auth.js:120`

Body 필드	타입	필수	설명 / 제약
`currentPassword`	string	Y	최대 64자
`newPassword`	string	Y	최대 64자

Request 예시

```
{
  "currentPassword": "pass1234",
  "newPassword": "newpass5678"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "changed": true,
    "token": "eyJhbGciOi...",
    "tokenType": "Bearer",
    "expiresAt": "2026-10-
19T02:00:00.000Z"
  }
}
```

오류 응답: 400, 401 — 401: 현재 비밀번호가 올바르지 않습니다.

API-006 GET `/api/auth/login-history`

내 로그인/로그아웃 이력 조회

요구사항 FR-02, FR-20 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/auth.js:136`

위치	이름	타입	필수	설명 / 제약
query	`result`	string		결과 필터 · 값: SUCCESS | FAIL
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: at (내림차순 '-', 기본 -at)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": "6710...",
      "event": "LOGIN",
      "result": "SUCCESS",
      "ip": "::1",
      "at": "2026-10-18T09:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401

API-007 GET `/api/auth/permissions`

내 역할, 권한 목록, 관리 범위(Crew) 조회

요구사항 FR-02, NFR-03 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/auth.js:153`

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "role": "CAPO",
    "level": 3,
    "permissions": [
      "task.create",
      "task.assign"
    ],
    "managedCrewIds": [
      1
    ],
    "scope": "CREWS"
  }
}
```


오류 응답: 401

02. Me (내 정보)

API-008 GET `/api/me`

내 프로필 조회 (현재 소속, 역할, 상태, 활동 요약)

요구사항 FR-02 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:37`

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 8,
    "username": "soldier1",
    "name": "Paulie Conti",
    "role": "SOLDIER",
    "roleName": "Soldier",
    "level": 2,
    "status": "ACTIVE",
    "statusLabel": "활동중",
    "crews": [
      {
        "crewId": 1,
        "crewName": "Brooklyn Crew",
        "membershipType": "FORMAL",
        "capoName": "Tony Russo"
      }
    ],
    "activity": {
      "openAssignments": 1,
      "completedTasks": 6,
      "activeAllocations": 0,
      "reputation": 10
    }
  }
}
```


오류 응답: 401

API-009 PATCH /api/me

내 연락처/별칭 수정 (역할·상태·소속은 변경 불가)

요구사항 FR-01, FR-02 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:60`

Body 필드	타입	필수	설명 / 제약
`alias`	string		최대 50자, null 허용
`email`	email		null 허용
`phone`	string		최대 30자, null 허용

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시


```
{
  "alias": "Paulie Walnuts",
  "email": "paulie@family.local",
  "phone": "010-2222-3333"
}
```


Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": 8,
    "alias": "Paulie Walnuts",
    "email": "paulie@family.local",
    "phone": "010-2222-3333"
  }
}
```

오류 응답: 400, 401

API-010 GET `/api/me/tasks`

나에게 배정된 업무 목록 (확인 필요 사항 포함)

배정된 업무와 함께 수락 필요 여부, 미완료 체크리스트 수, 배정된 자원 등 "내가 확인해야 할 정보"를 반환한다.

요구사항 FR-09, FR-08 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:76`

위치	이름	타입	필수	설명 / 제약
query	`view`	string		open=진행 예정/중, done=완료/취소, all=전체 · 값: open | done | all, 기본 "open"
query	`from`	datetime		시작일시 이후
query	`to`	datetime		종료일시 이전
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: startAt, priority, assignedAt (내림차순 '-', 기본 startAt)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "assignmentId": 1,
      "assignmentStatus": "ACCEPTED",
      "taskRole": "현장 책임",
      "instructions": "집결 30분 전 도착",
      "task": {
        "id": 1,
        "title": "물류 창고 정기 점검",
        "status": "PLANNED",
        "priority": "HIGH",
        "startAt": "2026-10-19T09:00:00.000Z",
        "endAt": "2026-10-19T12:00:00.000Z",
        "location": "Brooklyn 창고 3동",
        "crewName": "Brooklyn Crew"
      },
      "actionRequired": [
        "체크리스트 미완료 2건"
      ],
      "resources": [
        {
          "code": "VH-001",
          "name": "검정 세단"
        }
      ]
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401

API-011 GET `/api/me/tasks/:taskId`

내 업무 브리핑 (내 역할/지시사항, 동료, 자원, 체크리스트, 최근 코멘트)

요구사항 FR-09, FR-08, FR-11 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:126`

위치	이름	타입	필수	설명 / 제약
path	`taskId`	integer	Y	예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "task": {
      "id": 1,
      "title": "물류 창고 정기 점검",
      "status": "PLANNED"
    },
    "myAssignment": {
      "id": 1,
      "taskRole": "현장 책임",
      "instructions": "집결 30분 전 도착",
      "status": "ACCEPTED"
    },
    "teammates": [
      {
        "memberId": 9,
        "name": "Joey Marino",
        "taskRole": "경비",
        "status": "ASSIGNED"
      }
    ],
    "resources": [
      {
        "resourceId": 1,
        "code": "VH-001",
        "name": "검정 세단",
        "status": "RESERVED"
      }
    ],
    "checklist": [
      {
        "id": "1",
        "text": "출입구 잠금 상태 확인",
        "done": false
      }
    ],
    "recentComments": [
      {
        "author": "capo1",
        "body": "점검 체크리스트 반드시 확인"
      }
    ],
    "myEvaluation": null
  }
}
```


오류 응답: 400, 401, 403, 404

API-012 POST /api/me/assignments/:assignmentId/accept

배정된 업무 수락

요구사항 FR-09, FR-08 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:160`

위치	이름	타입	필수	설명 / 제약

path	`assignmentId`	integer	Y	예: 2

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "assignmentId": 2,
    "status": "ACCEPTED"
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-013 POST /api/me/assignments/:assignmentId/decline

배정된 업무 거절 (사유 필수, Crew 책임자에게 알림)

요구사항 FR-09, FR-08 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:175`

위치	이름	타입	필수	설명 / 제약
path	`assignmentId`	integer	Y	예: 2

Body 필드	타입	필수	설명 / 제약
`reason`	string	Y	최대 300자

Request 예시

```
{
  "reason": "같은 시간 가족 행사"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "assignmentId": 2,
    "status": "DECLINED"
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-014 GET /api/me/dashboard

내 대시보드 (7일 내 일정, 미읽은 알림, 대기 중 요청, 사용 중 자원)

요구사항 FR-02, FR-09, FR-14 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:191`

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "upcoming": [
      {
        "taskId": 1,
        "title": "물류 창고 정기 점검",
        "startAt": "2026-10-19T09:00:00.000Z"
      }
    ],
    "unreadNotifications": 2,
    "pendingRequests": {
      "resource": 1,
      "join": 0
    },
    "allocations": []
  }
}
```


오류 응답: 401

API-015 GET `/api/me/inbox`

내 처리 대기함 (내가 응답·승인해야 할 항목 집계)

배정 응답, 자원 사용 요청 승인, Crew 가입 요청, 승급 요청 심사, 평가 대기 등 역할과 관리 범위에 따라 현재 사용자가 처리해야 할 항목 수를 반환한다.

요구사항 FR-09, FR-06, FR-17, FR-18 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:211`

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "total": 4,
    "items": [
      {
        "type": "ASSIGNMENT",
        "label": "배정 응답 대기",
        "count": 1,
        "link": "/tasks?mine=1"
      },
      {
        "type": "RESOURCE_REQUEST",
        "label": "자원 사용 요청 승인",
        "count": 1,
        "link": "/approvals"
      }
    ]
  }
}
```

오류 응답: 401

API-016 GET `/api/me/notifications`

내 알림 목록

요구사항 FR-09, NFR-01 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:262`

위치	이름	타입	필수	설명 / 제약
query	`unread`	boolean		true 이면 읽지 않은 알림만
query	`type`	string		값: TASK_ASSIGNED | TASK_UNASSIGNED | TASK_STATUS | ASSIGNMENT_DECLINED | ROLE_CHANGED | STATUS_CHANGED | CREW_CHANGED | EVALUATION | RESOURCE | REQUEST_REVIEWED | ANNOUNCEMENT | MEETING | DISCIPLINE | BROADCAST
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: createdAt (내림차순 '-', 기본 -createdAt)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": "6711aa...",
      "type": "TASK_ASSIGNED",
      "title": "업무 배정",
      "message": "물류 창고 정기 점검 업무에 배정되었습니다.",
      "link": "/api/me/tasks/1",
      "read": false,
      "createdAt": "2026-10-18T09:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

오류 응답: 400, 401

API-017 PATCH /api/me/notifications/read-all

내 알림 모두 읽음 처리

요구사항 FR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:277`

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "updated": 3
  }
}
```

오류 응답: 401

API-018 PATCH /api/me/notifications/:id/read

알림 1건 읽음 처리

요구사항 FR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:286`

위치	이름	타입	필수	설명 / 제약
path	`id`	objectId	Y	예: "6711aa000000000000000001"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": "6711aa000000000000000001",
    "read": true
  }
}
```

오류 응답: 400, 401, 404

API-019 DELETE /api/me/notifications/:id

알림 1건 삭제

요구사항 FR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:298`

위치	이름	타입	필수	설명 / 제약
path	`id`	objectId	Y	예: "6711aa000000000000000001"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": "6711aa000000000000000001",
    "deleted": true
  }
}
```

오류 응답: 400, 401, 404

API-020 GET `/api/me/notification-settings`

내 알림 수신 설정 조회

요구사항 FR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:310`

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "mutedTypes": [
      "ANNOUNCEMENT"
    ],
    "availableTypes": [
      "TASK_ASSIGNED",
      "TASK_UNASSIGNED",
      "TASK_STATUS",
      "ASSIGNMENT_DECLINED",
      "ROLE_CHANGED",
      "STATUS_CHANGED",
      "CREW_CHANGED",
      "EVALUATION",
      "RESOURCE",
      "REQUEST_REVIEWED",
      "ANNOUNCEMENT",
      "MEETING",
      "DISCIPLINE",
      "BROADCAST"
    ]
  }
}
```

오류 응답: 401

API-021 PUT `/api/me/notification-settings`

내 알림 수신 설정 변경 (수신 거부할 알림 유형 지정)

요구사항 FR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:319`

Body 필드	타입	필수	설명 / 제약

`mutedTypes`	array<string>	Y	값: TASK_ASSIGNED | TASK_UNASSIGNED | TASK_STATUS | ASSIGNMENT_DECLINED | ROLE_CHANGED | STATUS_CHANGED | CREW_CHANGED | EVALUATION | RESOURCE | REQUEST_REVIEWED | ANNOUNCEMENT | MEETING | DISCIPLINE | BROADCAST

Request 예시

```
{
  "mutedTypes": [
    "ANNOUNCEMENT"
  ]
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "mutedTypes": [
      "ANNOUNCEMENT"
    ]
  }
}
```

오류 응답: 400, 401

API-022 GET `/api/me/saved-filters`

내가 저장한 검색 조건 목록

요구사항 FR-21, NFR-04 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:330`

위치	이름	타입	필수	설명 / 제약
query	`target`	string		값: tasks | members | resources | crews

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": "6712...",
    "name": "이번주 HIGH 업무",
    "target": "tasks",
    "query": {
      "priority": "HIGH",
      "status": "PLANNED"
    },
    "createdAt": "2026-10-18T09:00:00.000Z"
  }
}
```

오류 응답: 400, 401

API-023 POST `/api/me/saved-filters`

검색 조건 저장 (자주 쓰는 업무/구성원/자원 검색 조건)

요구사항 FR-21, NFR-04 · **권한** 로그인 사용자 · **성공** 201 · **구현 위치** `routes/me.js:341`

Body 필드	타입	필수	설명 / 제약
`name`	string	Y	최대 50자
`target`	string	Y	값: tasks | members | resources | crews
`query`	object	Y	목록 API 에 그대로 전달할 query parameter

Request 예시

```
{
  "name": "이번주 HIGH 업무",
  "target": "tasks",
  "query": {
    "priority": "HIGH",
    "status": "PLANNED"
  }
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": "6712...",
    "name": "이번주 HIGH 업무",
    "target": "tasks",
    "query": {
      "priority": "HIGH",
      "status": "PLANNED"
    },
    "url": "/api/tasks?
priority=HIGH&status=PLANNED"
  }
}
```

오류 응답: 400, 401

API-024 DELETE `/api/me/saved-filters/:id`

저장한 검색 조건 삭제

요구사항 FR-21 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/me.js:359`

위치	이름	타입	필수	설명 / 제약
path	`id`	objectId	Y	예: "6712aa000000000000000001"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": "6712aa000000000000000001",
    "deleted": true
  }
}
```

오류 응답: 400, 401, 404

03. Roles & Permissions

API-025 GET `/api/roles`

역할 목록 (권한 등급 및 역할별 인원 수)

요구사항 FR-23, DR-01 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/roles.js:32`

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 3,
    "code": "CAPO",
    "name": "Capo",
    "level": 3,
    "description": "Crew 책임자",
    "memberCount": 3,
    "permissionCount": 16
  }
}
```


오류 응답: 401

API-026 GET `/api/roles/:code`

역할 상세 (상태별 인원 분포 포함)

요구사항 FR-23, DR-01 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/roles.js:42`

위치	이름	타입	필수	설명 / 제약
path	`code`	string	Y	값: ASSOCIATE | SOLDIER | CAPO | UNDERBOSS | BOSS | ADMINISTRATOR · 예: "CAPO"

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": 3,
    "code": "CAPO",
    "name": "Capo",
    "level": 3,
    "membersByStatus": {
      "ACTIVE": 3
    }
  }
}
```


오류 응답: 400, 401, 404

API-027 GET `/api/roles/:code/permissions`

역할의 권한 매트릭스 (전체 권한 중 부여 여부)

요구사항 FR-23, BR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/roles.js:54`

위치	이름	타입	필수	설명 / 제약
path	`code`	string	Y	값: ASSOCIATE | SOLDIER | CAPO | UNDERBOSS | BOSS | ADMINISTRATOR · 예: "CAPO"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "role": "CAPO",
    "permissions": [
      {
        "code": "task.create",
        "category": "TASK",
        "description": "업무 등록/복제/삭제",
        "granted": true
      }
    ]
  }
}
```


오류 응답: 400, 401, 404

API-028 PUT `/api/roles/:code/permissions`

역할의 권한 집합 전체 교체

요구사항 FR-23, FR-20 · **권한** 로그인 + 권한 permission.manage · **성공** 200 · **구현 위치** `routes/roles.js:67`

위치	이름	타입	필수	설명 / 제약
path	`code`	string	Y	값: ASSOCIATE | SOLDIER | CAPO | UNDERBOSS | BOSS | ADMINISTRATOR · 예: "CAPO"

Body 필드	타입	필수	설명 / 제약
`permissions`	array<string>	Y	

Request 예시


```
{
  "permissions": [
    "task.create",
    "task.update",
    "task.assign"
  ]
}
```


Response 예시 (200)


```
{
  "success": true,
  "data": {
    "role": "CAPO",
    "added": [
      "task.assign"
    ],
    "removed": [
      "meeting.manage"
    ],
    "permissions": [
      "task.create",
      "task.update",
      "task.assign"
    ]
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-029 POST `/api/roles/:code/permissions`

역할에 권한 1개 부여

요구사항 FR-23, FR-20 · **권한** 로그인 + 권한 permission.manage · **성공** 201 · **구현 위치** `routes/roles.js:91`

위치	이름	타입	필수	설명 / 제약
path	`code`	string	Y	값: ASSOCIATE | SOLDIER | CAPO | UNDERBOSS | BOSS | ADMINISTRATOR · 예: "CAPO"

Body 필드	타입	필수	설명 / 제약
`permission`	string	Y	

Request 예시

```
{
  "permission": "stats.read"
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "role": "CAPO",
    "granted": "stats.read"
  }
}
```

오류 응답: 400, 401, 403, 404, 409

API-030 DELETE `/api/roles/:code/permissions/:permission`

역할에서 권한 1개 회수

요구사항 FR-23, FR-20 · **권한** 로그인 + 권한 permission.manage · **성공** 200 · **구현 위치** `routes/roles.js:108`

위치	이름	타입	필수	설명 / 제약
path	`code`	string	Y	값: ASSOCIATE | SOLDIER | CAPO | UNDERBOSS | BOSS | ADMINISTRATOR · 예: "CAPO"
path	`permission`	string	Y	예: "stats.read"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "role": "CAPO",
    "revoked": "stats.read"
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-031 GET `/api/permissions`

전체 권한 코드 목록 (각 권한을 보유한 역할 포함)

요구사항 FR-23, BR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/roles.js:125`

위치	이름	타입	필수	설명 / 제약
query	`category`	string		값: MEMBER | CREW | TASK | RESOURCE | EVALUATION | ADMIN

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "code": "task.create",
    "category": "TASK",
    "description": "업무 등록/복제/삭제",
    "roles": [
      "CAPO",
      "UNDERBOSS",
      "BOSS"
    ]
  }
}
```

오류 응답: 400, 401

04. Members

API-032 GET `/api/members`

구성원 검색/목록 (권한 범위 내, 필터·정렬·페이지)

Boss/Admin: 전체, Underboss: 감독 Crew + 무소속, Capo: 담당 Crew, Soldier/Associate: 자신이 속한 Crew. 관리 범위 밖 구성원은 연락처가 제외된다(limited=true).

요구사항 FR-21, FR-05, NFR-04, BR-09 · **권한** 로그인 사용자 (역할별 조회 범위 적용) · **성공** 200 · **구현 위치** `routes/members.js:80`

위치	이름	타입	필수	설명 / 제약
query	`q`	string		이름/아이디/별칭 검색어 · 예: "Paulie"
query	`role`	string		값: ASSOCIATE | SOLDIER | CAPO | UNDERBOSS | BOSS | ADMINISTRATOR
query	`status`	array<string>		쉼표 구분 다중 선택 (예: ACTIVE,STANDBY) · 값: PENDING | ACTIVE | STANDBY | SUSPENDED | WITHDRAWN
query	`crewId`	integer		소속 Crew
query	`skill`	string		보유 기술 · 예: "driving"

query	`unaffiliated`	boolean		true 이면 정식 소속이 없는 구성원만
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: id, name, joinedAt, level, status (내림차순 '-', 기본 id)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": 8,
      "username": "soldier1",
      "name": "Paulie Conti",
      "alias": "Paulie",
      "role": "SOLDIER",
      "roleName": "Soldier",
      "level": 2,
      "status": "ACTIVE",
      "crewId": 1,
      "crewName": "Brooklyn Crew",
      "email": "soldier1@family.local",
      "phone": "010-1000-1007",
      "joinedAt": "2025-11-01T00:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

오류 응답: 400, 401

API-033 GET `/api/members/export`

구성원 목록 CSV 내보내기 (검색 조건 동일 적용)

요구사항 FR-21, FR-22 · **권한** 로그인 + 권한 member.read · **성공** 200 text/csv · **구현 위치** `routes/members.js:100`

위치	이름	타입	필수	설명 / 제약
query	`q`	string		이름/아이디/별칭 검색어 · 예: "Paulie"
query	`role`	string		값: ASSOCIATE | SOLDIER | CAPO | UNDERBOSS | BOSS | ADMINISTRATOR

query	`status`	array<string>		쉼표 구분 다중 선택 (예: ACTIVE,STANDBY) · 값: PENDING | ACTIVE | STANDBY | SUSPENDED | WITHDRAWN
query	`crewId`	integer		소속 Crew
query	`skill`	string		보유 기술 · 예: "driving"
query	`unaffiliated`	boolean		true 이면 정식 소속이 없는 구성원만

Response 예시 (200)

```
id,username,name,role,status,crewName
8,soldier1,Paulie Conti,SOLDIER,ACTIVE,Brooklyn Crew
```

오류 응답: 400, 401, 403

API-034 GET `/api/members/:id`

구성원 상세 조회 (소속 Crew, 기술 포함)

요구사항 FR-02, FR-05, BR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/members.js:118`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 8,
    "username": "soldier1",
    "name": "Paulie Conti",
    "alias": "Paulie",
    "role": "SOLDIER",
    "roleName": "Soldier",
    "level": 2,
    "status": "ACTIVE",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "email": "soldier1@family.local",
    "phone": "010-1000-1007",
    "joinedAt": "2025-11-01T00:00:00.000Z",
    "crews": [
      {
        "crewId": 1,
        "crewName": "Brooklyn Crew",
        "membershipType": "FORMAL"
      }
    ],
    "skills": [
      {
        "skill": "driving",
        "level": 4
      }
    ]
  }
}
```


오류 응답: 400, 401, 403, 404

API-035 POST `/api/members`

구성원 등록 (역할/상태/소속 Crew 지정 가능)

요구사항 FR-01, FR-04, BR-01, BR-04 · **권한** 로그인 + 권한 member.create · **성공** 201 · **구현 위치** `routes/members.js:137`

Body 필드	타입	필수	설명 / 제약
`username`	string	Y	최대 30자
`password`	string	Y	최대 64자
`name`	string	Y	최대 50자
`alias`	string		최대 50자
`email`	email		
`phone`	string		최대 30자
`birthDate`	date		
`role`	string		값: ASSOCIATE | SOLDIER | CAPO | UNDERBOSS | BOSS | ADMINISTRATOR, 기본 "ASSOCIATE"

`status`	string		값: PENDING | ACTIVE | STANDBY, 기본 "ACTIVE"
`crewId`	integer		정식 소속 Crew (선택)
`notes`	string		최대 1000자

Request 예시

```
{
  "username": "soldier7",
  "password": "pass1234",
  "name": "Gino Vitale",
  "alias": "Gino",
  "email": "soldier7@family.local",
  "phone": "010-3333-4444",
  "birthDate": "1995-05-05",
  "role": "SOLDIER",
  "status": "ACTIVE",
  "crewId": 1
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": 20,
    "username": "soldier7",
    "name": "Gino Vitale",
    "alias": "Paulie",
    "role": "SOLDIER",
    "roleName": "Soldier",
    "level": 2,
    "status": "ACTIVE",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "email": "soldier1@family.local",
    "phone": "010-1000-1007",
    "joinedAt": "2025-11-01T00:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403, 409 — 409: 이미 존재하는 값입니다(중복).

API-036 PATCH `/api/members/:id`

구성원 기본 정보 수정 (이름/별칭/연락처/생년월일/메모)

요구사항 FR-01, BR-04 · **권한** 로그인 + 권한 member.update · **성공** 200 · **구현 위치** `routes/members.js:179`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8

Body 필드	타입	필수	설명 / 제약
`name`	string		최대 50자
`alias`	string		최대 50자, null 허용
`email`	email		null 허용
`phone`	string		최대 30자, null 허용
`birthDate`	date		null 허용
`notes`	string		최대 1000자, null 허용

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시

Response 예시 (200)

```
{
  "name": "Paulie Conti",
  "alias": "Paulie",
  "phone": "010-1111-2222",
  "notes": "운전 실력 우수"
}
```

```
{
  "success": true,
  "data": {
    "id": 8,
    "username": "soldier1",
    "name": "Paulie Conti",
    "alias": "Paulie",
    "role": "SOLDIER",
    "roleName": "Soldier",
    "level": 2,
    "status": "ACTIVE",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "email": "soldier1@family.local",
    "phone": "010-1000-1007",
    "joinedAt": "2025-11-01T00:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403, 404

API-037 DELETE `/api/members/:id`

가입대기 구성원 삭제 (활동 이력이 없는 PENDING 계정만)

활동 이력이 있는 구성원은 삭제할 수 없으며 상태를 WITHDRAWN 으로 변경해야 한다(이력 보존).

요구사항 FR-01, FR-04, DR-01 · **권한** 로그인 + 권한 member.delete · **성공** 200 · **구현 위치** `routes/members.js:203`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 19,
    "deleted": true
  }
}
```

오류 응답: 400, 401, 403, 404, 422 — 422: PENDING 상태이고 활동 이력이 없는 구성원만 삭제할 수 있습니다.

API-038 PATCH `/api/members/:id/role`

구성원 역할 변경 (자신보다 높은 권한자 변경 불가)

BR-04: 대상의 현재 역할과 새 역할이 모두 요청자보다 낮아야 한다(Administrator 예외). Capo 는 담당 Crew 의 Associate↔Soldier 만 변경 가능. 변경 이력은 member_history 에 기록된다.

요구사항 FR-03, BR-04, BR-05, BR-10 · **권한** 로그인 + 권한 member.role · **성공** 200 · **구현 위치** `routes/members.js:225`

위치	이름	타입	필수	설명 / 제약

path	`id`	integer	Y	구성원 ID · 예: 8

Body 필드	타입	필수	설명 / 제약
`role`	string	Y	값: ASSOCIATE | SOLDIER | CAPO | UNDERBOSS | BOSS | ADMINISTRATOR
`reason`	string		최대 300자

Request 예시

```
{
  "role": "SOLDIER",
  "reason": "6개월 성실 활동"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "memberId": 14,
    "from": "ASSOCIATE",
    "to": "SOLDIER",
    "vacatedPositions": []
  }
}
```

오류 응답: 400, 401, 403, 404, 422 — 403 BR-04: 자신보다 높거나 같은 권한을 가진 사용자의 역할은 변경할 수 없습니다.

API-039 PATCH `/api/members/:id/status`

구성원 상태 변경 (활동중/대기/정지/탈퇴)

허용 전이: PENDING→ACTIVE|WITHDRAWN, ACTIVE→STANDBY|SUSPENDED|WITHDRAWN, STANDBY→ACTIVE|SUSPENDED|WITHDRAWN, SUSPENDED→ACTIVE|STANDBY|WITHDRAWN. 탈퇴 시 Crew 소속 종료·진행 전 업무 배정 해제·자원 예약 취소·세션 만료가 함께 처리된다. Capo 는 탈퇴 처리 불가.

요구사항 FR-03, FR-04, BR-04, BR-10, FR-20 · **권한** 로그인 + 권한 member.status · **성공** 200 · **구현 위치** `routes/members.js:244`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8

Body 필드	타입	필수	설명 / 제약
`status`	string	Y	값: PENDING | ACTIVE | STANDBY | SUSPENDED | WITHDRAWN
`reason`	string		최대 300자

Request 예시

```
{
  "status": "STANDBY",
  "reason": "장기 휴가"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "memberId": 9,
    "from": "ACTIVE",
    "to": "STANDBY",
    "effects": {}
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-040 GET `/api/members/:id/status-transitions`

구성원의 현재 상태에서 요청자가 수행 가능한 상태 전이 목록

요구사항 FR-04 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/members.js:266`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "memberId": 9,
    "current": "ACTIVE",
    "allowed": [
      "STANDBY",
      "SUSPENDED",
      "WITHDRAWN"
    ],
    "canChange": true
  }
}
```

오류 응답: 400, 401, 403, 404

API-041 GET `/api/members/:id/crew-history`

구성원의 Crew 소속 이력 (가입/이동/탈퇴)

요구사항 FR-19, BR-10, DR-01 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/members.js:282`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "membershipType": "FORMAL",
    "joinedAt": "2026-01-01T00:00:00.000Z",
    "leftAt": null,
    "leftReason": null
  }
}
```

오류 응답: 400, 401, 403, 404

API-042 GET `/api/members/:id/assignments`

구성원의 업무 참여 이력 (참여 기록·기여도 포함)

요구사항 FR-11, FR-19, DR-02 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/members.js:298`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8
query	`taskStatus`	string		값: PLANNED | IN_PROGRESS | ON_HOLD | COMPLETED | CANCELLED
query	`from`	datetime		
query	`to`	datetime		
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: startAt, assignedAt (내림차순 '-', 기본 -startAt)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "assignmentId": 5,
      "taskId": 4,
      "title": "월간 장부 정리",
      "taskStatus": "COMPLETED",
      "resultOutcome": "SUCCESS",
      "taskRole": "장부 검토",
      "status": "ACCEPTED",
      "attendance": "PRESENT",
      "contribution": 80
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

오류 응답: 400, 401, 403, 404

API-043 GET `/api/members/:id/schedule`

구성원 일정 타임라인 (업무·자원 사용·회의)

요구사항 FR-09, BR-03 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/members.js:324`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8
query	`from`	datetime		기본: 현재 · 예: "2026-10-18T00:00:00Z"

query	`to`	datetime		기본: from + 14일 · 예: "2026-11-01T00:00:00Z"

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "kind": "TASK",
      "refId": 1,
      "title": "물류 창고 정기 점검",
      "startAt": "2026-10-19T09:00:00.000Z",
      "endAt": "2026-10-19T12:00:00.000Z",
      "exclusive": true
    }
  ]
}
```

오류 응답: 400, 401, 403, 404

API-044 GET `/api/members/:id/availability`

특정 시간대 배정 가능 여부 사전 점검 (상태·시간 중복)

요구사항 BR-02, BR-03, FR-08 · **권한** 로그인 + 권한 task.assign · **성공** 200 · **구현 위치** `routes/members.js:354`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8
query	`startAt`	datetime	Y	예: "2026-10-19T10:00:00Z"
query	`endAt`	datetime	Y	예: "2026-10-19T11:00:00Z"
query	`allowConcurrent`	boolean		배정하려는 업무가 동시 수행 가능한 업무인지 · 기본 false

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "memberId": 8,
    "available": false,
    "reasons": [
      "BR-03: 동일 시간대 업무 1건"
    ],
    "status": "ACTIVE",
    "conflicts": [
      {
        "taskId": 1,
        "title": "물류 창고 정기 점검"
      }
    ]
  }
}
```

오류 응답: 400, 401, 403, 404

API-045 GET `/api/members/:id/reputation`

구성원 평판 점수 (평가 평균, 가감점, 출석률)

요구사항 FR-18, FR-19 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/members.js:375`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "memberId": 8,
    "evaluationCount": 1,
    "evaluationAverage": 4.5,
    "ledgerPoints": 10,
    "completedTasks": 6,
    "attendanceRate": 100,
    "score": 100,
    "recentAdjustments": []
  }
}
```

오류 응답: 400, 401, 403, 404

API-046 POST `/api/members/:id/reputation-adjustments`

평판 가감점 부여 (공로/과실)

요구사항 FR-18, BR-10, FR-20 · **권한** 로그인 + 권한 reputation.adjust · **성공** 201 · **구현 위치** `routes/members.js:392`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8

Body 필드	타입	필수	설명 / 제약
`points`	integer	Y	≥-50, ≤50
`reason`	string	Y	최대 300자

Request 예시

```
{
  "points": 5,
  "reason": "위기 상황에서 신속 대응"
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": "6713...",
    "memberId": 8,
    "points": 5,
    "reason": "위기 상황에서 신속 대응",
    "score": 105
  }
}
```


오류 응답: 400, 401, 403, 404

API-047 GET `/api/members/:id/history`

구성원 주요 변경 이력 (역할·소속·상태·평가·징계)

요구사항 BR-10, FR-19, FR-20 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/members.js:416`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8
query	`type`	string		값: ROLE | STATUS | CREW | EVALUATION | REPUTATION | DISCIPLINE
query	`from`	datetime		
query	`to`	datetime		
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: at (내림차순 '-', 기본 -at)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": "6713...",
      "memberId": 16,
      "type": "STATUS",
      "from": "ACTIVE",
      "to": "SUSPENDED",
      "reason": "조직 정보 외부 유출",
      "by": {
        "id": 3,
        "username": "underboss1",
        "role": "UNDERBOSS"
      },
      "at": "2026-10-13T00:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401, 403, 404

API-048 GET `/api/members/:id/skills`

구성원 보유 기술 목록

요구사항 FR-01, FR-08 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/members.js:442`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8

Response 예시 (200)


```
{
  "success": true,
  "data": [
    {
      "skill": "driving",
      "level": 4
    }
  ]
}
```

오류 응답: 400, 401, 403, 404

API-049 PUT `/api/members/:id/skills/:skill`

구성원 기술 등록/수준 변경 (본인 또는 관리자)

요구사항 FR-01 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/members.js:454`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8
path	`skill`	string	Y	최대 50자 · 예: "security"

Body 필드	타입	필수	설명 / 제약
`level`	integer	Y	≥1, ≤5

Request 예시

```
{
  "level": 3
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "memberId": 8,
    "skill": "security",
    "level": 3
  }
}
```

오류 응답: 400, 401, 403, 404

API-050 DELETE `/api/members/:id/skills/:skill`

구성원 기술 삭제

요구사항 FR-01 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/members.js:471`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원 ID · 예: 8
path	`skill`	string	Y	최대 50자 · 예: "security"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "memberId": 8,
    "skill": "security",
    "deleted": true
  }
}
```

오류 응답: 400, 401, 403, 404

05. Promotion Requests

API-051 POST `/api/promotion-requests`

승급 요청 등록

현재보다 높은 역할로의 승급만 요청할 수 있으며, 같은 구성원에 대해 대기 중인 요청이 있으면 중복 등록할 수 없다.

요구사항 FR-03, BR-04 · **권한** 로그인 + 권한 promotion.request · **성공** 201 · **구현 위치** `routes/promotions.js:41`

Body 필드	타입	필수	설명 / 제약
`memberId`	integer	Y	
`targetRole`	string	Y	값: SOLDIER | CAPO | UNDERBOSS
`reason`	string	Y	최대 500자

Request 예시

```
{
  "memberId": 9,
  "targetRole": "CAPO",
  "reason": "경비 업무 리더십 탁월"
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": 1,
    "memberId": 14,
    "memberName": "Danny Lombardi",
    "currentRole": "ASSOCIATE",
    "targetRole": "SOLDIER",
    "requestedBy": 5,
    "requestedByName": "Tony Russo",
    "reason": "6개월간 성실 활동",
    "status": "PENDING",
    "reviewedBy": null,
    "reviewComment": null,
    "createdAt": "2026-10-
18T00:00:00.000Z",
    "reviewedAt": null
  }
}
```

오류 응답: 400, 401, 403, 404, 409, 422

API-052 GET `/api/promotion-requests`

승급 요청 목록 (내가 요청했거나 심사 가능한 요청)

요구사항 FR-03, FR-19 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/promotions.js:68`

위치	이름	타입	필수	설명 / 제약
query	`status`	string		값: PENDING | APPROVED | REJECTED | CANCELLED
query	`memberId`	integer		
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: createdAt, id (내림차순 '-', 기본 -createdAt)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "memberId": 14,
      "memberName": "Danny Lombardi",
      "currentRole": "ASSOCIATE",
      "targetRole": "SOLDIER",
      "requestedBy": 5,
      "requestedByName": "Tony Russo",
      "reason": "6개월간 성실 활동",
      "status": "PENDING",
      "reviewedBy": null,
      "reviewComment": null,
      "createdAt": "2026-10-18T00:00:00.000Z",
      "reviewedAt": null
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401

API-053 GET `/api/promotion-requests/:id`

승급 요청 상세

요구사항 FR-03 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/promotions.js:95`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "memberId": 14,
    "memberName": "Danny Lombardi",
    "currentRole": "ASSOCIATE",
    "targetRole": "SOLDIER",
    "requestedBy": 5,
    "requestedByName": "Tony Russo",
    "reason": "6개월간 성실 활동",
    "status": "PENDING",
    "reviewedBy": null,
    "reviewComment": null,
    "createdAt": "2026-10-18T00:00:00.000Z",
    "reviewedAt": null
  }
}
```


오류 응답: 400, 401, 403, 404

API-054 POST `/api/promotion-requests/:id/approve`

승급 요청 승인 (역할 변경 실행 + 이력 기록)

승인자는 대상 구성원과 목표 역할보다 높은 권한을 가져야 한다(BR-04). 요청자 본인은 승인할 수 없다.

요구사항 FR-03, BR-04, BR-10 · **권한** 로그인 + 권한 promotion.review · **성공** 200 · **구현 위치** `routes/promotions.js:107`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Body 필드	타입	필수	설명 / 제약
`comment`	string		최대 500자

Request 예시


```
{
  "comment": "승인합니다."
}
```


Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "memberId": 14,
    "memberName": "Danny Lombardi",
    "currentRole": "ASSOCIATE",
    "targetRole": "SOLDIER",
    "requestedBy": 5,
    "requestedByName": "Tony Russo",
    "reason": "6개월간 성실 활동",
    "status": "APPROVED",
    "reviewedBy": 3,
    "reviewComment": "승인합니다.",
    "createdAt": "2026-10-
18T00:00:00.000Z",
    "reviewedAt": null
  }
}
```


오류 응답: 400, 401, 403, 404, 422

API-055 POST `/api/promotion-requests/:id/reject`

승급 요청 반려 (사유 필수)

요구사항 FR-03 · **권한** 로그인 + 권한 promotion.review · **성공** 200 · **구현 위치** `routes/promotions.js:130`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Body 필드	타입	필수	설명 / 제약
`comment`	string	Y	최대 500자

Request 예시

```
{
  "comment": "추가 실적 필요"
}
```

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": 1,
    "memberId": 14,
    "memberName": "Danny Lombardi",
    "currentRole": "ASSOCIATE",
    "targetRole": "SOLDIER",
    "requestedBy": 5,
    "requestedByName": "Tony Russo",
    "reason": "6개월간 성실 활동",
    "status": "REJECTED",
    "reviewedBy": null,
    "reviewComment": "추가 실적 필요",
    "createdAt": "2026-10-
18T00:00:00.000Z",
    "reviewedAt": null
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-056 DELETE `/api/promotion-requests/:id`

승급 요청 취소 (요청자 본인, 대기 상태만)

요구사항 FR-03 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/promotions.js:145`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "memberId": 14,
    "memberName": "Danny Lombardi",
    "currentRole": "ASSOCIATE",
    "targetRole": "SOLDIER",
    "requestedBy": 5,
    "requestedByName": "Tony Russo",
    "reason": "6개월간 성실 활동",
    "status": "CANCELLED",
    "reviewedBy": null,
    "reviewComment": null,
    "createdAt": "2026-10-18T00:00:00.000Z",
    "reviewedAt": null
  }
}
```

오류 응답: 400, 401, 403, 404, 422

06. Discipline

API-057 POST `/api/members/:id/disciplinary-actions`

구성원 징계 등록 (SUSPENSION 은 상태를 정지로 변경)

정지 징계 시 구성원 상태가 SUSPENDED 로 변경되며, 이후 새로운 업무 배정이 차단된다(BR-02).

요구사항 FR-03, FR-04, BR-04, BR-10, BR-02 · **권한** 로그인 + 권한 member.discipline · **성공** 201 · **구현 위치** `routes/discipline.js:25`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 9

Body 필드	타입	필수	설명 / 제약
`actionType`	string	Y	값: WARNING | SUSPENSION
`reason`	string	Y	최대 500자

Request 예시

Response 예시 (201)

```
{
  "actionType": "WARNING",
  "reason": "보고 누락"
}
```

```
{
  "success": true,
  "data": {
    "id": 3,
    "memberId": 9,
    "memberName": "Mikey Bruno",
    "actionType": "WARNING",
    "reason": "보고 누락",
    "issuedBy": 3,
    "issuedByName": "Marco Bellini",
    "status": "ACTIVE",
    "revokedBy": null,
    "revokeReason": null,
    "revokedAt": null,
    "createdAt": "2026-10-
13T00:00:00.000Z",
    "statusChange": null
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-058 GET `/api/disciplinary-actions`

징계 목록 (관리 범위 내)

요구사항 FR-19, BR-10 · **권한** 로그인 + 권한 member.read · **성공** 200 · **구현 위치** `routes/discipline.js:56`

위치	이름	타입	필수	설명 / 제약
query	`memberId`	integer		
query	`actionType`	string		값: WARNING | SUSPENSION
query	`status`	string		값: ACTIVE | REVOKED
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: createdAt, id (내림차순 '-', 기본 -createdAt)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "memberId": 16,
      "memberName": "Mikey Bruno",
      "actionType": "SUSPENSION",
      "reason": "조직 정보 외부 유출",
      "issuedBy": 3,
      "issuedByName": "Marco Bellini",
      "status": "ACTIVE",
      "revokedBy": null,
      "revokeReason": null,
      "revokedAt": null,
      "createdAt": "2026-10-13T00:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401, 403

API-059 GET `/api/disciplinary-actions/:id`

징계 상세

요구사항 FR-19 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/discipline.js:83`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "memberId": 16,
    "memberName": "Mikey Bruno",
    "actionType": "SUSPENSION",
    "reason": "조직 정보 외부 유출",
    "issuedBy": 3,
    "issuedByName": "Marco Bellini",
    "status": "ACTIVE",
    "revokedBy": null,
    "revokeReason": null,
    "revokedAt": null,
    "createdAt": "2026-10-13T00:00:00.000Z"
  }
}
```


오류 응답: 400, 401, 403, 404

API-060 POST /api/disciplinary-actions/:id/revoke

징계 해제 (정지 징계 해제 시 다른 정지 징계가 없으면 상태를 ACTIVE 로 복구)

요구사항 FR-04, BR-10, FR-20 · **권한** 로그인 + 권한 member.discipline · **성공** 200 · **구현 위치** `routes/discipline.js:98`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Body 필드	타입	필수	설명 / 제약
`reason`	string	Y	최대 500자

Request 예시


```
{
  "reason": "소명 완료"
}
```


Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "memberId": 16,
    "memberName": "Mikey Bruno",
    "actionType": "SUSPENSION",
    "reason": "조직 정보 외부 유출",
    "issuedBy": 3,
    "issuedByName": "Marco Bellini",
    "status": "REVOKED",
    "revokedBy": null,
    "revokeReason": "소명 완료",
    "revokedAt": null,
    "createdAt": "2026-10-
13T00:00:00.000Z",
    "statusChange": {
      "memberId": 16,
      "from": "SUSPENDED",
      "to": "ACTIVE"
    }
  }
}
```


오류 응답: 400, 401, 403, 404, 422

07. Crews

API-061 GET `/api/crews`

Crew 목록/검색 (지역·상태·책임자 필터)

요구사항 FR-05, FR-21, NFR-04 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/crews.js:58`

위치	이름	타입	필수	설명 / 제약
query	`q`	string		이름 검색 · 예: "Brooklyn"
query	`regionId`	integer		
query	`status`	string		값: ACTIVE | DISSOLVED
query	`capoId`	integer		
query	`underbossId`	integer		
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: id, name, createdAt, memberCount (내림차순 '-', 기본 id)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Brooklyn Crew",
      "description": "브루클린 지역 운영 Crew",
      "status": "ACTIVE",
      "regionId": 1,
      "regionName": "Brooklyn",
      "capoId": 5,
      "capoName": "Tony Russo",
      "underbossId": 3,
      "underbossName": "Marco Bellini",
      "memberCount": 5
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401

API-062 GET `/api/crews/managed`

내가 관리할 수 있는 Crew 목록

요구사항 FR-06, BR-05 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/crews.js:82`

Response 예시 (200)


```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Brooklyn Crew",
      "description": "브루클린 지역 운영 Crew",
      "status": "ACTIVE",
      "regionId": 1,
      "regionName": "Brooklyn",
      "capoId": 5,
      "capoName": "Tony Russo",
      "underbossId": 3,
      "underbossName": "Marco Bellini",
      "memberCount": 5
    }
  ]
}
```

오류 응답: 401

API-063 GET `/api/crews/:id`

Crew 상세 (책임자, 감독자, 인원, 진행 업무 수)

요구사항 FR-05 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/crews.js:93`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Brooklyn Crew",
    "description": "브루클린 지역 운영 Crew",
    "status": "ACTIVE",
    "regionId": 1,
    "regionName": "Brooklyn",
    "capoId": 5,
    "capoName": "Tony Russo",
    "underbossId": 3,
    "underbossName": "Marco Bellini",
    "memberCount": 5,
    "openTasks": 2,
    "affiliateCount": 1
  }
}
```

오류 응답: 400, 401, 404

API-064 POST `/api/crews`

Crew 생성

요구사항 FR-05, FR-20 · **권한** 로그인 + 권한 crew.create · **성공** 201 · **구현 위치** `routes/crews.js:109`

Body 필드	타입	필수	설명 / 제약
`name`	string	Y	최대 50자
`regionId`	integer		
`description`	string		최대 255자
`underbossId`	integer		감독 Underboss (선택)

Request 예시

```
{
  "name": "Queens Crew",
  "regionId": 1,
  "description": "퀸즈 지역 신규 Crew",
  "underbossId": 4
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": 4,
    "name": "Queens Crew",
    "description": "브루클린 지역 운영 Crew",
    "status": "ACTIVE",
    "regionId": 1,
    "regionName": "Brooklyn",
    "capoId": null,
    "capoName": "Tony Russo",
    "underbossId": 3,
    "underbossName": "Marco Bellini",
    "memberCount": 0
  }
}
```


오류 응답: 400, 401, 403, 409 — 409: 이미 존재하는 값입니다(중복).

API-065 PATCH `/api/crews/:id`

Crew 정보 수정 (이름/지역/설명)

요구사항 FR-05, BR-05 · **권한** 로그인 + 권한 crew.update · **성공** 200 · **구현 위치** `routes/crews.js:131`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`name`	string		최대 50자
`regionId`	integer		null 허용
`description`	string		최대 255자, null 허용

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시


```
{
  "name": "Brooklyn Crew",
  "regionId": 1,
  "description": "브루클린 및 인근 지역 담당"
}
```


Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Brooklyn Crew",
    "description": "브루클린 지역 운영 Crew",
    "status": "ACTIVE",
    "regionId": 1,
    "regionName": "Brooklyn",
    "capoId": 5,
    "capoName": "Tony Russo",
    "underbossId": 3,
    "underbossName": "Marco Bellini",
    "memberCount": 5
  }
}
```

오류 응답: 400, 401, 403, 404

API-066 DELETE `/api/crews/:id`

Crew 해산 (정식 구성원이 없고 진행 업무가 없을 때만)

요구사항 FR-05, FR-20 · **권한** 로그인 + 권한 crew.create · **성공** 200 · **구현 위치** `routes/crews.js:152`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1
query	`reason`	string		최대 255자 · 예: "지역 통합"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Brooklyn Crew",
    "description": "브루클린 지역 운영 Crew",
    "status": "DISSOLVED",
    "regionId": 1,
    "regionName": "Brooklyn",
    "capoId": 5,
    "capoName": "Tony Russo",
    "underbossId": 3,
    "underbossName": "Marco Bellini",
    "memberCount": 0
  }
}
```

오류 응답: 400, 401, 403, 404, 422 — 422: 소속 구성원 또는 진행 중 업무가 있어 해산할 수 없습니다.

API-067 POST `/api/crews/:id/restore`

해산된 Crew 복구

요구사항 FR-05, FR-20 · **권한** 로그인 + 권한 crew.create · **성공** 200 · **구현 위치** `routes/crews.js:174`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Brooklyn Crew",
    "description": "브루클린 지역 운영 Crew",
    "status": "ACTIVE",
    "regionId": 1,
    "regionName": "Brooklyn",
    "capoId": 5,
    "capoName": "Tony Russo",
    "underbossId": 3,
    "underbossName": "Marco Bellini",
    "memberCount": 5
  }
}
```


오류 응답: 400, 401, 403, 404, 422

API-068 PUT `/api/crews/:id/capo`

Crew 책임자(Capo) 지정

지정 대상은 CAPO 역할의 활동중 구성원이어야 하며, 다른 Crew 에 정식 소속되어 있다면 이 Crew 로 이동된다(BR-01 유지). 기존 책임자는 해제된다.

요구사항 FR-05, BR-01, BR-10 · **권한** 로그인 + 권한 crew.appoint · **성공** 200 · **구현 위치** `routes/crews.js:188`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`memberId`	integer	Y	

Request 예시


```
{
  "memberId": 5
}
```


Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Brooklyn Crew",
    "description": "브루클린 지역 운영 Crew",
    "status": "ACTIVE",
    "regionId": 1,
    "regionName": "Brooklyn",
    "capoId": 5,
    "capoName": "Tony Russo",
    "underbossId": 3,
    "underbossName": "Marco Bellini",
    "memberCount": 5
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-069 DELETE `/api/crews/:id/capo`

Crew 책임자 해제 (공석 처리)

요구사항 FR-05, BR-10 · **권한** 로그인 + 권한 crew.appoint · **성공** 200 · **구현 위치** `routes/crews.js:221`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Brooklyn Crew",
    "description": "브루클린 지역 운영 Crew",
    "status": "ACTIVE",
    "regionId": 1,
    "regionName": "Brooklyn",
    "capoId": null,
    "capoName": null,
    "underbossId": 3,
    "underbossName": "Marco Bellini",
    "memberCount": 5
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-070 PUT `/api/crews/:id/underboss`

Crew 감독 Underboss 지정/변경

요구사항 FR-05, FR-06 · **권한** 로그인 + 권한 crew.create · **성공** 200 · **구현 위치** `routes/crews.js:238`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`memberId`	integer	Y	null 이면 감독자 해제 · null 허용

Request 예시

```
{
  "memberId": 4
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Brooklyn Crew",
    "description": "브루클린 지역 운영 Crew",
    "status": "ACTIVE",
    "regionId": 1,
    "regionName": "Brooklyn",
    "capoId": 5,
    "capoName": "Tony Russo",
    "underbossId": 4,
    "underbossName": "Luca Ferraro",
    "memberCount": 5
  }
}
```


오류 응답: 400, 401, 403, 404, 422

API-071 GET `/api/crews/:id/members`

Crew 구성원 목록 (정식/준소속 구분)

요구사항 FR-05, FR-02 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/crews.js:256`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1
query	`membershipType`	string		값: FORMAL | AFFILIATE
query	`status`	string		값: PENDING | ACTIVE | STANDBY | SUSPENDED | WITHDRAWN

Response 예시 (200)


```
{
  "success": true,
  "data": [
    {
      "memberId": 8,
      "username": "soldier1",
      "name": "Paulie Conti",
      "role": "SOLDIER",
      "status": "ACTIVE",
      "membershipType": "FORMAL",
      "joinedAt": "2026-04-01T00:00:00.000Z",
      "isCapo": false
    }
  ]
}
```


오류 응답: 400, 401, 403, 404

API-072 POST `/api/crews/:id/members`

Crew 에 구성원 편성 (정식/준소속)

BR-01: 정식(FORMAL) 소속은 1개만 가능. BR-05: 요청자가 관리하는 Crew 에만 편성 가능. 다른 Crew 정식 소속자는

transfer API 로 이동해야 한다.

요구사항 FR-06, BR-01, BR-05, BR-10 · **권한** 로그인 + 권한 crew.members · **성공** 201 · **구현 위치** `routes/crews.js:278`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`memberId`	integer	Y	
`membershipType`	string		값: FORMAL | AFFILIATE, 기본 "FORMAL"
`reason`	string		최대 255자

Request 예시

```
{
  "memberId": 17,
  "membershipType": "FORMAL",
  "reason": "신규 편성"
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "crewId": 1,
    "memberId": 17,
    "membershipType": "FORMAL"
  }
}
```

오류 응답: 400, 401, 403, 404, 409 — 409 BR-01: 구성원은 동시에 둘 이상의 Crew 에 정식 소속될 수 없습니다.

API-073 DELETE `/api/crews/:id/members/:memberId`

Crew 에서 구성원 제외

요구사항 FR-06, BR-05, BR-10 · **권한** 로그인 + 권한 crew.members · **성공** 200 · **구현 위치** `routes/crews.js:301`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1
path	`memberId`	integer	Y	예: 9
query	`reason`	string		최대 255자 · 예: "편성 조정"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "crewId": 1,
    "memberId": 9,
    "removed": true
  }
}
```

오류 응답: 400, 401, 403, 404

API-074 POST `/api/crews/:id/members/:memberId/transfer`

구성원을 다른 Crew 로 이동 (정식 소속 원자적 변경)

출발/도착 Crew 모두 요청자의 관리 범위여야 한다(BR-05). 소속 종료와 신규 소속이 하나의 트랜잭션으로 처리되어 BR-01 이 항상 유지된다.

요구사항 FR-03, FR-06, BR-01, BR-05, BR-10 · **권한** 로그인 + 권한 crew.members · **성공** 200 · **구현 위치** `routes/crews.js:320`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1
path	`memberId`	integer	Y	예: 9

Body 필드	타입	필수	설명 / 제약
`targetCrewId`	integer	Y	
`reason`	string		최대 255자

Request 예시

```
{
  "targetCrewId": 2,
  "reason": "항만 인력 보강"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "memberId": 9,
    "fromCrewId": 1,
    "toCrewId": 2
  }
}
```

오류 응답: 400, 401, 403, 404, 409, 422

API-075 GET `/api/crews/:id/membership-history`

Crew 구성원 편성 변경 이력 (가입/제외/이동)

요구사항 FR-06, BR-10, FR-19 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/crews.js:350`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1
query	`from`	datetime		
query	`to`	datetime		
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: joinedAt, leftAt (내림차순 '-', 기본 -joinedAt)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "memberId": 8,
      "memberName": "Paulie Conti",
      "membershipType": "FORMAL",
      "joinedAt": "2026-04-01T00:00:00.000Z",
      "leftAt": null,
      "leftReason": null,
      "changedBy": "boss"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401, 403, 404

API-076 GET `/api/crews/:id/stats`

Crew 운영 현황 (구성원 상태 분포, 업무 상태 분포, 완료율, 평균 기여도)

요구사항 FR-22, FR-05, FR-11 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/crews.js:375`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1
query	`from`	datetime		업무 집계 시작
query	`to`	datetime		업무 집계 종료

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "crewId": 1,
    "membersByStatus": {
      "ACTIVE": 4,
      "SUSPENDED": 1
    },
    "tasksByStatus": {
      "PLANNED": 2,
      "COMPLETED": 6
    },
    "completionRate": 75,
    "successRate": 66.7,
    "avgContribution": 72.5,
    "resources": 2
  }
}
```

오류 응답: 400, 401, 403, 404

08. Regions

API-077 GET `/api/regions`

지역 목록 (활성 Crew 수 포함)

요구사항 FR-05, FR-21 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/regions.js:19`

위치	이름	타입	필수	설명 / 제약
query	`q`	string		이름 검색

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Brooklyn",
    "description": "동부 항만 인근 상업 지구",
    "crewCount": 1,
    "createdAt": "2026-01-01T00:00:00.000Z"
  }
}
```

오류 응답: 400, 401

API-078 GET `/api/regions/:id`

지역 상세 (소속 Crew 목록 포함)

요구사항 FR-05 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/regions.js:26`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Brooklyn",
    "description": "동부 항만 인근 상업 지구",
    "crewCount": 1,
    "createdAt": "2026-01-01T00:00:00.000Z",
    "crews": [
      {
        "id": 1,
        "name": "Brooklyn Crew",
        "status": "ACTIVE",
        "capoName": "Tony Russo"
      }
    ]
  }
}
```


오류 응답: 400, 401, 404

API-079 POST /api/regions

지역 등록

요구사항 FR-05, FR-20 · **권한** 로그인 + 권한 region.manage · **성공** 201 · **구현 위치** `routes/regions.js:38`

Body 필드	타입	필수	설명 / 제약
`name`	string	Y	최대 50자
`description`	string		최대 255자

Request 예시


```
{
  "name": "Queens",
  "description": "동부 주거 지역"
}
```


Response 예시 (201)


```
{
  "success": true,
  "data": {
    "id": 4,
    "name": "Queens",
    "description": "동부 항만 인근 상업 지구",
    "crewCount": 0,
    "createdAt": "2026-01-
01T00:00:00.000Z"
  }
}
```


오류 응답: 400, 401, 403, 409 — 409: 이미 존재하는 값입니다(중복).

API-080 PATCH /api/regions/:id

지역 정보 수정

요구사항 FR-05 · **권한** 로그인 + 권한 region.manage · **성공** 200 · **구현 위치** `routes/regions.js:53`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Body 필드	타입	필수	설명 / 제약
`name`	string		최대 50자
`description`	string		최대 255자, null 허용

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시

```
{
  "name": "Brooklyn",
  "description": "브루클린 전역"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Brooklyn",
    "description": "동부 항만 인근 상업 지구",
    "crewCount": 1,
    "createdAt": "2026-01-
01T00:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403, 404, 409

API-081 DELETE `/api/regions/:id`

지역 삭제 (연결된 Crew 가 없을 때만)

요구사항 FR-05, DR-01 · **권한** 로그인 + 권한 region.manage · **성공** 200 · **구현 위치** `routes/regions.js:68`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 4

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 4,
    "deleted": true
  }
}
```

오류 응답: 400, 401, 403, 404, 422 — 422: 이 지역에 연결된 Crew 가 있어 삭제할 수 없습니다.

API-082 GET `/api/regions/:id/stats`

지역별 운영 현황 (구성원·업무·자원 집계)

요구사항 FR-22 · **권한** 로그인 + 권한 stats.read · **성공** 200 · **구현 위치** `routes/regions.js:83`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "regionId": 1,
    "name": "Brooklyn",
    "crews": 1,
    "members": 5,
    "tasksByStatus": {
      "PLANNED": 2,
      "COMPLETED": 6
    },
    "resourcesByStatus": {
      "AVAILABLE": 1
    }
  }
}
```


오류 응답: 400, 401, 403, 404

09. Crew Join Requests

API-083 POST /api/crews/:id/join-requests

Crew 가입 요청 (본인 신청, 가입대기 계정도 가능)

요구사항 FR-01, FR-06, BR-01 · **권한** 로그인 사용자 · **성공** 201 · **구현 위치** `routes/joinRequests.js:33`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 3

Body 필드	타입	필수	설명 / 제약
`membershipType`	string		값: FORMAL | AFFILIATE, 기본 "FORMAL"
`message`	string		최대 500자

Request 예시


```
{
  "membershipType": "FORMAL",
  "message": "합류를 희망합니다."
}
```


Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": 1,
    "crewId": 3,
    "crewName": "Uptown Crew",
    "memberId": 19,
    "memberName": "Sonny Amato",
    "memberStatus": "PENDING",
    "membershipType": "FORMAL",
    "message": "Uptown Crew 합류를 희망합니
다.",
    "status": "PENDING",
    "reviewedBy": null,
    "reviewComment": null,
    "createdAt": "2026-10-
18T00:00:00.000Z"
  }
}
```


오류 응답: 400, 401, 404, 409, 422

API-084 GET `/api/join-requests`

가입 요청 목록 (내 요청 또는 관리 Crew 로 들어온 요청)

요구사항 FR-06, FR-21 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/joinRequests.js:60`

위치	이름	타입	필수	설명 / 제약
query	`status`	string		값: PENDING | APPROVED | REJECTED | CANCELLED
query	`crewId`	integer		
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: createdAt, id (내림차순 '-', 기본 -createdAt)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "crewId": 3,
      "crewName": "Uptown Crew",
      "memberId": 19,
      "memberName": "Sonny Amato",
      "memberStatus": "PENDING",
      "membershipType": "FORMAL",
      "message": "Uptown Crew 합류를 희망합니다.",
      "status": "PENDING",
      "reviewedBy": null,
      "reviewComment": null,
      "createdAt": "2026-10-18T00:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401

API-085 GET `/api/join-requests/:id`

가입 요청 상세

요구사항 FR-06 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/joinRequests.js:83`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "crewId": 3,
    "crewName": "Uptown Crew",
    "memberId": 19,
    "memberName": "Sonny Amato",
    "memberStatus": "PENDING",
    "membershipType": "FORMAL",
    "message": "Uptown Crew 합류를 희망합니다.",
    "status": "PENDING",
    "reviewedBy": null,
    "reviewComment": null,
    "createdAt": "2026-10-18T00:00:00.000Z"
  }
}
```


오류 응답: 400, 401, 403, 404

API-086 POST `/api/join-requests/:id/approve`

가입 요청 승인 (Crew 편성 + 가입대기 계정은 활동중으로 전환)

요구사항 FR-01, FR-04, FR-06, BR-01, BR-05, BR-10 · **권한** 로그인 + 권한 crew.members · **성공** 200 · **구현 위치** `routes/joinRequests.js:95`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Body 필드	타입	필수	설명 / 제약
`comment`	string		최대 500자

Request 예시


```
{
  "comment": "환영합니다."
}
```


Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": 1,
    "crewId": 3,
    "crewName": "Uptown Crew",
    "memberId": 19,
    "memberName": "Sonny Amato",
    "memberStatus": "ACTIVE",
    "membershipType": "FORMAL",
    "message": "Uptown Crew 합류를 희망합니다.",
    "status": "APPROVED",
    "reviewedBy": null,
    "reviewComment": "환영합니다.",
    "createdAt": "2026-10-18T00:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403, 404, 409, 422

API-087 POST `/api/join-requests/:id/reject`

가입 요청 반려 (사유 필수)

요구사항 FR-06 · **권한** 로그인 + 권한 crew.members · **성공** 200 · **구현 위치** `routes/joinRequests.js:117`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Body 필드	타입	필수	설명 / 제약
`comment`	string	Y	최대 500자

Request 예시

```
{
  "comment": "현재 인원 충원 계획 없음"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "crewId": 3,
    "crewName": "Uptown Crew",
    "memberId": 19,
    "memberName": "Sonny Amato",
    "memberStatus": "PENDING",
    "membershipType": "FORMAL",
    "message": "Uptown Crew 합류를 희망합니
다.",
    "status": "REJECTED",
    "reviewedBy": null,
    "reviewComment": "현재 인원 충원 계획 없
음",
    "createdAt": "2026-10-
18T00:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-088 DELETE `/api/join-requests/:id`

내 가입 요청 취소 (대기 상태만)

요구사항 FR-06 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/joinRequests.js:131`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "crewId": 3,
    "crewName": "Uptown Crew",
    "memberId": 19,
    "memberName": "Sonny Amato",
    "memberStatus": "PENDING",
    "membershipType": "FORMAL",
    "message": "Uptown Crew 합류를 희망합니다.",
    "status": "CANCELLED",
    "reviewedBy": null,
    "reviewComment": null,
    "createdAt": "2026-10-18T00:00:00.000Z"
  }
}
```


오류 응답: 400, 401, 403, 404, 422

10. Crew Announcements

API-089 GET `/api/crews/:id/announcements`

Crew 공지 목록 (고정 공지 우선)

요구사항 FR-05, FR-02 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/announcements.js:22`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1
query	`q`	string		제목/본문 검색
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: createdAt (내림차순 '-', 기본 -createdAt)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": "6714...",
      "crewId": 1,
      "title": "이번 주 창고 점검 일정",
      "body": "목요일 오전 9시 집결",
      "pinned": true,
      "author": {
        "id": 5,
        "username": "capo1",
        "role": "CAPO"
      },
      "createdAt": "2026-10-18T00:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401, 403, 404

API-090 GET `/api/crews/:id/announcements/:annId`

Crew 공지 상세

요구사항 FR-05 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/announcements.js:46`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1
path	`annId`	objectId	Y	예: "6714aa000000000000000001"

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": "6714...",
    "crewId": 1,
    "title": "이번 주 창고 점검 일정",
    "body": "목요일 오전 9시 집결",
    "pinned": true,
    "author": {
      "id": 5,
      "username": "capo1",
      "role": "CAPO"
    },
    "createdAt": "2026-10-18T00:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403, 404

API-091 POST `/api/crews/:id/announcements`

Crew 공지 등록 (소속 구성원에게 알림)

요구사항 FR-05, FR-06 · **권한** 로그인 + 권한 crew.announce · **성공** 201 · **구현 위치** `routes/announcements.js:57`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`title`	string	Y	최대 100자
`body`	string	Y	최대 5000자
`pinned`	boolean		기본 false

Request 예시

```
{
  "title": "주간 회의 장소 변경",
  "body": "이번 주 회의는 Brooklyn 본부 2층에서 진행합니다.",
  "pinned": false
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": "6714...",
    "crewId": 1,
    "title": "이번 주 창고 점검 일정",
    "body": "목요일 오전 9시 집결",
    "pinned": true,
    "author": {
      "id": 5,
      "username": "capo1",
      "role": "CAPO"
    },
    "createdAt": "2026-10-18T00:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403, 404

API-092 PATCH `/api/crews/:id/announcements/:annId`

Crew 공지 수정 / 고정 설정

요구사항 FR-05 · **권한** 로그인 + 권한 crew.announce · **성공** 200 · **구현 위치** `routes/announcements.js:78`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1
path	`annId`	objectId	Y	예: "6714aa000000000000000001"

Body 필드	타입	필수	설명 / 제약
`title`	string		최대 100자
`body`	string		최대 5000자
`pinned`	boolean		

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시

```
{
  "title": "주간 회의 장소 변경(수정)",
  "pinned": true
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": "6714...",
    "crewId": 1,
    "title": "이번 주 창고 점검 일정",
    "body": "목요일 오전 9시 집결",
    "pinned": true,
    "author": {
      "id": 5,
      "username": "capo1",
      "role": "CAPO"
    },
    "createdAt": "2026-10-
18T00:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403, 404

API-093 DELETE /api/crews/:id/announcements/:annId

Crew 공지 삭제

요구사항 FR-05 · **권한** 로그인 + 권한 crew.announce · **성공** 200 · **구현 위치** `routes/announcements.js:96`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	Crew ID · 예: 1
path	`annId`	objectId	Y	예: "6714aa000000000000000001"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": "6714aa000000000000000001",
    "deleted": true
  }
}
```

오류 응답: 400, 401, 403, 404

11. Tasks

API-094 GET `/api/tasks`

업무 탐색 (기간·Crew·상태·우선순위·담당자·검색어, 정렬, 페이지)

권한 범위 내 업무만 조회된다. Boss/Admin: 전체, Underboss/Capo: 관리 Crew + 본인 배정, Soldier: 소속 Crew + 본인 배정, Associate: 본인 배정.

요구사항 FR-12, FR-21, NFR-04, AR-03 · **권한** 로그인 사용자 (역할별 업무 조회 범위 적용) · **성공** 200 · **구현 위치** `routes/tasks.js:57`

위치	이름	타입	필수	설명 / 제약
query	`from`	datetime		기간 시작 (업무 기간이 겹치는 항목) · 예: "2026-10-01T00:00:00Z"
query	`to`	datetime		기간 종료 · 예: "2026-12-31T00:00:00Z"
query	`crewId`	integer		Crew
query	`status`	array<string>		쉼표 구분 다중 (예: PLANNED,IN_PROGRESS) · 값: PLANNED | IN_PROGRESS | ON_HOLD | COMPLETED | CANCELLED
query	`priority`	array<string>		값: LOW | NORMAL | HIGH | CRITICAL
query	`memberId`	integer		담당(배정) 구성원
query	`q`	string		제목/설명/장소 검색 · 예: "창고"
query	`overdue`	boolean		true: 종료 시각이 지났는데 완료/취소되지 않은 업무
query	`unassigned`	boolean		true: 배정 인원이 없는 업무
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: id, startAt, priority, createdAt, status (내림차순 '-', 기본 -startAt)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "title": "물류 창고 정기 점검",
      "description": "물류 창고 정기 점검 업무",
      "crewId": 1,
      "crewName": "Brooklyn Crew",
      "priority": "HIGH",
      "status": "PLANNED",
      "startAt": "2026-10-19T09:00:00.000Z",
      "endAt": "2026-10-19T12:00:00.000Z",
      "location": "Brooklyn 창고 3동",
      "allowConcurrent": false,
      "requiredSkills": [
        "security"
      ],
      "resultOutcome": null,
      "resultSummary": null,
      "resultValue": null,
      "completedAt": null,
      "assigneeCount": 2
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401

API-095 GET `/api/tasks/export`

업무 목록 CSV 내보내기 (탐색 조건 동일 적용)

요구사항 FR-12, FR-22 · **권한** 로그인 + 권한 task.create 또는 stats.read · **성공** 200 text/csv · **구현 위치** `routes/tasks.js:75`

위치	이름	타입	필수	설명 / 제약
query	`from`	datetime		기간 시작 (업무 기간이 겹치는 항목) · 예: "2026-10-01T00:00:00Z"
query	`to`	datetime		기간 종료 · 예: "2026-12-31T00:00:00Z"
query	`crewId`	integer		Crew
query	`status`	array<string>		쉼표 구분 다중 (예: PLANNED,IN_PROGRESS) · 값: PLANNED | IN_PROGRESS | ON_HOLD | COMPLETED | CANCELLED
query	`priority`	array<string>		값: LOW | NORMAL | HIGH | CRITICAL

query	`memberId`	integer		담당(배정) 구성원
query	`q`	string		제목/설명/장소 검색 · 예: "창고"
query	`overdue`	boolean		true: 종료 시각이 지났는데 완료/취소되지 않은 업무
query	`unassigned`	boolean		true: 배정 인원이 없는 업무

Response 예시 (200)

```
id,title,crewName,priority,status,startAt,endAt,assigneeCount,resultOutcome
1,물류 창고 정기 점검,Brooklyn Crew,HIGH,PLANNED,...
```

오류 응답: 400, 401, 403

API-096 GET `/api/tasks/calendar`

업무 캘린더 (일자별 그룹)

요구사항 FR-12, FR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/tasks.js:88`

위치	이름	타입	필수	설명 / 제약
query	`from`	datetime	Y	예: "2026-10-01T00:00:00Z"
query	`to`	datetime	Y	예: "2026-10-31T23:59:59Z"
query	`crewId`	integer		
query	`memberId`	integer		담당(배정) 구성원
query	`status`	array<string>		쉼표 구분 다중 (예: PLANNED,IN_PROGRESS) · 값: PLANNED | IN_PROGRESS | ON_HOLD | COMPLETED | CANCELLED

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "date": "2026-10-19",
      "count": 2,
      "tasks": [
        {
          "id": 1,
          "title": "물류 창고 정기 점검",
          "status": "PLANNED",
          "priority": "HIGH",
          "startAt": "2026-10-19T09:00:00.000Z"
        }
      ]
    }
  ]
}
```


오류 응답: 400, 401

API-097 GET `/api/tasks/:id`

업무 상세 (참여 구성원, 배정 자원 포함)

요구사항 FR-07, FR-08, DR-02 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/tasks.js:112`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "title": "물류 창고 정기 점검",
    "description": "물류 창고 정기 점검 업무",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "priority": "HIGH",
    "status": "PLANNED",
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "location": "Brooklyn 창고 3동",
    "allowConcurrent": false,
    "requiredSkills": [
      "security"
    ],
    "resultOutcome": null,
    "resultSummary": null,
    "resultValue": null,
    "completedAt": null,
    "assigneeCount": 2,
    "assignees": [
      {
        "assignmentId": 1,
        "memberId": 8,
        "name": "Paulie Conti",
        "taskRole": "현장 책임",
        "status": "ACCEPTED"
      }
    ],
    "resources": [
      {
        "allocationId": 1,
        "resourceId": 1,
        "code": "VH-001",
        "name": "검정 세단",
        "status": "RESERVED"
      }
    ]
  }
}
```


오류 응답: 400, 401, 403, 404

API-098 POST /api/tasks

업무 등록 (기본 정보, 일정, 중요도)

요구사항 FR-07, BR-09 · **권한** 로그인 + 권한 task.create · **성공** 201 · **구현 위치** `routes/tasks.js:127`

Body 필드	타입	필수	설명 / 제약
`title`	string	Y	최대 100자
`description`	string		최대 5000자
`crewId`	integer	Y	

`priority`	string		값: LOW | NORMAL | HIGH | CRITICAL, 기본 "NORMAL"
`startAt`	datetime	Y	
`endAt`	datetime	Y	
`location`	string		최대 100자
`allowConcurrent`	boolean		true 이면 다른 업무와 동시 수행 가능(BR-03 예외) · 기본 false
`requiredSkills`	array<string>		

Request 예시

```
{
  "title": "신규 거래처 실사",
  "description": "신규 거래처 매장 실사 및 보
고",
  "crewId": 1,
  "priority": "HIGH",
  "startAt": "2026-10-25T09:00:00Z",
  "endAt": "2026-10-25T12:00:00Z",
  "location": "Brooklyn 5번가",
  "requiredSkills": [
    "negotiation"
  ]
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": 25,
    "title": "신규 거래처 실사",
    "description": "물류 창고 정기 점검 업무",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "priority": "HIGH",
    "status": "PLANNED",
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "location": "Brooklyn 창고 3동",
    "allowConcurrent": false,
    "requiredSkills": [
      "security"
    ],
    "resultOutcome": null,
    "resultSummary": null,
    "resultValue": null,
    "completedAt": null,
    "assigneeCount": 2
  }
}
```

오류 응답: 400, 401, 403, 404

API-099 PATCH `/api/tasks/:id`

업무 기본 정보 수정 (제목/설명/중요도/장소/필요 기술)

BR-06: 완료된 업무는 수정할 수 없다. 일정 변경은 PATCH /schedule, 상태 변경은 PATCH /status 를 사용한다.

요구사항 FR-07, BR-06 · **권한** 로그인 + 권한 task.update · **성공** 200 · **구현 위치** `routes/tasks.js:151`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`title`	string		최대 100자
`description`	string		최대 5000자, null 허용
`priority`	string		값: LOW | NORMAL | HIGH | CRITICAL
`location`	string		최대 100자, null 허용
`requiredSkills`	array<string>		

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시

```
{
  "title": "물류 창고 정기 점검 (2차)",
  "priority": "CRITICAL"
}
```

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": 1,
    "title": "물류 창고 정기 점검",
    "description": "물류 창고 정기 점검 업무",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "priority": "HIGH",
    "status": "PLANNED",
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "location": "Brooklyn 창고 3동",
    "allowConcurrent": false,
    "requiredSkills": [
      "security"
    ],
    "resultOutcome": null,
    "resultSummary": null,
    "resultValue": null,
    "completedAt": null,
    "assigneeCount": 2
  }
}
```


오류 응답: 400, 401, 403, 404, 409 — 409 BR-06: 완료된 업무의 정보는 변경할 수 없습니다.

API-100 PATCH `/api/tasks/:id/schedule`

업무 일정 변경 (참여자 시간 중복·자원 중복 재검사)

새 일정이 배정된 구성원의 다른 업무와 겹치면 BR-03, 예약된 자원이 다른 배정과 겹치면 BR-07 위반으로 거부된다. 통과 시 해당 업무의 자원 예약 기간도 함께 이동한다.

요구사항 FR-07, BR-03, BR-07 · **권한** 로그인 + 권한 task.update · **성공** 200 · **구현 위치** `routes/tasks.js:176`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`startAt`	datetime	Y	
`endAt`	datetime	Y	
`allowConcurrent`	boolean		

Request 예시

```
{
  "startAt": "2026-10-19T10:30:00Z",
  "endAt": "2026-10-19T13:00:00Z",
  "allowConcurrent": false
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "title": "물류 창고 정기 점검",
    "description": "물류 창고 정기 점검 업무",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "priority": "HIGH",
    "status": "PLANNED",
    "startAt": "2026-10-19T10:30:00.000Z",
    "endAt": "2026-10-19T13:00:00.000Z",
    "location": "Brooklyn 창고 3동",
    "allowConcurrent": false,
    "requiredSkills": [
      "security"
    ],
    "resultOutcome": null,
    "resultSummary": null,
    "resultValue": null,
    "completedAt": null,
    "assigneeCount": 2,
    "movedReservations": 1
  }
}
```


오류 응답: 400, 401, 403, 404, 409 — 409 BR-03: 구성원에게 동일 시간대에 이미 배정된 업무가 있습니다.

API-101 PATCH `/api/tasks/:id/status`

업무 진행 상태 변경 (전이 규칙 검사, 이력 기록)

허용 전이: PLANNED→IN_PROGRESS|CANCELLED, IN_PROGRESS→ON_HOLD|CANCELLED, ON_HOLD→IN_PROGRESS|CANCELLED, CANCELLED→PLANNED. 완료(COMPLETED)는 결과 입력이 필요한 POST /complete 로만 가능. 취소 시 자원 예약도 취소된다.

요구사항 FR-07, FR-10, FR-20 · **권한** 로그인 + 권한 task.progress · **성공** 200 · **구현 위치** `routes/tasks.js:217`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`status`	string	Y	값: PLANNED | IN_PROGRESS | ON_HOLD | CANCELLED
`reason`	string		최대 300자

Request 예시


```
{
  "status": "IN_PROGRESS",
  "reason": "현장 도착, 작업 시작"
}
```


Response 예시 (200)

```
{
  "success": true,
  "data": {
    "taskId": 1,
    "from": "PLANNED",
    "to": "IN_PROGRESS",
    "cancelledReservations": 0
  }
}
```


오류 응답: 400, 401, 403, 404, 422 — 422: 상태 전이 COMPLETED → PLANNED 는 허용되지 않습니다.

API-102 GET `/api/tasks/:id/transitions`

현재 상태에서 가능한 다음 상태 목록

요구사항 FR-10 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/tasks.js:257`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "taskId": 1,
    "current": "PLANNED",
    "allowed": [
      "IN_PROGRESS",
      "CANCELLED"
    ],
    "canComplete": false,
    "canReopen": false
  }
}
```


오류 응답: 400, 401, 403, 404

API-103 GET `/api/tasks/:id/status-history`

업무 상태·배정 변경 이력 (MongoDB)

요구사항 FR-10, FR-20 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/tasks.js:271`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1
query	`event`	string		값: CREATED | UPDATED | RESCHEDULED | STATUS_CHANGED | COMPLETED | REOPENED | RESULT_AMENDED | ASSIGNED | UNASSIGNED | REPLACED | ASSIGNMENT_ACCEPTED | ASSIGNMENT_DECLINED | PARTICIPATION_RECORDED
query	`page`	integer		페이지 번호 (기본 1)

query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: at (내림차순 '-', 기본 -at)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": "6715...",
      "taskId": 1,
      "event": "STATUS_CHANGED",
      "from": "PLANNED",
      "to": "IN_PROGRESS",
      "note": "현장 도착",
      "by": {
        "id": 5,
        "username": "capo1",
        "role": "CAPO"
      },
      "at": "2026-10-19T09:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

오류 응답: 400, 401, 403, 404

API-104 POST `/api/tasks/:id/complete`

업무 완료 처리 (결과 및 참여 기록 입력)

진행중(IN_PROGRESS) 업무만 완료 가능. 결과(성공/부분/실패, 요약, 성과값)와 참여자별 출석·기여도를 기록하며, 이후 평가·통계에 활용된다(FR-11). 미사용 자원 예약은 자동 취소된다.

요구사항 FR-10, FR-11, FR-20 · **권한** 로그인 + 권한 task.progress · **성공** 200 · **구현 위치** `routes/tasks.js:288`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`outcome`	string	Y	값: SUCCESS | PARTIAL | FAILURE
`summary`	string	Y	최대 5000자
`value`	number		성과 수치(금액 등) · ≥0

`participation`	array<object>		[{ memberId, attendance: PRESENT|PARTIAL|ABSENT, contribution: 0~100, note }]

Request 예시

```
{
  "outcome": "SUCCESS",
  "summary": "화물 인수 완료, 이상 없음",
  "value": 15000,
  "participation": [
    {
      "memberId": 10,
      "attendance": "PRESENT",
      "contribution": 90
    },
    {
      "memberId": 11,
      "attendance": "PRESENT",
      "contribution": 70
    }
  ]
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 3,
    "title": "물류 창고 정기 점검",
    "description": "물류 창고 정기 점검 업무",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "priority": "HIGH",
    "status": "COMPLETED",
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "location": "Brooklyn 창고 3동",
    "allowConcurrent": false,
    "requiredSkills": [
      "security"
    ],
    "resultOutcome": "SUCCESS",
    "resultSummary": "화물 인수 완료, 이상 없
음",
    "resultValue": 15000,
    "completedAt": null,
    "assigneeCount": 2
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-105 GET `/api/tasks/:id/result`

업무 결과 및 참여 기록 조회

요구사항 FR-11, FR-19 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/tasks.js:332`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "taskId": 4,
    "status": "COMPLETED",
    "outcome": "SUCCESS",
    "summary": "장부 정리 완료",
    "value": 12500,
    "completedAt": "2026-10-11T17:00:00.000Z",
    "completedBy": "capo1",
    "participants": [
      {
        "memberId": 8,
        "name": "Paulie Conti",
        "taskRole": "장부 검토",
        "attendance": "PRESENT",
        "contribution": 80
      }
    ],
    "amendments": 0
  }
}
```


오류 응답: 400, 401, 403, 404, 422

API-106 PATCH `/api/tasks/:id/result`

완료된 업무 결과 수정 (Underboss 이상, 사유 필수, 수정 전 값 이력 보존)

BR-06: 완료된 업무의 핵심 결과는 일반 구성원(Soldier/Capo)이 변경할 수 없다. task.result.amend 권한(Underboss 이상)만 수정 가능하며 변경 전/후 값이 이력에 남는다.

요구사항 BR-06, FR-11, FR-20 · **권한** 로그인 + 권한 task.result.amend · **성공** 200 · **구현 위치** `routes/tasks.js:355`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`outcome`	string		값: SUCCESS | PARTIAL | FAILURE
`summary`	string		최대 5000자
`value`	number		≥0, null 허용
`reason`	string	Y	최대 300자

Request 예시


```
{
  "outcome": "PARTIAL",
  "summary": "재검토 결과 일부 누락 확인",
  "value": 11000,
  "reason": "감사 결과 반영"
}
```


Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 4,
    "title": "물류 창고 정기 점검",
    "description": "물류 창고 정기 점검 업무",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "priority": "HIGH",
    "status": "COMPLETED",
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "location": "Brooklyn 창고 3동",
    "allowConcurrent": false,
    "requiredSkills": [
      "security"
    ],
    "resultOutcome": "PARTIAL",
    "resultSummary": null,
    "resultValue": 11000,
    "completedAt": null,
    "assigneeCount": 2
  }
}
```


오류 응답: 400, 401, 403, 404, 422 — 403: 권한(task.result.amend)이 필요합니다.

API-107 POST /api/tasks/:id/reopen

완료된 업무 재개 (Underboss 이상, COMPLETED → IN_PROGRESS)

요구사항 BR-06, FR-10 · **권한** 로그인 + 권한 task.result.amend · **성공** 200 · **구현 위치** `routes/tasks.js:382`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`reason`	string	Y	최대 300자

Request 예시

```
{
  "reason": "추가 작업 필요"
}
```

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "taskId": 4,
    "from": "COMPLETED",
    "to": "IN_PROGRESS"
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-108 POST /api/tasks/:id/duplicate

업무 복제 (새 일정으로 PLANNED 업무 생성, 체크리스트 포함)

요구사항 FR-07 · **권한** 로그인 + 권한 task.create · **성공** 201 · **구현 위치** `routes/tasks.js:398`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`startAt`	datetime	Y	
`endAt`	datetime	Y	
`title`	string		최대 100자

Request 예시

```
{
  "startAt": "2026-11-02T09:00:00Z",
  "endAt": "2026-11-02T12:00:00Z",
  "title": "물류 창고 정기 점검 (11월)"
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": 26,
    "title": "물류 창고 정기 점검 (11월)",
    "description": "물류 창고 정기 점검 업무",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "priority": "HIGH",
    "status": "PLANNED",
    "startAt": "2026-11-02T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "location": "Brooklyn 창고 3동",
    "allowConcurrent": false,
    "requiredSkills": [
      "security"
    ],
    "resultOutcome": null,
    "resultSummary": null,
    "resultValue": null,
    "completedAt": null,
    "assigneeCount": 2
  }
}
```

오류 응답: 400, 401, 403, 404

API-109 DELETE /api/tasks/:id

업무 삭제 (배정·자원 이력이 없는 PLANNED/CANCELLED 업무만)

참여·자원 이력이 있는 업무는 삭제할 수 없으며 CANCELLED 상태로 관리해야 한다(이력 보존).

요구사항 FR-07, DR-02 · **권한** 로그인 + 권한 task.create · **성공** 200 · **구현 위치** `routes/tasks.js:421`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 25,
    "deleted": true
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-110 GET `/api/tasks/:id/timeline`

업무 통합 타임라인 (상태 이력 + 코멘트 + 자원 사용 이벤트)

요구사항 FR-10, FR-15, FR-20 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/tasks.js:439`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "kind": "HISTORY",
      "event": "CREATED",
      "by": "capo1",
      "at": "2026-10-14T00:00:00.000Z"
    },
    {
      "kind": "COMMENT",
      "body": "점검 체크리스트 반드시 확인",
      "by": "capo1",
      "at": "2026-10-18T00:00:00.000Z"
    }
  ]
}
```

오류 응답: 400, 401, 403, 404

API-111 GET `/api/tasks/:id/candidates`

배정 후보 추천 (상태·시간 충돌 없는 구성원, 기술 적합도·실무 인력·업무량 순)

요구사항 FR-08, BR-02, BR-03 · **권한** 로그인 + 권한 task.assign · **성공** 200 · **구현 위치** `routes/tasks.js:460`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1
query	`includeOtherCrews`	boolean		관리 범위 내 다른 Crew 구성원도 포함 · 기본 false

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "memberId": 9,
      "name": "Joey Marino",
      "role": "SOLDIER",
      "status": "ACTIVE",
      "crewId": 1,
      "skillMatch": 1,
      "matchedSkills": [
        "security"
      ],
      "openAssignments": 1
    }
  ]
}
```


오류 응답: 400, 401, 403, 404, 422

API-112 GET `/api/tasks/:id/conflicts`

업무 충돌 진단 (참여자 일정 중복 BR-03, 자원 중복 BR-07)

요구사항 BR-03, BR-07, FR-12 · **권한** 로그인 + 권한 task.assign · **성공** 200 · **구현 위치** `routes/tasks.js:497`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "taskId": 1,
    "ok": true,
    "memberConflicts": [],
    "resourceConflicts": []
  }
}
```


오류 응답: 400, 401, 403, 404

12. Task Assignments

API-113 GET `/api/tasks/:id/assignments`

업무 참여 구성원 목록 (배정 상태 포함)

요구사항 FR-08, DR-02 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/assignments.js:41`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 2

query	`includeInactive`	boolean		true 이면 거절/해제된 배정도 포함 · 기본 false

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "assignmentId": 12,
      "taskId": 2,
      "memberId": 9,
      "name": "Joey Marino",
      "taskRole": "경비",
      "instructions": "정문 경비",
      "status": "ASSIGNED",
      "assignedAt": "2026-10-18T09:00:00.000Z"
    }
  ]
}
```

오류 응답: 400, 401, 403, 404

API-114 POST `/api/tasks/:id/assignments`

업무에 구성원 배정

BR-02: 정지(SUSPENDED)/탈퇴(WITHDRAWN)/가입대기 구성원 배정 불가. BR-03: 동일 시간대에 동시 수행 불가 업무가 있으면 배정 불가. 구성원 행을 잠근 트랜잭션에서 검사하여 동시 요청에도 규칙이 보장된다.

요구사항 FR-08, BR-02, BR-03, BR-05 · **권한** 로그인 + 권한 task.assign · **성공** 201 · **구현 위치** `routes/assignments.js:58`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 2

Body 필드	타입	필수	설명 / 제약
`memberId`	integer	Y	
`taskRole`	string		최대 50자
`instructions`	string		최대 2000자

Request 예시

```
{
  "memberId": 9,
  "taskRole": "경비",
  "instructions": "정문 경비, 10시 집결"
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "assignmentId": 12,
    "taskId": 2,
    "memberId": 9,
    "name": "Joey Marino",
    "taskRole": "경비",
    "instructions": "정문 경비",
    "status": "ASSIGNED",
    "assignedAt": "2026-10-
18T09:00:00.000Z"
  }
}
```


오류 응답: 400, 401, 403, 404, 409 — 409 BR-02: SUSPENDED 상태의 구성원은 새로운 업무에 배정될 수 없습니다. / 409 BR-03: 구성원에게 동일 시간대에 이미 배정된 업무가 있습니다.

API-115 PATCH /api/tasks/:id/assignments/:memberId

배정 정보 수정 (업무 내 역할, 지시사항)

요구사항 FR-08, FR-09, BR-06 · **권한** 로그인 + 권한 task.assign · **성공** 200 · **구현 위치** `routes/assignments.js:83`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 2
path	`memberId`	integer	Y	구성원 ID · 예: 9

Body 필드	타입	필수	설명 / 제약
`taskRole`	string		최대 50자, null 허용
`instructions`	string		최대 2000자, null 허용

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시


```
{
  "taskRole": "현장 보조",
  "instructions": "후문 경비로 변경"
}
```


Response 예시 (200)


```
{
  "success": true,
  "data": {
    "assignmentId": 12,
    "taskId": 2,
    "memberId": 9,
    "name": "Joey Marino",
    "taskRole": "현장 보조",
    "instructions": "후문 경비로 변경",
    "status": "ASSIGNED",
    "assignedAt": "2026-10-
18T09:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403, 404, 409

API-116 DELETE `/api/tasks/:id/assignments/:memberId`

배정 해제

요구사항 FR-08, BR-06 · **권한** 로그인 + 권한 task.assign · **성공** 200 · **구현 위치** `routes/assignments.js:106`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 2
path	`memberId`	integer	Y	구성원 ID · 예: 9
query	`reason`	string		최대 300자 · 예: "인원 조정"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "taskId": 2,
    "memberId": 9,
    "status": "REMOVED"
  }
}
```

오류 응답: 400, 401, 403, 404, 409

API-117 POST `/api/tasks/:id/assignments/:memberId/replace`

배정 인원 교체 (기존 해제 + 신규 배정을 하나의 트랜잭션으로)

요구사항 FR-08, BR-02, BR-03 · **권한** 로그인 + 권한 task.assign · **성공** 200 · **구현 위치** `routes/assignments.js:125`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 2
path	`memberId`	integer	Y	구성원 ID · 예: 9

Body 필드	타입	필수	설명 / 제약
`newMemberId`	integer	Y	
`reason`	string		최대 300자

Request 예시

```
{
  "newMemberId": 14,
  "reason": "부상으로 교체"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "taskId": 1,
    "removedMemberId": 9,
    "newAssignmentId": 13,
    "newMemberId": 14
  }
}
```

오류 응답: 400, 401, 403, 404, 409

API-118 PUT /api/tasks/:id/assignments/:memberId/participation

완료 업무의 참여 기록(출석·기여도) 입력

완료된 업무에 대해 참여 기록을 입력한다. 이미 입력된 기록의 수정은 BR-06 에 따라 task.result.amend 권한(Underboss 이상)이 필요하다.

요구사항 FR-11, BR-06, FR-18 · **권한** 로그인 + 권한 task.progress · **성공** 200 · **구현 위치** `routes/assignments.js:149`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 4
path	`memberId`	integer	Y	예: 8

Body 필드	타입	필수	설명 / 제약
`attendance`	string	Y	값: PRESENT | PARTIAL | ABSENT
`contribution`	integer	Y	≥0, ≤100
`note`	string		최대 500자

Request 예시

```
{
  "attendance": "PRESENT",
  "contribution": 85,
  "note": "핵심 역할 수행"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "taskId": 4,
    "memberId": 8,
    "attendance": "PRESENT",
    "contribution": 85,
    "note": "핵심 역할 수행"
  }
}
```

오류 응답: 400, 401, 403, 404, 409, 422 — 409 BR-06: 이미 기록된 참여 결과는 Underboss 이상만 수정할 수 있습니다.

13. Task Comments & Checklist

API-119 GET /api/tasks/:id/comments

업무 코멘트 목록

요구사항 FR-09, DR-04 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/taskCollab.js:28`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: createdAt (내림차순 '-', 기본 createdAt)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": "6716...",
      "taskId": 1,
      "author": {
        "id": 5,
        "username": "capo1",
        "role": "CAPO"
      },
      "body": "점검 체크리스트 반드시 확인",
      "createdAt": "2026-10-18T00:00:00.000Z",
      "updatedAt": "2026-10-18T00:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401, 403, 404

API-120 POST /api/tasks/:id/comments

업무 코멘트 작성 (업무 조회 권한자)

요구사항 FR-09, DR-04 · **권한** 로그인 사용자 · **성공** 201 · **구현 위치** `routes/taskCollab.js:42`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`body`	string	Y	최대 2000자

Request 예시

```
{
  "body": "현장 도착했습니다."
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": "6716...",
    "taskId": 1,
    "author": {
      "id": 5,
      "username": "capo1",
      "role": "CAPO"
    },
    "body": "점검 체크리스트 반드시 확인",
    "createdAt": "2026-10-
18T00:00:00.000Z",
    "updatedAt": "2026-10-
18T00:00:00.000Z"
  }
}
```


오류 응답: 400, 401, 403, 404

API-121 PATCH `/api/tasks/:id/comments/:commentId`

내 코멘트 수정 (작성자 본인)

요구사항 FR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/taskCollab.js:56`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1
path	`commentId`	objectId	Y	예: "6716aa0000000000000000001"

Body 필드	타입	필수	설명 / 제약
`body`	string	Y	최대 2000자

Request 예시


```
{
  "body": "현장 도착 (10분 지연)"
}
```


Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": "6716...",
    "taskId": 1,
    "author": {
      "id": 5,
      "username": "capo1",
      "role": "CAPO"
    },
    "body": "점검 체크리스트 반드시 확인",
    "createdAt": "2026-10-
18T00:00:00.000Z",
    "updatedAt": "2026-10-
18T00:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403, 404

API-122 DELETE `/api/tasks/:id/comments/:commentId`

코멘트 삭제 (작성자 또는 업무 관리자)

요구사항 FR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/taskCollab.js:71`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1
path	`commentId`	objectId	Y	예: "6716aa000000000000000001"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": "6716aa000000000000000001",
    "deleted": true
  }
}
```

오류 응답: 400, 401, 403, 404

API-123 GET `/api/tasks/:id/checklist`

업무 체크리스트 조회 (진행률 포함)

요구사항 FR-09, FR-10 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/taskCollab.js:89`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "taskId": 1,
    "items": [
      {
        "id": "1",
        "text": "출입구 잠금 상태 확인",
        "done": true,
        "doneBy": "soldier1"
      }
    ],
    "progress": {
      "done": 1,
      "total": 2
    }
  }
}
```

오류 응답: 400, 401, 403, 404

API-124 POST /api/tasks/:id/checklist

체크리스트 항목 추가 (업무 관리자)

요구사항 FR-07, FR-09 · **권한** 로그인 + 권한 task.update · **성공** 201 · **구현 위치** `routes/taskCollab.js:100`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`text`	string	Y	최대 200자

Request 예시

```
{
  "text": "비상 연락망 확인"
}
```

Response 예시 **(201)**

```
{
  "success": true,
  "data": {
    "taskId": 1,
    "items": [
      {
        "id": "1",
        "text": "출입구 잠금 상태 확인",
        "done": true,
        "doneBy": "soldier1"
      }
    ],
    "progress": {
      "done": 1,
      "total": 2
    }
  }
}
```

오류 응답: 400, 401, 403, 404, 409

API-125 PATCH /api/tasks/:id/checklist/:itemId

체크리스트 항목 완료 체크/수정 (참여자: 체크만, 관리자: 문구 수정)

요구사항 FR-09, FR-10, BR-06 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/taskCollab.js:118`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1
path	`itemId`	string	Y	최대 10자 · 예: "1"

Body 필드	타입	필수	설명 / 제약
`done`	boolean		
`text`	string		최대 200자

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시

Response 예시 **(200)**

```
{
  "done": true,
  "text": "출입구 및 비상구 잠금 확인"
}
```

```
{
  "success": true,
  "data": {
    "taskId": 1,
    "items": [
      {
        "id": "1",
        "text": "출입구 잠금 상태 확인",
        "done": true,
        "doneBy": "soldier1"
      }
    ],
    "progress": {
      "done": 1,
      "total": 2
    }
  }
}
```

오류 응답: 400, 401, 403, 404, 409

API-126 DELETE /api/tasks/:id/checklist/:itemId

체크리스트 항목 삭제 (업무 관리자)

요구사항 FR-07 · **권한** 로그인 + 권한 task.update · **성공** 200 · **구현 위치** `routes/taskCollab.js:143`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	업무 ID · 예: 1
path	`itemId`	string	Y	최대 10자 · 예: "2"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "taskId": 1,
    "items": [
      {
        "id": "1",
        "text": "출입구 잠금 상태 확인",
        "done": true,
        "doneBy": "soldier1"
      }
    ],
    "progress": {
      "done": 1,
      "total": 2
    }
  }
}
```

오류 응답: 400, 401, 403, 404, 409

14. Task Templates

API-127 GET `/api/task-templates`

업무 템플릿 목록

요구사항 FR-07 · **권한** 로그인 + 권한 template.manage · **성공** 200 · **구현 위치** `routes/templates.js:36`

위치	이름	타입	필수	설명 / 제약
query	`q`	string		이름 검색

Response 예시 (200)


```
{
  "success": true,
  "data": [
    {
      "id": "6717...",
      "name": "정기 수금",
      "description": "구역별 정기 수금 업무",
      "defaults": {
        "priority": "NORMAL",
        "durationHours": 4,
        "location": "구역 순회",
        "allowConcurrent": false,
        "requiredSkills": [
          "negotiation"
        ]
      },
      "checklist": [
        "수금 대상 명단 확인",
        "수금액 기록",
        "본부 입금"
      ],
      "createdBy": {
        "id": 2,
        "username": "boss",
        "role": "BOSS"
      }
    }
  ]
}
```


오류 응답: 400, 401, 403

API-128 GET `/api/task-templates/:id`

업무 템플릿 상세

요구사항 FR-07 · **권한** 로그인 + 권한 template.manage · **성공** 200 · **구현 위치** `routes/templates.js:46`

위치	이름	타입	필수	설명 / 제약
path	`id`	objectId	Y	예: "6717aa000000000000000001"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": "6717...",
    "name": "정기 수금",
    "description": "구역별 정기 수금 업무",
    "defaults": {
      "priority": "NORMAL",
      "durationHours": 4,
      "location": "구역 순회",
      "allowConcurrent": false,
      "requiredSkills": [
        "negotiation"
      ]
    },
    "checklist": [
      "수금 대상 명단 확인",
      "수금액 기록",
      "본부 입금"
    ],
    "createdBy": {
      "id": 2,
      "username": "boss",
      "role": "BOSS"
    }
  }
}
```

오류 응답: 400, 401, 403, 404

API-129 POST /api/task-templates

업무 템플릿 등록

요구사항 FR-07 · **권한** 로그인 + 권한 template.manage · **성공** 201 · **구현 위치** `routes/templates.js:54`

Body 필드	타입	필수	설명 / 제약
`name`	string	Y	최대 100자
`description`	string		최대 1000자
`defaults`	object	Y	{ priority, durationHours, location, allowConcurrent, requiredSkills }
`checklist`	array<string>		

Request 예시

Response 예시 (201)

```
{
  "name": "거점 보안 점검",
  "description": "거점별 보안 상태 점검",
  "defaults": {
    "priority": "HIGH",
    "durationHours": 3,
    "location": "각 거점",
    "allowConcurrent": false,
    "requiredSkills": [
      "security"
    ]
  },
  "checklist": [
    "출입문 확인",
    "CCTV 확인"
  ]
}
```

```
{
  "success": true,
  "data": {
    "id": "6717...",
    "name": "정기 수금",
    "description": "구역별 정기 수금 업무",
    "defaults": {
      "priority": "NORMAL",
      "durationHours": 4,
      "location": "구역 순회",
      "allowConcurrent": false,
      "requiredSkills": [
        "negotiation"
      ]
    },
    "checklist": [
      "수금 대상 명단 확인",
      "수금액 기록",
      "본부 입금"
    ],
    "createdBy": {
      "id": 2,
      "username": "boss",
      "role": "BOSS"
    }
  }
}
```

오류 응답: 400, 401, 403

API-130 PATCH `/api/task-templates/:id`

업무 템플릿 수정

요구사항 FR-07 · **권한** 로그인 + 권한 template.manage · **성공** 200 · **구현 위치** `routes/templates.js:66`

위치	이름	타입	필수	설명 / 제약
path	`id`	objectId	Y	예: "6717aa000000000000000001"

Body 필드	타입	필수	설명 / 제약
`name`	string		최대 100자
`description`	string		최대 1000자
`defaults`	object		{ priority, durationHours, location, allowConcurrent, requiredSkills }
`checklist`	array<string>		

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시

Response 예시 (200)

```
{
  "name": "거점 보안 점검",
  "description": "거점별 보안 상태 점검",
  "defaults": {
    "priority": "HIGH",
    "durationHours": 3,
    "location": "각 거점",
    "allowConcurrent": false,
    "requiredSkills": [
      "security"
    ]
  },
  "checklist": [
    "출입문 확인",
    "CCTV 확인"
  ]
}
```

```
{
  "success": true,
  "data": {
    "id": "6717...",
    "name": "정기 수금",
    "description": "구역별 정기 수금 업무",
    "defaults": {
      "priority": "NORMAL",
      "durationHours": 4,
      "location": "구역 순회",
      "allowConcurrent": false,
      "requiredSkills": [
        "negotiation"
      ]
    },
    "checklist": [
      "수금 대상 명단 확인",
      "수금액 기록",
      "본부 입금"
    ],
    "createdBy": {
      "id": 2,
      "username": "boss",
      "role": "BOSS"
    }
  }
}
```

오류 응답: 400, 401, 403, 404

API-131 DELETE `/api/task-templates/:id`

업무 템플릿 삭제

요구사항 FR-07 · **권한** 로그인 + 권한 template.manage · **성공** 200 · **구현 위치** `routes/templates.js:80`

위치	이름	타입	필수	설명 / 제약
path	`id`	objectId	Y	예: "6717aa000000000000000001"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": "6717aa000000000000000001",
    "deleted": true
  }
}
```

오류 응답: 400, 401, 403, 404

API-132 POST `/api/task-templates/:id/instantiate`

템플릿으로 업무 생성 (기본값 + 체크리스트 자동 적용)

요구사항 FR-07 · **권한** 로그인 + 권한 template.manage 또는 task.create · **성공** 201 · **구현 위치** `routes/templates.js:92`

위치	이름	타입	필수	설명 / 제약
path	`id`	objectId	Y	예: "6717aa000000000000000001"

Body 필드	타입	필수	설명 / 제약
`crewId`	integer	Y	
`startAt`	datetime	Y	
`title`	string		최대 100자

Request 예시

```
{
  "crewId": 1,
  "startAt": "2026-10-28T10:00:00Z",
  "title": "정기 수금 - 10월 4주차"
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": 27,
    "title": "정기 수금 - 10월 4주차",
    "status": "PLANNED",
    "crewId": 1,
    "checklistItems": 3
  }
}
```

오류 응답: 400, 401, 403, 404

15. Meetings

API-133 GET `/api/meetings`

회의 목록 (내가 참석/주관하거나 관리 Crew 회의)

요구사항 FR-21, FR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/meetings.js:41`

위치	이름	타입	필수	설명 / 제약
query	`from`	datetime		
query	`to`	datetime		
query	`crewId`	integer		
query	`status`	string		값: SCHEDULED | CANCELLED | DONE
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: startAt, id (내림차순 '-', 기본 startAt)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "title": "Brooklyn 주간 회의",
      "agenda": "이번 주 업무 배정",
      "crewId": 1,
      "crewName": "Brooklyn Crew",
      "location": "Brooklyn 본부",
      "startAt": "2026-10-20T18:00:00.000Z",
      "endAt": "2026-10-20T19:00:00.000Z",
      "status": "SCHEDULED",
      "organizerId": 5,
      "organizerName": "Tony Russo",
      "attendeeCount": 4
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401

API-134 GET `/api/meetings/:id`

회의 상세 (참석자 응답 현황, 회의록 포함)

요구사항 FR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/meetings.js:67`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	회의 ID · 예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "title": "Brooklyn 주간 회의",
    "agenda": "이번 주 업무 배정",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "location": "Brooklyn 본부",
    "startAt": "2026-10-20T18:00:00.000Z",
    "endAt": "2026-10-20T19:00:00.000Z",
    "status": "SCHEDULED",
    "organizerId": 5,
    "organizerName": "Tony Russo",
    "attendeeCount": 4,
    "attendees": [
      {
        "memberId": 8,
        "name": "Paulie Conti",
        "rsvp": "YES"
      }
    ],
    "minutes": {
      "content": "...",
      "decisions": [
        "창고 점검 목요일 확정"
      ]
    }
  }
}
```


오류 응답: 400, 401, 403, 404

API-135 POST /api/meetings

회의 생성 (참석자 초대 및 알림)

요구사항 FR-05, FR-09 · **권한** 로그인 + 권한 meeting.manage · **성공** 201 · **구현 위치** `routes/meetings.js:83`

Body 필드	타입	필수	설명 / 제약
`title`	string	Y	최대 100자
`agenda`	string		최대 2000자
`crewId`	integer		
`location`	string		최대 100자
`startAt`	datetime	Y	
`endAt`	datetime	Y	
`attendeeIds`	array<integer>		

Request 예시

Response 예시 (201)

```
{
  "title": "Harbor 긴급 회의",
  "agenda": "항만 출입 절차 변경 대응",
  "crewId": 2,
  "location": "Harbor 사무실",
  "startAt": "2026-10-21T18:00:00Z",
  "endAt": "2026-10-21T19:00:00Z",
  "attendeeIds": [
    10,
    11
  ]
}
```

```
{
  "success": true,
  "data": {
    "id": 1,
    "title": "Brooklyn 주간 회의",
    "agenda": "이번 주 업무 배정",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "location": "Brooklyn 본부",
    "startAt": "2026-10-20T18:00:00.000Z",
    "endAt": "2026-10-20T19:00:00.000Z",
    "status": "SCHEDULED",
    "organizerId": 5,
    "organizerName": "Tony Russo",
    "attendeeCount": 4
  }
}
```

오류 응답: 400, 401, 403

API-136 PATCH `/api/meetings/:id`

회의 정보/일정 수정

요구사항 FR-09 · **권한** 로그인 + 권한 meeting.manage · **성공** 200 · **구현 위치** `routes/meetings.js:116`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	회의 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`title`	string		최대 100자
`agenda`	string		최대 2000자, null 허용
`location`	string		최대 100자, null 허용
`startAt`	datetime		
`endAt`	datetime		

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시

```
{
  "location": "Brooklyn 본부 2층"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "title": "Brooklyn 주간 회의",
    "agenda": "이번 주 업무 배정",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "location": "Brooklyn 본부",
    "startAt": "2026-10-20T18:00:00.000Z",
    "endAt": "2026-10-20T19:00:00.000Z",
    "status": "SCHEDULED",
    "organizerId": 5,
    "organizerName": "Tony Russo",
    "attendeeCount": 4
  }
}
```


오류 응답: 400, 401, 403, 404, 422

API-137 DELETE /api/meetings/:id

회의 취소

요구사항 FR-09 · **권한** 로그인 + 권한 meeting.manage · **성공** 200 · **구현 위치** `routes/meetings.js:142`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	회의 ID · 예: 1

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": 1,
    "title": "Brooklyn 주간 회의",
    "agenda": "이번 주 업무 배정",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "location": "Brooklyn 본부",
    "startAt": "2026-10-20T18:00:00.000Z",
    "endAt": "2026-10-20T19:00:00.000Z",
    "status": "CANCELLED",
    "organizerId": 5,
    "organizerName": "Tony Russo",
    "attendeeCount": 4
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-138 POST `/api/meetings/:id/attendees`

회의 참석자 추가

요구사항 FR-09 · **권한** 로그인 + 권한 meeting.manage · **성공** 201 · **구현 위치** `routes/meetings.js:158`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	회의 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`memberId`	integer	Y	

Request 예시

```
{
  "memberId": 15
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "meetingId": 1,
    "memberId": 15,
    "rsvp": "PENDING"
  }
}
```

오류 응답: 400, 401, 403, 404, 409, 422

API-139 DELETE `/api/meetings/:id/attendees/:memberId`

회의 참석자 제외

요구사항 FR-09 · **권한** 로그인 + 권한 meeting.manage · **성공** 200 · **구현 위치** `routes/meetings.js:177`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	회의 ID · 예: 1
path	`memberId`	integer	Y	예: 15

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "meetingId": 1,
    "memberId": 15,
    "removed": true
  }
}
```

오류 응답: 400, 401, 403, 404

API-140 POST /api/meetings/:id/rsvp

회의 참석 여부 응답 (초대된 본인)

요구사항 FR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/meetings.js:193`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	회의 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`rsvp`	string	Y	값: YES | NO | MAYBE

Request 예시

```
{
  "rsvp": "YES"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "meetingId": 1,
    "memberId": 8,
    "rsvp": "YES"
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-141 PUT /api/meetings/:id/minutes

회의록 작성/갱신 (MongoDB, 결정사항 포함, 시작 이후 작성 시 DONE 처리)

요구사항 DR-04, FR-20 · **권한** 로그인 + 권한 meeting.manage · **성공** 200 · **구현 위치** `routes/meetings.js:208`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	회의 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`content`	string	Y	최대 20000자
`decisions`	array<string>		

Request 예시

```
{
  "content": "창고 점검 일정 및 인원 확정",
  "decisions": [
    "창고 점검 목요일 09시 확정",
    "VH-001 차량 사용"
  ]
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "meetingId": 1,
    "content": "창고 점검 일정 및 인원 확정",
    "decisions": [
      "창고 점검 목요일 09시 확정"
    ],
    "updatedBy": "capo1",
    "status": "DONE"
  }
}
```


오류 응답: 400, 401, 403, 404, 422

16. Resources

API-142 GET `/api/resources`

자원 목록/검색 (유형·상태·Crew·검색어)

FR-14: 요청자의 역할 등급(minRoleLevel)과 소속 Crew 에 따라 볼 수 있는 자원만 반환된다. usable=false 는 사용 불가/점검 필요 자원(FR-16).

요구사항 FR-13, FR-14, FR-16, FR-21, NFR-04 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/resources.js:41`

위치	이름	타입	필수	설명 / 제약
query	`type`	string		유형 코드 · 예: "VEHICLE"
query	`status`	array<string>		값: AVAILABLE | IN_USE | NEEDS_INSPECTION | MAINTENANCE | UNAVAILABLE | RETIRED
query	`crewId`	integer		
query	`q`	string		코드/이름/위치 검색 · 예: "세단"
query	`usable`	boolean		true: 정상 자원만, false: 사용 불가/점검 필요 자원만
query	`includeRetired`	boolean		기본 false
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: id, code, name, status, updatedAt (내림차순 '-', 기본 code)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "code": "VH-001",
      "name": "검정 세단",
      "typeId": 1,
      "typeCode": "VEHICLE",
      "typeName": "차량",
      "crewId": 1,
      "crewName": "Brooklyn Crew",
      "status": "AVAILABLE",
      "location": "Brooklyn 차고",
      "minRoleLevel": 2,
      "description": "검정 세단 (VH-001)",
      "usable": true
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401

API-143 GET `/api/resources/available`

지정 시간대에 내가 이용 가능한 자원 (권한·상태·기존 배정 고려)

요구사항 FR-14, BR-07, BR-08 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/resources.js:71`

위치	이름	타입	필수	설명 / 제약
query	`startAt`	datetime	Y	예: "2026-10-19T10:00:00Z"
query	`endAt`	datetime	Y	예: "2026-10-19T11:00:00Z"
query	`type`	string		예: "VEHICLE"

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": 9,
      "code": "VH-004",
      "name": "오토바이",
      "typeId": 1,
      "typeCode": "VEHICLE",
      "typeName": "차량",
      "crewId": 1,
      "crewName": "Brooklyn Crew",
      "status": "AVAILABLE",
      "location": "Brooklyn 차고",
      "minRoleLevel": 2,
      "description": "검정 세단 (VH-001)",
      "usable": true
    }
  ]
}
```


오류 응답: 400, 401

API-144 GET `/api/resources/attention`

관리 필요 자원 현황 (점검 필요/정비중/사용 불가 + 미해결 이슈 + 반납 지연)

요구사항 FR-16, FR-13 · **권한** 로그인 + 권한 resource.manage 또는 resource.allocate · **성공** 200 · **구현 위치** `routes/resources.js:95`

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "blocked": [
      {
        "id": 8,
        "code": "EQ-001",
        "name": "검정 세단",
        "typeId": 1,
        "typeCode": "VEHICLE",
        "typeName": "차량",
        "crewId": 1,
        "crewName": "Brooklyn Crew",
        "status": "NEEDS_INSPECTION",
        "location": "Brooklyn 차고",
        "minRoleLevel": 2,
        "description": "검정 세단 (VH-001)",
        "usable": false,
        "openIssues": 1
      }
    ],
    "overdueReturns": [
      {
        "allocationId": 2,
        "resourceCode": "VH-002",
        "endAt": "2026-10-18T12:00:00.000Z",
        "overdueHours": 3
      }
    ]
  }
}
```


오류 응답: 401, 403

API-145 GET `/api/resources/:id`

자원 상세 (현재 사용 현황, 다음 예약, 미해결 이슈)

요구사항 FR-13, DR-03 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/resources.js:117`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	자원 ID · 예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "code": "VH-001",
    "name": "검정 세단",
    "typeId": 1,
    "typeCode": "VEHICLE",
    "typeName": "차량",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "status": "AVAILABLE",
    "location": "Brooklyn 차고",
    "minRoleLevel": 2,
    "description": "검정 세단 (VH-001)",
    "usable": true,
    "currentUse": null,
    "upcoming": [
      {
        "allocationId": 1,
        "taskId": 1,
        "startAt": "2026-10-19T09:00:00.000Z",
        "endAt": "2026-10-19T12:00:00.000Z",
        "status": "RESERVED"
      }
    ],
    "openIssues": []
  }
}
```


오류 응답: 400, 401, 403, 404

API-146 POST /api/resources

자원 등록

요구사항 FR-13, FR-20 · **권한** 로그인 + 권한 resource.manage · **성공** 201 · **구현 위치** `routes/resources.js:133`

Body 필드	타입	필수	설명 / 제약
`code`	string	Y	최대 30자
`name`	string	Y	최대 100자
`typeCode`	string	Y	
`crewId`	integer		소유 Crew (null 이면 조직 공용) · null 허용
`location`	string		최대 100자
`minRoleLevel`	integer		이용 가능한 최소 역할 등급 (미지정 시 유형 기본값) · ≥1, ≤6
`description`	string		최대 500자

Request 예시

Response 예시 (201)

```
{
  "code": "VH-005",
  "name": "소형 승합차",
  "typeCode": "VEHICLE",
  "crewId": 1,
  "location": "Brooklyn 차고",
  "minRoleLevel": 2
}
```

```
{
  "success": true,
  "data": {
    "id": 10,
    "code": "VH-005",
    "name": "소형 승합차",
    "typeId": 1,
    "typeCode": "VEHICLE",
    "typeName": "차량",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "status": "AVAILABLE",
    "location": "Brooklyn 차고",
    "minRoleLevel": 2,
    "description": "검정 세단 (VH-001)",
    "usable": true
  }
}
```

오류 응답: 400, 401, 403, 404, 409

API-147 PATCH `/api/resources/:id`

자원 정보 수정 (이름/위치/소유 Crew/이용 등급/설명)

요구사항 FR-13 · **권한** 로그인 + 권한 resource.manage · **성공** 200 · **구현 위치** `routes/resources.js:160`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	자원 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`name`	string		최대 100자
`location`	string		최대 100자, null 허용
`crewId`	integer		null 허용
`minRoleLevel`	integer		≥1, ≤6
`description`	string		최대 500자, null 허용

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시

```
{
  "location": "Harbor 차고",
  "minRoleLevel": 3
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "code": "VH-001",
    "name": "검정 세단",
    "typeId": 1,
    "typeCode": "VEHICLE",
    "typeName": "차량",
    "crewId": 1,
    "crewName": "Brooklyn Crew",
    "status": "AVAILABLE",
    "location": "Brooklyn 차고",
    "minRoleLevel": 2,
    "description": "검정 세단 (VH-001)",
    "usable": true
  }
}
```


오류 응답: 400, 401, 403, 404

API-148 DELETE /api/resources/:id

자원 삭제 (사용 이력이 전혀 없는 경우만, 그 외는 RETIRED 상태로 관리)

요구사항 FR-13, DR-03 · **권한** 로그인 + 권한 resource.manage · **성공** 200 · **구현 위치** `routes/resources.js:184`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	자원 ID · 예: 1

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": 10,
    "deleted": true
  }
}
```


오류 응답: 400, 401, 403, 404, 422

API-149 PATCH /api/resources/:id/status

자원 상태 변경 (정상/점검 필요/정비중/사용 불가/폐기)

IN_USE 는 체크아웃/반납에 의해 자동 관리된다. 사용 불가 계열 상태로 변경하면 이후 새 배정이 차단된다(BR-08). cancelReservations=true 이면 기존 예약도 취소한다.

요구사항 FR-13, FR-15, FR-16, BR-08 · **권한** 로그인 + 권한 resource.manage · **성공** 200 · **구현 위치** `routes/resources.js:200`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	자원 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`status`	string	Y	값: AVAILABLE | NEEDS_INSPECTION | MAINTENANCE | UNAVAILABLE | RETIRED
`reason`	string	Y	최대 300자
`cancelReservations`	boolean		기본 false

Request 예시

```
{
  "status": "MAINTENANCE",
  "reason": "정기 정비"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "resourceId": 1,
    "from": "AVAILABLE",
    "to": "MAINTENANCE",
    "affectedReservations": [
      {
        "id": 1,
        "taskId": 1
      }
    ],
    "cancelledReservations": 0
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-150 GET `/api/resources/:id/history`

자원 사용·반납·상태 변경 이력 (MongoDB)

요구사항 FR-15, DR-03 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/resources.js:237`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	자원 ID · 예: 1
query	`event`	string		값: REGISTERED | UPDATED | STATUS_CHANGED | ALLOCATED | RESCHEDULED | CANCELLED | CHECKED_OUT | RETURNED | ISSUE_REPORTED | ISSUE_RESOLVED
query	`from`	datetime		
query	`to`	datetime		
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: at (내림차순 '-', 기본 -at)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": "6718...",
      "resourceId": 2,
      "event": "CHECKED_OUT",
      "from": "RESERVED",
      "to": "IN_USE",
      "by": {
        "id": 6,
        "username": "capo2",
        "role": "CAPO"
      },
      "allocationId": 2,
      "at": "2026-10-18T08:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401, 403, 404

API-151 GET `/api/resources/:id/availability`

자원의 기간 내 사용 가능/불가 구간 (예약 현황)

요구사항 FR-14, BR-07 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/resources.js:255`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	자원 ID · 예: 1
query	`from`	datetime		기본: 현재
query	`to`	datetime		기본: from + 7일

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "resourceId": 1,
    "status": "AVAILABLE",
    "usable": true,
    "busy": [
      {
        "allocationId": 1,
        "startAt": "2026-10-19T09:00:00.000Z",
        "endAt": "2026-10-19T12:00:00.000Z",
        "status": "RESERVED"
      }
    ]
  }
}
```


오류 응답: 400, 401, 403, 404

17. Resource Types

API-152 GET `/api/resource-types`

자원 유형 목록 (유형별 보유/가용 수량)

요구사항 FR-13, FR-14 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/resourceTypes.js:20`

Response 예시 (200)


```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "code": "VEHICLE",
      "name": "차량",
      "description": "업무용 차량",
      "defaultMinLevel": 2,
      "resourceCount": 4,
      "availableCount": 2
    }
  ]
}
```


오류 응답: 401

API-153 GET `/api/resource-types/:id`

자원 유형 상세

요구사항 FR-13 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/resourceTypes.js:26`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "code": "VEHICLE",
    "name": "차량",
    "description": "업무용 차량",
    "defaultMinLevel": 2,
    "resourceCount": 4,
    "availableCount": 2
  }
}
```

오류 응답: 400, 401, 404

API-154 POST /api/resource-types

자원 유형 등록

요구사항 FR-13 · **권한** 로그인 + 권한 resource.manage · **성공** 201 · **구현 위치** `routes/resourceTypes.js:34`

Body 필드	타입	필수	설명 / 제약
`code`	string	Y	최대 30자
`name`	string	Y	최대 50자
`description`	string		최대 255자
`defaultMinLevel`	integer		≥1, ≤6, 기본 2

Request 예시

```
{
  "code": "DOCUMENT",
  "name": "문서",
  "description": "기밀 문서 보관함",
  "defaultMinLevel": 3
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": 6,
    "code": "DOCUMENT",
    "name": "문서",
    "description": "업무용 차량",
    "defaultMinLevel": 3,
    "resourceCount": 0,
    "availableCount": 0
  }
}
```

오류 응답: 400, 401, 403, 409

API-155 PATCH /api/resource-types/:id

자원 유형 수정

요구사항 FR-13 · **권한** 로그인 + 권한 resource.manage · **성공** 200 · **구현 위치**

`routes/resourceTypes.js:52`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Body 필드	타입	필수	설명 / 제약
`name`	string		최대 50자
`description`	string		최대 255자, null 허용
`defaultMinLevel`	integer		≥1, ≤6

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시

```
{
  "defaultMinLevel": 2
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "code": "VEHICLE",
    "name": "차량",
    "description": "업무용 차량",
    "defaultMinLevel": 2,
    "resourceCount": 4,
    "availableCount": 2
  }
}
```

오류 응답: 400, 401, 403, 404

API-156 DELETE `/api/resource-types/:id`

자원 유형 삭제 (해당 유형 자원이 없을 때만)

요구사항 FR-13, DR-01 · **권한** 로그인 + 권한 resource.manage · **성공** 200 · **구현 위치** `routes/resourceTypes.js:71`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 6

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 6,
    "deleted": true
  }
}
```

오류 응답: 400, 401, 403, 404, 422

18. Resource Allocations

API-157 GET `/api/allocations`

자원 배정/사용 기록 조회 (자원·업무·구성원·상태·기간)

관리자는 관리 범위 전체, 일반 구성원은 본인 또는 본인이 참여한 업무의 배정만 조회된다.

요구사항 FR-15, FR-17, DR-03, FR-21 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/allocations.js:42`

위치	이름	타입	필수	설명 / 제약
query	`resourceId`	integer		
query	`taskId`	integer		
query	`memberId`	integer		
query	`status`	array<string>		값: RESERVED | IN_USE | RETURNED | CANCELLED
query	`from`	datetime		
query	`to`	datetime		
query	`overdue`	boolean		true: 반납 예정 시각이 지난 사용중 배정
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: startAt, id, createdAt (내림차순 '-', 기본 -startAt)

Response 예시 (200)


```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "resourceId": 1,
      "resourceCode": "VH-001",
      "resourceName": "검정 세단",
      "taskId": 1,
      "taskTitle": "물류 창고 정기 점검",
      "memberId": null,
      "memberName": null,
      "startAt": "2026-10-19T09:00:00.000Z",
      "endAt": "2026-10-19T12:00:00.000Z",
      "status": "RESERVED",
      "purpose": "창고 점검 이동",
      "allocatedBy": 5,
      "checkedOutAt": null,
      "returnedAt": null,
      "returnCondition": null
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

오류 응답: 400, 401

API-158 GET `/api/allocations/:id`

자원 배정 상세

요구사항 FR-15, FR-17 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/allocations.js:82`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	배정 ID · 예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "resourceId": 1,
    "resourceCode": "VH-001",
    "resourceName": "검정 세단",
    "taskId": 1,
    "taskTitle": "물류 창고 정기 점검",
    "memberId": null,
    "memberName": null,
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "status": "RESERVED",
    "purpose": "창고 점검 이동",
    "allocatedBy": 5,
    "checkedOutAt": null,
    "returnedAt": null,
    "returnCondition": null
  }
}
```

오류 응답: 400, 401, 403, 404

API-159 POST `/api/allocations`

자원을 업무 또는 구성원에게 배정 (예약)

BR-08: 사용 불가/점검 필요/정비중/폐기 자원은 배정 불가. BR-07: 동일 자원이 같은 시간대에 이미 배정되어 있으면 불가. 업무 배정 시 기간을 생략하면 업무 일정이 적용된다. 자원 행을 잠근 트랜잭션에서 검사한다.

요구사항 FR-17, BR-07, BR-08, DR-03 · **권한** 로그인 + 권한 resource.allocate · **성공** 201 · **구현 위치** `routes/allocations.js:94`

Body 필드	타입	필수	설명 / 제약
`resourceId`	integer	Y	
`taskId`	integer		업무 배정 시
`memberId`	integer		구성원 개인 배정 시
`startAt`	datetime		업무 배정 시 생략 가능
`endAt`	datetime		
`purpose`	string		최대 255자

Request 예시

```
{
  "resourceId": 1,
  "taskId": 2,
  "purpose": "거래처 미팅 이동"
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": 1,
    "resourceId": 1,
    "resourceCode": "VH-001",
    "resourceName": "검정 세단",
    "taskId": 1,
    "taskTitle": "물류 창고 정기 점검",
    "memberId": null,
    "memberName": null,
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "status": "RESERVED",
    "purpose": "창고 점검 이동",
    "allocatedBy": 5,
    "checkedOutAt": null,
    "returnedAt": null,
    "returnCondition": null
  }
}
```

오류 응답: 400, 401, 403, 404, 409 — 409 BR-07: 자원(검정 세단)은 해당 시간대에 이미 배정되어 있습니다. / 409 BR-08: 사용 불가 상태(MAINTENANCE)의 자원은 배정할 수 없습니다.

API-160 PATCH `/api/allocations/:id`

예약 기간 변경 (중복 재검사)

요구사항 FR-17, BR-07, BR-08 · **권한** 로그인 + 권한 resource.allocate · **성공** 200 · **구현 위치** `routes/allocations.js:122`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	배정 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`startAt`	datetime	Y	
`endAt`	datetime	Y	

Request 예시

```
{
  "startAt": "2026-10-19T13:00:00Z",
  "endAt": "2026-10-19T15:00:00Z"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "resourceId": 1,
    "resourceCode": "VH-001",
    "resourceName": "검정 세단",
    "taskId": 1,
    "taskTitle": "물류 창고 정기 점검",
    "memberId": null,
    "memberName": null,
    "startAt": "2026-10-19T13:00:00.000Z",
    "endAt": "2026-10-19T15:00:00.000Z",
    "status": "RESERVED",
    "purpose": "창고 점검 이동",
    "allocatedBy": 5,
    "checkedOutAt": null,
    "returnedAt": null,
    "returnCondition": null
  }
}
```


오류 응답: 400, 401, 403, 404, 409, 422

API-161 DELETE `/api/allocations/:id`

자원 예약 취소 (RESERVED 상태만)

요구사항 FR-15, FR-17 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/allocations.js:149`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	배정 ID · 예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "resourceId": 1,
    "resourceCode": "VH-001",
    "resourceName": "검정 세단",
    "taskId": 1,
    "taskTitle": "물류 창고 정기 점검",
    "memberId": null,
    "memberName": null,
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "status": "CANCELLED",
    "purpose": "창고 점검 이동",
    "allocatedBy": 5,
    "checkedOutAt": null,
    "returnedAt": null,
    "returnCondition": null
  }
}
```


오류 응답: 400, 401, 403, 404, 422

API-162 POST `/api/allocations/:id/checkout`

자원 사용 시작 (체크아웃: RESERVED → IN_USE)

요구사항 FR-15, BR-07, BR-08, DR-03 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/allocations.js:164`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	배정 ID · 예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "resourceId": 1,
    "resourceCode": "VH-001",
    "resourceName": "검정 세단",
    "taskId": 1,
    "taskTitle": "물류 창고 정기 점검",
    "memberId": null,
    "memberName": null,
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "status": "IN_USE",
    "purpose": "창고 점검 이동",
    "allocatedBy": 5,
    "checkedOutAt": "2026-10-19T09:00:00.000Z",
    "returnedAt": null,
    "returnCondition": null
  }
}
```


오류 응답: 400, 401, 403, 404, 409, 422

API-163 POST /api/allocations/:id/return

자원 반납 (상태 기록, 파손 시 점검 필요 / 분실 시 사용 불가 자동 전환)

요구사항 FR-15, FR-16, DR-03 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/allocations.js:189`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	배정 ID · 예: 1

Body 필드	타입	필수	설명 / 제약
`condition`	string	Y	값: GOOD | DAMAGED | LOST
`note`	string		최대 500자

Request 예시


```
{
  "condition": "GOOD",
  "note": "이상 없음"
}
```


Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "resourceId": 1,
    "resourceCode": "VH-001",
    "resourceName": "검정 세단",
    "taskId": 1,
    "taskTitle": "물류 창고 정기 점검",
    "memberId": null,
    "memberName": null,
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "status": "RETURNED",
    "purpose": "창고 점검 이동",
    "allocatedBy": 5,
    "checkedOutAt": null,
    "returnedAt": null,
    "returnCondition": "GOOD",
    "resourceStatus": "AVAILABLE"
  }
}
```


오류 응답: 400, 401, 403, 404, 422

19. Resource Requests

API-164 POST /api/resource-requests

자원 사용 요청 (권한 등급·상태·시간 중복 사전 검사)

요구사항 FR-14, FR-17, BR-07, BR-08 · **권한** 로그인 사용자 · **성공** 201 · **구현 위치** `routes/resourceRequests.js:34`

Body 필드	타입	필수	설명 / 제약
`resourceId`	integer	Y	
`startAt`	datetime	Y	
`endAt`	datetime	Y	
`taskId`	integer		관련 업무 (본인 참여 업무)
`purpose`	string	Y	최대 255자

Request 예시


```
{
  "resourceId": 9,
  "startAt": "2026-10-22T09:00:00Z",
  "endAt": "2026-10-22T12:00:00Z",
  "purpose": "거래처 방문"
}
```


Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": 1,
    "resourceId": 5,
    "resourceCode": "CM-001",
    "resourceName": "무전기 세트 1",
    "requesterId": 9,
    "requesterName": "Joey Marino",
    "taskId": null,
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "purpose": "창고 점검 시 연락용",
    "status": "PENDING",
    "reviewedBy": null,
    "reviewComment": null,
    "allocationId": null
  }
}
```


오류 응답: 400, 401, 403, 404, 409

API-165 GET `/api/resource-requests`

자원 사용 요청 목록 (내 요청 또는 심사 가능한 요청)

요구사항 FR-14, FR-17, FR-21 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/resourceRequests.js:63`

위치	이름	타입	필수	설명 / 제약
query	`status`	string		값: PENDING | APPROVED | REJECTED | CANCELLED
query	`resourceId`	integer		
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: createdAt, startAt (내림차순 '-', 기본 -createdAt)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "resourceId": 5,
      "resourceCode": "CM-001",
      "resourceName": "무전기 세트 1",
      "requesterId": 9,
      "requesterName": "Joey Marino",
      "taskId": null,
      "startAt": "2026-10-19T09:00:00.000Z",
      "endAt": "2026-10-19T12:00:00.000Z",
      "purpose": "창고 점검 시 연락용",
      "status": "PENDING",
      "reviewedBy": null,
      "reviewComment": null,
      "allocationId": null
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401

API-166 GET `/api/resource-requests/:id`

자원 사용 요청 상세

요구사항 FR-14 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/resourceRequests.js:87`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "resourceId": 5,
    "resourceCode": "CM-001",
    "resourceName": "무전기 세트 1",
    "requesterId": 9,
    "requesterName": "Joey Marino",
    "taskId": null,
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "purpose": "창고 점검 시 연락용",
    "status": "PENDING",
    "reviewedBy": null,
    "reviewComment": null,
    "allocationId": null
  }
}
```


오류 응답: 400, 401, 403, 404

API-167 POST `/api/resource-requests/:id/approve`

자원 사용 요청 승인 (배정 생성 시 BR-07/BR-08 재검사)

요구사항 FR-17, BR-07, BR-08 · **권한** 로그인 + 권한 resource.allocate · **성공** 200 · **구현 위치** `routes/resourceRequests.js:99`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Body 필드	타입	필수	설명 / 제약
`comment`	string		최대 500자

Request 예시


```
{
  "comment": "승인"
}
```


Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "resourceId": 5,
    "resourceCode": "CM-001",
    "resourceName": "무전기 세트 1",
    "requesterId": 9,
    "requesterName": "Joey Marino",
    "taskId": null,
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "purpose": "창고 점검 시 연락용",
    "status": "APPROVED",
    "reviewedBy": 5,
    "reviewComment": null,
    "allocationId": 4
  }
}
```


오류 응답: 400, 401, 403, 404, 409, 422

API-168 POST `/api/resource-requests/:id/reject`

자원 사용 요청 반려 (사유 필수)

요구사항 FR-17 · **권한** 로그인 + 권한 resource.allocate · **성공** 200 · **구현 위치** `routes/resourceRequests.js:121`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Body 필드	타입	필수	설명 / 제약
`comment`	string	Y	최대 500자

Request 예시

```
{
  "comment": "해당 시간 다른 업무 우선"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "resourceId": 5,
    "resourceCode": "CM-001",
    "resourceName": "무전기 세트 1",
    "requesterId": 9,
    "requesterName": "Joey Marino",
    "taskId": null,
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "purpose": "창고 점검 시 연락용",
    "status": "REJECTED",
    "reviewedBy": null,
    "reviewComment": "해당 시간 다른 업무 우
선",
    "allocationId": null
  }
}
```


오류 응답: 400, 401, 403, 404, 422

API-169 DELETE /api/resource-requests/:id

내 자원 사용 요청 취소 (대기 상태만)

요구사항 FR-14 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/resourceRequests.js:135`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": 1,
    "resourceId": 5,
    "resourceCode": "CM-001",
    "resourceName": "무전기 세트 1",
    "requesterId": 9,
    "requesterName": "Joey Marino",
    "taskId": null,
    "startAt": "2026-10-19T09:00:00.000Z",
    "endAt": "2026-10-19T12:00:00.000Z",
    "purpose": "창고 점검 시 연락용",
    "status": "CANCELLED",
    "reviewedBy": null,
    "reviewComment": null,
    "allocationId": null
  }
}
```

오류 응답: 400, 401, 403, 404, 422

20. Resource Issues

API-170 POST /api/resources/:id/issues

자원 이상 신고 (정상 자원은 점검 필요 상태로 자동 전환)

요구사항 FR-16, FR-15, BR-08 · **권한** 로그인 사용자 · **성공** 201 · **구현 위치** `routes/resourceIssues.js:25`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Body 필드	타입	필수	설명 / 제약
`severity`	string		값: LOW | MEDIUM | HIGH, 기본 "MEDIUM"
`description`	string	Y	최대 500자

Request 예시

```
{
  "severity": "MEDIUM",
  "description": "타이어 공기압 경고등 점등"
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": 2,
    "resourceId": 1,
    "resourceCode": "VH-001",
    "resourceName": "금고 개폐 장비",
    "resourceStatus": "NEEDS_INSPECTION",
    "reportedBy": 8,
    "reporterName": "Paulie Conti",
    "severity": "MEDIUM",
    "description": "다이얼 잠금 장치 오작동",
    "status": "OPEN",
    "resolvedBy": null,
    "resolution": null,
    "resolvedAt": null,
    "createdAt": "2026-10-
18T00:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403, 404

API-171 GET /api/resource-issues

자원 이슈 목록 (상태·심각도·자원)

요구사항 FR-16, FR-15 · **권한** 로그인 + 권한 resource.manage 또는 resource.allocate · **성공** 200 · **구현 위치** `routes/resourceIssues.js:52`

위치	이름	타입	필수	설명 / 제약
query	`status`	string		값: OPEN | RESOLVED
query	`severity`	string		값: LOW | MEDIUM | HIGH
query	`resourceId`	integer		

query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: createdAt, severity (내림차순 '-', 기본 -createdAt)

Response 예시 (200)


```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "resourceId": 8,
      "resourceCode": "EQ-001",
      "resourceName": "금고 개폐 장비",
      "resourceStatus": "NEEDS_INSPECTION",
      "reportedBy": 8,
      "reporterName": "Paulie Conti",
      "severity": "HIGH",
      "description": "다이얼 잠금 장치 오작동",
      "status": "OPEN",
      "resolvedBy": null,
      "resolution": null,
      "resolvedAt": null,
      "createdAt": "2026-10-18T00:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401, 403

API-172 GET `/api/resource-issues/:id`

자원 이슈 상세

요구사항 FR-16 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/resourceIssues.js:71`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "resourceId": 8,
    "resourceCode": "EQ-001",
    "resourceName": "금고 개폐 장비",
    "resourceStatus": "NEEDS_INSPECTION",
    "reportedBy": 8,
    "reporterName": "Paulie Conti",
    "severity": "HIGH",
    "description": "다이얼 잠금 장치 오작동",
    "status": "OPEN",
    "resolvedBy": null,
    "resolution": null,
    "resolvedAt": null,
    "createdAt": "2026-10-18T00:00:00.000Z"
  }
}
```


오류 응답: 400, 401, 403, 404

API-173 POST `/api/resource-issues/:id/resolve`

자원 이슈 해결 처리 (다른 미해결 이슈가 없으면 자원 상태 복구)

요구사항 FR-16, FR-15, FR-13 · **권한** 로그인 + 권한 resource.manage · **성공** 200 · **구현 위치** `routes/resourceIssues.js:83`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	예: 1

Body 필드	타입	필수	설명 / 제약
`resolution`	string	Y	최대 500자
`restoreStatus`	string		해결 후 자원 상태 · 값: AVAILABLE | MAINTENANCE | UNAVAILABLE | RETIRED, 기본 "AVAILABLE"

Request 예시


```
{
  "resolution": "다이얼 교체 완료"
}
```


Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 1,
    "resourceId": 8,
    "resourceCode": "EQ-001",
    "resourceName": "금고 개폐 장비",
    "resourceStatus": "AVAILABLE",
    "reportedBy": 8,
    "reporterName": "Paulie Conti",
    "severity": "HIGH",
    "description": "다이얼 잠금 장치 오작동",
    "status": "RESOLVED",
    "resolvedBy": null,
    "resolution": "다이얼 교체 완료",
    "resolvedAt": null,
    "createdAt": "2026-10-
18T00:00:00.000Z"
  }
}
```


오류 응답: 400, 401, 403, 404, 422

21. Evaluations & Reputation

API-174 GET `/api/evaluations`

평가 이력 조회 (구성원·업무·평가자·기간)

evaluation.read 권한자는 관리 범위 구성원의 평가를, 일반 구성원은 본인 평가만(평가자 익명) 조회한다.

요구사항 FR-19, FR-18, DR-04 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/evaluations.js:59`

위치	이름	타입	필수	설명 / 제약
query	`memberId`	integer		예: 8
query	`taskId`	integer		
query	`evaluatorId`	integer		
query	`status`	string		값: ACTIVE | VOIDED
query	`from`	datetime		
query	`to`	datetime		
query	`minOverall`	number		≥1, ≤5
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: createdAt, overall (내림차순 '-', 기본 -createdAt)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": "6719...",
      "memberId": 8,
      "taskId": 4,
      "evaluator": {
        "id": 5,
        "username": "capo1",
        "role": "CAPO"
      },
      "scores": {
        "loyalty": 5,
        "skill": 4,
        "discretion": 4,
        "teamwork": 5
      },
      "overall": 4.5,
      "comment": "꼼꼼한 장부 검토",
      "status": "ACTIVE",
      "revisions": [],
      "createdAt": "2026-10-12T00:00:00.000Z",
      "memberName": "Paulie Conti"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401

API-175 GET `/api/evaluations/pending`

평가 대기 목록 (관리 범위의 최근 완료 업무 참여자 중 미평가)

요구사항 FR-18, FR-11 · **권한** 로그인 + 권한 evaluation.write · **성공** 200 · **구현 위치** `routes/evaluations.js:101`

위치	이름	타입	필수	설명 / 제약
query	`days`	integer		최근 N일 내 완료 업무 · ≥1, ≤365, 기본 30

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "taskId": 3,
      "title": "항만 화물 인수",
      "completedAt": "2026-10-18T12:00:00.000Z",
      "memberId": 10,
      "memberName": "Frankie Costa",
      "attendance": "PRESENT",
      "contribution": 90
    }
  ]
}
```


오류 응답: 400, 401, 403

API-176 GET `/api/evaluations/:id`

평가 상세

요구사항 FR-19 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/evaluations.js:121`

위치	이름	타입	필수	설명 / 제약
path	`id`	objectId	Y	예: "6719aa000000000000000001"

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": "6719...",
    "memberId": 8,
    "taskId": 4,
    "evaluator": {
      "id": 5,
      "username": "capo1",
      "role": "CAPO"
    },
    "scores": {
      "loyalty": 5,
      "skill": 4,
      "discretion": 4,
      "teamwork": 5
    },
    "overall": 4.5,
    "comment": "꼼꼼한 장부 검토",
    "status": "ACTIVE",
    "revisions": [],
    "createdAt": "2026-10-12T00:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403, 404

API-177 POST /api/evaluations

평가 등록 (업무 결과 기반, 항목별 1~5점, 가중 평균 자동 계산)

업무를 지정하면 완료된 업무의 참여자만 평가할 수 있고, 같은 업무·구성원에 대한 유효 평가는 1건만 허용된다. 평가 등록은 구성원 이력(BR-10)에 기록된다.

요구사항 FR-18, FR-11, BR-10, DR-04 · **권한** 로그인 + 권한 evaluation.write · **성공** 201 · **구현 위치** `routes/evaluations.js:129`

Body 필드	타입	필수	설명 / 제약
`memberId`	integer	Y	
`taskId`	integer		평가 근거 업무 (완료된 업무)
`scores`	object	Y	평가 항목 코드별 1~5점 (GET /api/evaluation-criteria 참고)
`comment`	string		최대 1000자

Request 예시

```
{
  "memberId": 10,
  "taskId": 3,
  "scores": {
    "loyalty": 4,
    "skill": 5,
    "discretion": 4,
    "teamwork": 4
  },
  "comment": "화물 인수 책임 완수"
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": "6719...",
    "memberId": 10,
    "taskId": 3,
    "evaluator": {
      "id": 5,
      "username": "capo1",
      "role": "CAPO"
    },
    "scores": {
      "loyalty": 5,
      "skill": 4,
      "discretion": 4,
      "teamwork": 5
    },
    "overall": 4.5,
    "comment": "꼼꼼한 장부 검토",
    "status": "ACTIVE",
    "revisions": [],
    "createdAt": "2026-10-
12T00:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403, 404, 409, 422

API-178 PATCH /api/evaluations/:id

평가 수정 (작성자: 설정된 기간 내 / Underboss 이상: 언제나, 수정 전 내용 보존)

요구사항 FR-18, BR-10 · **권한** 로그인 + 권한 evaluation.write · **성공** 200 · **구현 위치** `routes/evaluations.js:164`

위치	이름	타입	필수	설명 / 제약

path	`id`	objectId	Y	예: "6719aa000000000000000001"

Body 필드	타입	필수	설명 / 제약
`scores`	object		
`comment`	string		최대 1000자

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시

```
{
  "scores": {
    "loyalty": 5,
    "skill": 4,
    "discretion": 4,
    "teamwork": 5
  },
  "comment": "재검토 후 상향"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": "6719...",
    "memberId": 8,
    "taskId": 4,
    "evaluator": {
      "id": 5,
      "username": "capo1",
      "role": "CAPO"
    },
    "scores": {
      "loyalty": 5,
      "skill": 4,
      "discretion": 4,
      "teamwork": 5
    },
    "overall": 4.5,
    "comment": "꼼꼼한 장부 검토",
    "status": "ACTIVE",
    "revisions": [
      {
        "scores": {
          "loyalty": 4
        },
        "overall": 4.25,
        "comment": "...",
        "revisedAt": "2026-10-
18T00:00:00.000Z",
        "revisedBy": "capo1"
      }
    ],
    "createdAt": "2026-10-
12T00:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-179 DELETE `/api/evaluations/:id`

평가 무효화 (삭제하지 않고 VOIDED 처리, 사유 기록)

요구사항 FR-18, BR-10, FR-20 · **권한** 로그인 + 권한 evaluation.void · **성공** 200 · **구현 위치**

`routes/evaluations.js:195`

위치	이름	타입	필수	설명 / 제약
path	`id`	objectId	Y	예: "6719aa000000000000000001"
query	`reason`	string	Y	최대 300자 · 예: "평가 근거 오류"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": "6719...",
    "memberId": 8,
    "taskId": 4,
    "evaluator": {
      "id": 5,
      "username": "capo1",
      "role": "CAPO"
    },
    "scores": {
      "loyalty": 5,
      "skill": 4,
      "discretion": 4,
      "teamwork": 5
    },
    "overall": 4.5,
    "comment": "꼼꼼한 장부 검토",
    "status": "VOIDED",
    "revisions": [],
    "createdAt": "2026-10-12T00:00:00.000Z",
    "voidReason": "평가 근거 오류"
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-180 GET `/api/evaluations/:id/revisions`

평가 수정 이력

요구사항 FR-19, BR-10 · **권한** 로그인 + 권한 evaluation.read · **성공** 200 · **구현 위치** `routes/evaluations.js:212`

위치	이름	타입	필수	설명 / 제약
path	`id`	objectId	Y	예: "6719aa000000000000000001"

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "scores": {
        "loyalty": 4
      },
      "overall": 4.25,
      "comment": "...",
      "revisedAt": "2026-10-18T00:00:00.000Z",
      "revisedBy": "capo1"
    }
  ]
}
```

오류 응답: 400, 401, 403, 404

API-181 GET `/api/evaluation-criteria`

평가 항목 목록 (코드, 가중치)

요구사항 FR-18 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/evaluations.js:228`

위치	이름	타입	필수	설명 / 제약
query	`includeInactive`	boolean		기본 false

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": "671a...",
      "code": "loyalty",
      "name": "충성도",
      "description": "조직과 Crew에 대한 헌신",
      "weight": 1.5,
      "active": true
    }
  ]
}
```

오류 응답: 400, 401

API-182 POST `/api/evaluation-criteria`

평가 항목 추가

요구사항 FR-18, DR-04 · **권한** 로그인 + 권한 evaluation.criteria · **성공** 201 · **구현 위치** `routes/evaluations.js:235`

Body 필드	타입	필수	설명 / 제약
`code`	string	Y	최대 30자
`name`	string	Y	최대 50자

`description`	string		최대 255자
`weight`	number		≥0.1, ≤5, 기본 1

Request 예시

```
{
  "code": "punctuality",
  "name": "시간 엄수",
  "description": "집결 시간 준수",
  "weight": 0.7
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "id": "671a...",
    "code": "punctuality",
    "name": "시간 엄수",
    "description": "조직과 Crew에 대한 헌신",
    "weight": 0.7,
    "active": true
  }
}
```

오류 응답: 400, 401, 403, 409

API-183 PATCH `/api/evaluation-criteria/:code`

평가 항목 수정 (이름/설명/가중치/활성 여부)

요구사항 FR-18 · **권한** 로그인 + 권한 evaluation.criteria · **성공** 200 · **구현 위치** `routes/evaluations.js:254`

위치	이름	타입	필수	설명 / 제약
path	`code`	string	Y	예: "teamwork"

Body 필드	타입	필수	설명 / 제약
`name`	string		최대 50자
`description`	string		최대 255자
`weight`	number		≥0.1, ≤5
`active`	boolean		

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시

```
{
  "weight": 1,
  "active": true
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": "671a...",
    "code": "loyalty",
    "name": "충성도",
    "description": "조직과 Crew에 대한 헌신",
    "weight": 1.5,
    "active": true
  }
}
```

오류 응답: 400, 401, 403, 404

API-184 DELETE `/api/evaluation-criteria/:code`

평가 항목 삭제 (평가에 사용된 적 없는 항목만, 그 외는 비활성화)

요구사항 FR-18, DR-04 · **권한** 로그인 + 권한 evaluation.criteria · **성공** 200 · **구현 위치** `routes/evaluations.js:271`

위치	이름	타입	필수	설명 / 제약
path	`code`	string	Y	예: "punctuality"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "code": "punctuality",
    "deleted": true
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-185 GET `/api/reputation/leaderboard`

평판 순위 (조회 가능한 Crew 구성원 대상)

요구사항 FR-18, FR-19, FR-22 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/evaluations.js:289`

위치	이름	타입	필수	설명 / 제약
query	`crewId`	integer		
query	`limit`	integer		≥1, ≤100, 기본 10

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "rank": 1,
      "memberId": 13,
      "name": "Carlo Rizzi",
      "crewName": "Uptown Crew",
      "score": 92,
      "evaluationAverage": 3.9,
      "ledgerPoints": 0,
      "attendanceRate": 94
    }
  ]
}
```

오류 응답: 400, 401

22. Activities

API-186 GET `/api/activities`

활동 기록 조회 (본인 또는 관리 범위 구성원)

요구사항 FR-19, FR-02, DR-04, NFR-04 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/activities.js:14`

위치	이름	타입	필수	설명 / 제약
query	`memberId`	integer		
query	`type`	string		값: FIELD_REPORT | TASK_WORK | PATROL | MEETING | TRAINING | TASK_ACCEPTED | TASK_COMPLETED | RESOURCE_CHECKOUT
query	`from`	datetime		
query	`to`	datetime		
query	`taskId`	integer		
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: at (내림차순 '-', 기본 -at)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": "671b...",
      "memberId": 8,
      "type": "FIELD_REPORT",
      "description": "창고 주변 특이사항 없음",
      "refs": {
        "taskId": 1
      },
      "location": "Brooklyn 창고",
      "details": {
        "checkpoints": 3
      },
      "source": "MEMBER",
      "at": "2026-10-18T09:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401

API-187 GET `/api/activities/:id`

활동 기록 상세

요구사항 FR-19 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/activities.js:44`

위치	이름	타입	필수	설명 / 제약
path	`id`	objectId	Y	예: "671baa000000000000000001"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": "671b...",
    "memberId": 8,
    "type": "FIELD_REPORT",
    "description": "창고 주변 특이사항 없음",
    "refs": {
      "taskId": 1
    },
    "location": "Brooklyn 창고",
    "details": {
      "checkpoints": 3
    },
    "source": "MEMBER",
    "at": "2026-10-18T09:00:00.000Z"
  }
}
```


오류 응답: 400, 401, 403, 404

API-188 POST /api/activities

내 활동 보고 등록 (현장 보고·순찰·훈련 등, 유연한 상세 정보)

요구사항 FR-18, DR-04 · **권한** 로그인 사용자 · **성공** 201 · **구현 위치** `routes/activities.js:60`

Body 필드	타입	필수	설명 / 제약
`type`	string	Y	값: FIELD_REPORT | TASK_WORK | PATROL | MEETING | TRAINING
`description`	string	Y	최대 2000자
`taskId`	integer		관련 업무 (본인 참여 업무)
`location`	string		최대 100자
`details`	object		자유 형식 상세 정보 (MongoDB 유연 스키마)

Request 예시


```
{
  "type": "FIELD_REPORT",
  "description": "창고 주변 특이사항 없음",
  "taskId": 1,
  "location": "Brooklyn 창고",
  "details": {
    "checkpoints": 3,
    "incidents": 0
  }
}
```


Response 예시 (201)


```
{
  "success": true,
  "data": {
    "id": "671b...",
    "memberId": 8,
    "type": "FIELD_REPORT",
    "description": "창고 주변 특이사항 없음",
    "refs": {
      "taskId": 1
    },
    "location": "Brooklyn 창고",
    "details": {
      "checkpoints": 3
    },
    "source": "MEMBER",
    "at": "2026-10-18T09:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403

23. Search

API-189 GET `/api/search`

통합 검색 (구성원·Crew·업무·자원, 권한 범위 내)

요구사항 FR-21, NFR-04, BR-09 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/search.js:40`

위치	이름	타입	필수	설명 / 제약
query	`q`	string	Y	최대 50자 · 예: "Brooklyn"
query	`types`	array<string>		검색 대상 (쉼표 구분, 기본 전체) · 값: members | crews | tasks | resources · 예: "members,tasks"
query	`limit`	integer		대상별 최대 건수 · ≥1, ≤50, 기본 10

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "query": "Brooklyn",
    "counts": {
      "members": 0,
      "crews": 1,
      "tasks": 2,
      "resources": 0
    },
    "results": {
      "crews": [
        {
          "id": 1,
          "name": "Brooklyn Crew",
          "status": "ACTIVE",
          "regionName": "Brooklyn"
        }
      ]
    }
  }
}
```

오류 응답: 400, 401

API-190 GET `/api/search/suggest`

검색어 자동완성 (이름 접두어 일치, 최대 10건)

요구사항 FR-21, NFR-04 · **권한** 로그인 사용자 · **성공** 200 · **구현 위치** `routes/search.js:54`

위치	이름	타입	필수	설명 / 제약
query	`q`	string	Y	최대 30자 · 예: "Pa"

Response 예시 (200)


```
{
  "success": true,
  "data": [
    {
      "type": "member",
      "id": 8,
      "label": "Paulie Conti (Paulie)"
    },
    {
      "type": "task",
      "id": 4,
      "label": "Paulie 장부 정리"
    }
  ]
}
```


오류 응답: 400, 401

24. Statistics

API-191 GET `/api/stats/overview`

운영 현황 요약 (KPI 대시보드)

요구사항 FR-22 · **권한** 로그인 + 권한 stats.read · **성공** 200 · **구현 위치** `routes/stats.js:35`

위치	이름	타입	필수	설명 / 제약
query	`crewId`	integer		특정 Crew 로 한정

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "members": {
      "total": 18,
      "active": 14,
      "suspended": 1
    },
    "crews": 3,
    "tasks": {
      "open": 5,
      "overdue": 0,
      "inProgress": 1,
      "completedLast30Days": 3
    },
    "resources": {
      "total": 9,
      "available": 5,
      "blocked": 3,
      "inUse": 1
    },
    "openIssues": 1,
    "pendingRequests": {
      "resource": 1,
      "promotion": 1,
      "join": 1
    }
  }
}
```


오류 응답: 400, 401, 403

API-192 GET `/api/stats/members`

구성원 현황 (상태별·역할별·Crew별 분포)

요구사항 FR-22, FR-04 · **권한** 로그인 + 권한 stats.read · **성공** 200 · **구현 위치** `routes/stats.js:74`

위치	이름	타입	필수	설명 / 제약
query	`crewId`	integer		특정 Crew 로 한정

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "byStatus": {
      "ACTIVE": 14,
      "STANDBY": 1,
      "SUSPENDED": 1
    },
    "byRole": {
      "SOLDIER": 8,
      "CAPO": 3
    },
    "byCrew": [
      {
        "crewId": 1,
        "crewName": "Brooklyn Crew",
        "ACTIVE": 4,
        "SUSPENDED": 1,
        "total": 5
      }
    ],
    "unaffiliated": 4
  }
}
```


오류 응답: 400, 401, 403

API-193 GET `/api/stats/crews`

Crew 별 운영 비교 (인원, 업무 완료율, 성공률, 평균 기여도, 성과값)

요구사항 FR-22, FR-11 · **권한** 로그인 + 권한 stats.read · **성공** 200 · **구현 위치** `routes/stats.js:100`

위치	이름	타입	필수	설명 / 제약
query	`from`	datetime		기간 시작 (기본: 90일 전) · 예: "2026-07-01T00:00:00Z"
query	`to`	datetime		기간 종료 (기본: 현재) · 예: "2026-12-31T00:00:00Z"

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "crewId": 1,
      "crewName": "Brooklyn Crew",
      "members": 5,
      "tasksTotal": 9,
      "completed": 6,
      "cancelled": 1,
      "completionRate": 85.7,
      "successRate": 66.7,
      "avgContribution": 72.3,
      "totalValue": 45500
    }
  ]
}
```


오류 응답: 400, 401, 403

API-194 GET /api/stats/tasks

업무 현황 (기간 내 상태별·우선순위별·결과별 분포)

요구사항 FR-22, FR-12, FR-11 · **권한** 로그인 + 권한 stats.read · **성공** 200 · **구현 위치** `routes/stats.js:131`

위치	이름	타입	필수	설명 / 제약
query	`from`	datetime		기간 시작 (기본: 90일 전) · 예: "2026-07-01T00:00:00Z"
query	`to`	datetime		기간 종료 (기본: 현재) · 예: "2026-12-31T00:00:00Z"
query	`crewId`	integer		특정 Crew 로 한정

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "period": {
      "from": "2026-07-20T00:00:00.000Z",
      "to": "2026-10-18T00:00:00.000Z"
    },
    "total": 24,
    "byStatus": {
      "COMPLETED": 17,
      "PLANNED": 4
    },
    "byPriority": {
      "HIGH": 6,
      "NORMAL": 12
    },
    "byOutcome": {
      "SUCCESS": 11,
      "PARTIAL": 4,
      "FAILURE": 2
    },
    "avgDurationHours": 4.1
  }
}
```


오류 응답: 400, 401, 403

API-195 GET `/api/stats/tasks/trend`

업무 추이 (일/주/월 단위 생성·완료 건수 시계열)

요구사항 FR-22 · **권한** 로그인 + 권한 stats.read · **성공** 200 · **구현 위치** `routes/stats.js:154`

위치	이름	타입	필수	설명 / 제약
query	`from`	datetime		기간 시작 (기본: 90일 전) · 예: "2026-07-01T00:00:00Z"
query	`to`	datetime		기간 종료 (기본: 현재) · 예: "2026-12-31T00:00:00Z"
query	`crewId`	integer		특정 Crew 로 한정
query	`interval`	string		값: day | week | month, 기본 "week"

Response 예시 (200)


```
{
  "success": true,
  "data": [
    {
      "bucket": "2026-W40",
      "created": 3,
      "completed": 2
    }
  ]
}
```

오류 응답: 400, 401, 403

API-196 GET `/api/stats/members/workload`

구성원별 업무량 (기간 내 배정 건수·배정 시간·진행중 업무)

요구사항 FR-22, FR-08, BR-03 · **권한** 로그인 + 권한 stats.read · **성공** 200 · **구현 위치** `routes/stats.js:174`

위치	이름	타입	필수	설명 / 제약
query	`from`	datetime		기간 시작 (기본: 90일 전) · 예: "2026-07-01T00:00:00Z"
query	`to`	datetime		기간 종료 (기본: 현재) · 예: "2026-12-31T00:00:00Z"
query	`crewId`	integer		특정 Crew 로 한정
query	`limit`	integer		≥1, ≤100, 기본 20

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "memberId": 8,
      "name": "Paulie Conti",
      "crewName": "Brooklyn Crew",
      "assignments": 7,
      "hours": 31,
      "openAssignments": 1
    }
  ]
}
```

오류 응답: 400, 401, 403

API-197 GET `/api/stats/resources/utilization`

자원 가동률 (기간 대비 배정·사용 시간 비율)

요구사항 FR-22, FR-15 · **권한** 로그인 + 권한 stats.read · **성공** 200 · **구현 위치** `routes/stats.js:196`

위치	이름	타입	필수	설명 / 제약
query	`from`	datetime		기간 시작 (기본: 90일 전) · 예: "2026-07-01T00:00:00Z"
query	`to`	datetime		기간 종료 (기본: 현재) · 예: "2026-12-31T00:00:00Z"
query	`type`	string		예: "VEHICLE"

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "resourceId": 1,
      "code": "VH-001",
      "name": "검정 세단",
      "status": "AVAILABLE",
      "allocations": 3,
      "allocatedHours": 9,
      "utilizationRate": 0.4
    }
  ]
}
```


오류 응답: 400, 401, 403

API-198 GET /api/stats/resources/issues

자원 이슈 통계 (심각도·상태·유형별, 평균 해결 시간)

요구사항 FR-22, FR-16 · **권한** 로그인 + 권한 stats.read · **성공** 200 · **구현 위치** `routes/stats.js:215`

위치	이름	타입	필수	설명 / 제약
query	`from`	datetime		기간 시작 (기본: 90일 전) · 예: "2026-07-01T00:00:00Z"
query	`to`	datetime		기간 종료 (기본: 현재) · 예: "2026-12-31T00:00:00Z"

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "bySeverity": {
      "HIGH": 2,
      "MEDIUM": 1
    },
    "byStatus": {
      "OPEN": 1,
      "RESOLVED": 2
    },
    "byType": {
      "VEHICLE": 2
    },
    "avgResolveHours": 26.5,
    "blockedNow": {
      "NEEDS_INSPECTION": 1,
      "MAINTENANCE": 1,
      "UNAVAILABLE": 1
    }
  }
}
```

오류 응답: 400, 401, 403

API-199 GET `/api/stats/evaluations`

평가 통계 (항목별 평균, 종합 점수 분포, 월별 평균)

요구사항 FR-22, FR-18, FR-19 · **권한** 로그인 + 권한 stats.read · **성공** 200 · **구현 위치** `routes/stats.js:233`

위치	이름	타입	필수	설명 / 제약
query	`from`	datetime		기간 시작 (기본: 90일 전) · 예: "2026-07-01T00:00:00Z"
query	`to`	datetime		기간 종료 (기본: 현재) · 예: "2026-12-31T00:00:00Z"
query	`crewId`	integer		특정 Crew 로 한정

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "count": 4,
    "avgOverall": 3.9,
    "byCriterion": {
      "loyalty": 4,
      "skill": 3.75
    },
    "distribution": {
      "1": 0,
      "2": 0,
      "3": 2,
      "4": 2,
      "5": 0
    },
    "monthly": [
      {
        "month": "2026-10",
        "avg": 3.9,
        "count": 4
      }
    ]
  }
}
```

오류 응답: 400, 401, 403

API-200 GET `/api/stats/activities/heatmap`

활동 히트맵 (요일 × 시간대 활동 건수, MongoDB 집계)

요구사항 FR-22, DR-04 · **권한** 로그인 + 권한 stats.read · **성공** 200 · **구현 위치** `routes/stats.js:262`

위치	이름	타입	필수	설명 / 제약
query	`from`	datetime		기간 시작 (기본: 90일 전) · 예: "2026-07-01T00:00:00Z"
query	`to`	datetime		기간 종료 (기본: 현재) · 예: "2026-12-31T00:00:00Z"
query	`crewId`	integer		특정 Crew 로 한정

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "cells": [
      {
        "dayOfWeek": 2,
        "hour": 9,
        "count": 4
      }
    ],
    "total": 60,
    "byType": {
      "FIELD_REPORT": 15
    }
  }
}
```

오류 응답: 400, 401, 403

API-201 GET `/api/stats/status-changes`

구성원 변경 추이 (역할·상태·소속·평가 변경 이력의 주별 집계)

요구사항 FR-22, BR-10, FR-04 · **권한** 로그인 + 권한 stats.read · **성공** 200 · **구현 위치** `routes/stats.js:287`

위치	이름	타입	필수	설명 / 제약
query	`from`	datetime		기간 시작 (기본: 90일 전) · 예: "2026-07-01T00:00:00Z"
query	`to`	datetime		기간 종료 (기본: 현재) · 예: "2026-12-31T00:00:00Z"
query	`crewId`	integer		특정 Crew 로 한정

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "byType": {
      "STATUS": 3,
      "ROLE": 1,
      "CREW": 2
    },
    "statusTransitions": [
      {
        "from": "ACTIVE",
        "to": "SUSPENDED",
        "count": 1
      }
    ],
    "weekly": [
      {
        "week": "2026-W41",
        "count": 4
      }
    ]
  }
}
```


오류 응답: 400, 401, 403

25. Audit Logs

API-202 GET `/api/audit-logs`

감사 로그 조회 (행위자·행위·대상·결과·기간)

중요 상태 변경, 관리 활동, 권한 거부(DENIED) 시도가 모두 기록된다.

요구사항 FR-20, BR-10, BR-09, NFR-04 · **권한** 로그인 + 권한 audit.read · **성공** 200 · **구현 위치** `routes/audit.js:35`

위치	이름	타입	필수	설명 / 제약
query	`actorId`	integer		
query	`actor`	string		사용자명
query	`action`	string		예: "MEMBER_ROLE_CHANGED"
query	`entityType`	string		값: member | crew | task | resource | role | region | evaluation | criterion | resourceType | system | api | account
query	`entityId`	string		
query	`result`	string		값: SUCCESS | DENIED
query	`from`	datetime		

query	`to`	datetime		
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: at, action (내림차순 '-', 기본 -at)

Response 예시 (200)


```
{
  "success": true,
  "data": [
    {
      "id": "671c...",
      "at": "2026-10-18T09:00:00.000Z",
      "actor": {
        "id": 5,
        "username": "capo1",
        "role": "CAPO"
      },
      "action": "MEMBER_ROLE_CHANGED",
      "entity": {
        "type": "member",
        "id": "14"
      },
      "details": {
        "from": "ASSOCIATE",
        "to": "SOLDIER"
      },
      "result": "SUCCESS",
      "ip": "::1"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```


오류 응답: 400, 401, 403

API-203 GET `/api/audit-logs/summary`

감사 로그 집계 (행위별·결과별·일자별 건수, 상위 행위자)

요구사항 FR-20, FR-22 · **권한** 로그인 + 권한 audit.read · **성공** 200 · **구현 위치** `routes/audit.js:50`

위치	이름	타입	필수	설명 / 제약
query	`from`	datetime		
query	`to`	datetime		

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "total": 42,
    "byResult": {
      "SUCCESS": 38,
      "DENIED": 4
    },
    "byAction": [
      {
        "action": "TASK_CREATED",
        "count": 5
      }
    ],
    "daily": [
      {
        "date": "2026-10-18",
        "count": 12
      }
    ],
    "topActors": [
      {
        "username": "capo1",
        "count": 9
      }
    ]
  }
}
```

오류 응답: 400, 401, 403

API-204 GET `/api/audit-logs/export`

감사 로그 CSV 내보내기 (최대 10,000건)

요구사항 FR-20 · **권한** 로그인 + 권한 audit.read · **성공** 200 text/csv · **구현 위치** `routes/audit.js:74`

위치	이름	타입	필수	설명 / 제약
query	`actorId`	integer		
query	`actor`	string		사용자명
query	`action`	string		예: "MEMBER_ROLE_CHANGED"
query	`entityType`	string		값: member | crew | task | resource | role | region | evaluation | criterion | resourceType | system | api | account
query	`entityId`	string		
query	`result`	string		값: SUCCESS | DENIED

query	`from`	datetime		
query	`to`	datetime		

Response 예시 (200)

```
at,actor,role,action,entityType,entityId,result,details
2026-10-18T09:00:00.000Z,capo1,CAPO,MEMBER_ROLE_CHANGED,member,14,SUCCESS,...
```

오류 응답: 400, 401, 403

API-205 GET `/api/audit-logs/:id`

감사 로그 상세

요구사항 FR-20 · **권한** 로그인 + 권한 audit.read · **성공** 200 · **구현 위치** `routes/audit.js:88`

위치	이름	타입	필수	설명 / 제약
path	`id`	objectId	Y	예: "671caa000000000000000001"

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": "671c...",
    "at": "2026-10-18T09:00:00.000Z",
    "actor": {
      "id": 5,
      "username": "capo1",
      "role": "CAPO"
    },
    "action": "MEMBER_ROLE_CHANGED",
    "entity": {
      "type": "member",
      "id": "14"
    },
    "details": {
      "from": "ASSOCIATE",
      "to": "SOLDIER"
    },
    "result": "SUCCESS",
    "ip": "::1"
  }
}
```

오류 응답: 400, 401, 403, 404

26. Admin

API-206 GET `/api/admin/accounts`

계정 목록 (잠금 여부, 로그인 실패 횟수, 최근 로그인)

요구사항 FR-23, FR-21 · **권한** 로그인 + 권한 account.manage · **성공** 200 · **구현 위치** `routes/admin.js:34`

위치	이름	타입	필수	설명 / 제약
query	`q`	string		아이디/이름 검색
query	`locked`	boolean		
query	`role`	string		값: ASSOCIATE | SOLDIER | CAPO | UNDERBOSS | BOSS | ADMINISTRATOR
query	`inactiveDays`	integer		N일 이상 로그인하지 않은 계정 · ≥1
query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: id, username, lastLoginAt, failedLogins (내림차순 '-', 기본 id)

Response 예시 (200)


```
{
  "success": true,
  "data": [
    {
      "id": 9,
      "username": "soldier2",
      "name": "Joey Marino",
      "role": "SOLDIER",
      "status": "ACTIVE",
      "isLocked": false,
      "lockedReason": null,
      "failedLogins": 0,
      "lastLoginAt": "2026-10-18T09:00:00.000Z",
      "tokenVersion": 0
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

오류 응답: 400, 401, 403

API-207 POST `/api/admin/accounts/:id/lock`

계정 잠금 (로그인 차단 + 기존 세션 즉시 무효)

요구사항 FR-23, FR-20, NFR-03 · **권한** 로그인 + 권한 account.manage · **성공** 200 · **구현 위치** `routes/admin.js:62`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원(계정) ID · 예: 9

Body 필드	타입	필수	설명 / 제약
`reason`	string	Y	최대 255자

Request 예시

```
{
  "reason": "보안 사고 조사"
}
```

Response 예시 **(200)**

```
{
  "success": true,
  "data": {
    "id": 9,
    "username": "soldier2",
    "name": "Joey Marino",
    "role": "SOLDIER",
    "status": "ACTIVE",
    "isLocked": true,
    "lockedReason": "보안 사고 조사",
    "failedLogins": 0,
    "lastLoginAt": "2026-10-
18T09:00:00.000Z",
    "tokenVersion": 1
  }
}
```

오류 응답: 400, 401, 403, 404, 422

API-208 POST `/api/admin/accounts/:id/unlock`

계정 잠금 해제 (로그인 실패 횟수 초기화)

요구사항 FR-23, FR-20 · **권한** 로그인 + 권한 account.manage · **성공** 200 · **구현 위치** `routes/admin.js:78`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원(계정) ID · 예: 9

Response 예시 **(200)**

```
{
  "success": true,
  "data": {
    "id": 9,
    "username": "soldier2",
    "name": "Joey Marino",
    "role": "SOLDIER",
    "status": "ACTIVE",
    "isLocked": false,
    "lockedReason": null,
    "failedLogins": 0,
    "lastLoginAt": "2026-10-18T09:00:00.000Z",
    "tokenVersion": 0
  }
}
```


오류 응답: 400, 401, 403, 404, 422

API-209 POST /api/admin/accounts/:id/reset-password

비밀번호 초기화 (임시 비밀번호 발급, 기존 세션 만료)

요구사항 FR-23, FR-20 · **권한** 로그인 + 권한 account.manage · **성공** 200 · **구현 위치** `routes/admin.js:92`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원(계정) ID · 예: 9

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": 9,
    "username": "soldier2",
    "temporaryPassword": "Tmp-8f2a9c1d",
    "sessionsRevoked": true
  }
}
```


오류 응답: 400, 401, 403, 404

API-210 POST /api/admin/accounts/:id/force-logout

강제 로그아웃 (해당 계정의 모든 발급 토큰 무효화)

요구사항 FR-23, NFR-03 · **권한** 로그인 + 권한 account.manage · **성공** 200 · **구현 위치** `routes/admin.js:108`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원(계정) ID · 예: 9

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "id": 9,
    "username": "soldier2",
    "tokenVersion": 2,
    "sessionsRevoked": true
  }
}
```


오류 응답: 400, 401, 403, 404

API-211 PATCH `/api/admin/accounts/:id/username`

로그인 아이디 변경

요구사항 FR-23, FR-01 · **권한** 로그인 + 권한 account.manage · **성공** 200 · **구현 위치** `routes/admin.js:122`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원(계정) ID · 예: 9

Body 필드	타입	필수	설명 / 제약
`username`	string	Y	최대 30자

Request 예시

```
{
  "username": "joey_m"
}
```

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "id": 9,
    "username": "joey_m",
    "name": "Joey Marino",
    "role": "SOLDIER",
    "status": "ACTIVE",
    "isLocked": false,
    "lockedReason": null,
    "failedLogins": 0,
    "lastLoginAt": "2026-10-
18T09:00:00.000Z",
    "tokenVersion": 0
  }
}
```


오류 응답: 400, 401, 403, 404, 409

API-212 GET `/api/admin/accounts/:id/login-history`

특정 계정의 로그인 이력 (성공/실패, IP)

요구사항 FR-23, FR-20 · **권한** 로그인 + 권한 account.manage · **성공** 200 · **구현 위치** `routes/admin.js:137`

위치	이름	타입	필수	설명 / 제약
path	`id`	integer	Y	구성원(계정) ID · 예: 9
query	`result`	string		값: SUCCESS | FAIL

query	`page`	integer		페이지 번호 (기본 1)
query	`size`	integer		페이지 크기 (기본 20, 최대 100)
query	`sort`	string		정렬: at (내림차순 '-', 기본 -at)

Response 예시 (200)

```
{
  "success": true,
  "data": [
    {
      "id": "671d...",
      "memberId": 9,
      "username": "soldier2",
      "event": "LOGIN",
      "result": "FAIL",
      "reason": "BAD_PASSWORD",
      "ip": "::1",
      "at": "2026-10-18T09:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "size": 20,
    "total": 1,
    "totalPages": 1
  }
}
```

오류 응답: 400, 401, 403, 404

API-213 GET `/api/admin/security/failed-logins`

로그인 실패 집계 (계정·IP 별, 의심 계정 탐지)

요구사항 FR-23, FR-22 · **권한** 로그인 + 권한 account.manage · **성공** 200 · **구현 위치** `routes/admin.js:154`

위치	이름	타입	필수	설명 / 제약
query	`hours`	integer		최근 N시간 · ≥1, ≤720, 기본 24
query	`threshold`	integer		의심 판단 실패 횟수 · ≥1, 기본 3

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "since": "2026-10-17T09:00:00.000Z",
    "byAccount": [
      {
        "username": "soldier2",
        "failures": 4,
        "lastAt": "2026-10-18T08:00:00.000Z",
        "suspicious": true
      }
    ],
    "byIp": [
      {
        "ip": "::1",
        "failures": 4
      }
    ]
  }
}
```


오류 응답: 400, 401, 403

API-214 GET `/api/admin/settings`

시스템 운영 설정 조회

요구사항 FR-23 · **권한** 로그인 + 권한 system.manage · **성공** 200 · **구현 위치** `routes/admin.js:175`

Response 예시 (200)


```
{
  "success": true,
  "data": {
    "evaluationEditWindowHours": 72,
    "maxActiveAssignmentsPerMember": 5,
    "loginLockThreshold": 5,
    "resourceRequestAutoExpireHours": 48
  }
}
```


오류 응답: 401, 403

API-215 PATCH `/api/admin/settings`

시스템 운영 설정 변경 (평가 수정 가능 시간, 1인 최대 동시 업무 수, 로그인 잠금 임계값)

요구사항 FR-23, FR-20 · **권한** 로그인 + 권한 system.manage · **성공** 200 · **구현 위치** `routes/admin.js:181`

Body 필드	타입	필수	설명 / 제약
`evaluationEditWindowHours`	integer		≥0, ≤720
`maxActiveAssignmentsPerMember`	integer		≥1, ≤50
`loginLockThreshold`	integer		≥3, ≤20
`resourceRequestAutoExpireHours`	integer		≥1, ≤720

부분 수정: 변경할 필드만 1개 이상 전달

Request 예시

```
{
  "evaluationEditWindowHours": 48,
  "maxActiveAssignmentsPerMember": 5,
  "loginLockThreshold": 5,
  "resourceRequestAutoExpireHours": 48
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "evaluationEditWindowHours": 48,
    "maxActiveAssignmentsPerMember": 5,
    "loginLockThreshold": 5,
    "resourceRequestAutoExpireHours": 48
  }
}
```

오류 응답: 400, 401, 403

API-216 GET `/api/admin/system/info`

시스템 정보 (DB 이름, 버전, 테이블/컬렉션별 데이터 건수)

요구사항 FR-23, DR-05, DR-06 · **권한** 로그인 + 권한 system.manage · **성공** 200 · **구현 위치** `routes/admin.js:198`

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "studentId": "202600000",
    "node": "v20.18.0",
    "mysql": {
      "database": "family_202600000",
      "version": "8.0.46",
      "tables": {
        "members": 19
      }
    },
    "mongo": {
      "database": "family_log_202600000",
      "collections": {
        "audit_logs": 42
      }
    },
    "apiCount": 230
  }
}
```

오류 응답: 401, 403

API-217 GET `/api/admin/integrity-check`

업무 규칙 무결성 점검 (BR-01/03/07/08 위반 데이터 탐지 + 운영 참고 정보)

저장된 데이터 전체를 대상으로 업무 규칙 위반 여부를 점검한다. 정상 운영 시 모든 항목이 0건이어야 한다.

요구사항 FR-23, BR-01, BR-03, BR-07, BR-08, DR-01 · **권한** 로그인 + 권한 system.manage · **성공** 200 ·

구현 위치 `routes/admin.js:218`

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "ok": true,
    "checks": {
      "BR-01 multipleFormalCrews": [],
      "BR-03 overlappingAssignments": [],
      "BR-07 overlappingAllocations": [],
      "BR-08 blockedResourcesWithReservations": [],
      "info: inactiveMembersWithOpenAssignments": [],
      "info: crewsWithoutCapo": []
    }
  }
}
```

오류 응답: 401, 403

API-218 POST `/api/admin/system/reset-demo`

데모 데이터 초기화 (테스트 재현용: 모든 데이터를 초기 상태로)

채점/테스트 재현을 위해 MySQL·MongoDB 데이터를 초기 데모 상태로 되돌린다. 실행 후 모든 사용자는 다시 로그인해야 한다.

요구사항 FR-23, DR-05 · **권한** 로그인 + 권한 system.manage · **성공** 200 · **구현 위치** `routes/admin.js:249`

Body 필드	타입	필수	설명 / 제약
`confirm`	string	Y	확인 문구 RESET · 값: RESET

Request 예시

```
{
  "confirm": "RESET"
}
```

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "reset": true,
    "at": "2026-10-18T09:00:00.000Z"
  }
}
```

오류 응답: 400, 401, 403

API-219 POST `/api/admin/broadcast`

전체/대상 공지 알림 발송 (역할·Crew 필터)

요구사항 FR-23 · **권한** 로그인 + 권한 system.manage · **성공** 201 · **구현 위치** `routes/admin.js:263`

Body 필드	타입	필수	설명 / 제약
`title`	string	Y	최대 100자
`message`	string	Y	최대 1000자
`roles`	array<string>		값: ASSOCIATE | SOLDIER | CAPO | UNDERBOSS | BOSS | ADMINISTRATOR
`crewId`	integer		

Request 예시

```
{
  "title": "시스템 점검 안내",
  "message": "오늘 22시~23시 시스템 점검이 있습
니다."
}
```

Response 예시 (201)

```
{
  "success": true,
  "data": {
    "recipients": 15
  }
}
```

오류 응답: 400, 401, 403

27. System

API-220 GET /api/health

서버 및 DB 연결 상태 확인 (공개)

요구사항 NFR-01, NFR-02, CR-06 · **권한** 공개 · **성공** 200 · **구현 위치** `routes/admin.js:287`

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "status": "UP",
    "mysql": "UP",
    "mongo": "UP",
    "time": "2026-10-18T09:00:00.000Z"
  }
}
```

오류 응답:

API-221 GET /api/api-catalog

API 카탈로그 (전체 API 와 연결된 요구사항 - 요구사항 추적용, 공개)

요구사항 AR-06, AR-08, CR-04 · **권한** 공개 · **성공** 200 · **구현 위치** `routes/admin.js:297`

위치	이름	타입	필수	설명 / 제약
query	`req`	string		요구사항 ID 로 필터 (예: BR-03) · 예: "BR-03"
query	`tag`	string		

Response 예시 (200)

```
{
  "success": true,
  "data": {
    "total": 230,
    "items": [
      {
        "id": "API-001",
        "method": "POST",
        "path": "/api/auth/register",
        "summary": "회원 가입 신청",
        "reqs": [
          "FR-01",
          "FR-04"
        ]
      }
    ]
  }
}
```


오류 응답: 400